Master of Science in Computer Science
January 2026

Benchmarking Machine Learning Techniques For A Generative AI Troubleshooting Assistant In Automotive Diagnostics

Gazi Samia Ahmed

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfilment of the requirements for the degree of Master of Science in Computer Science. The thesis is equivalent to 20 weeks of full time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:
Author(s):
Gazi Samia Ahmed
E-mail: gaah17@student.bth.se

University advisor:
Dr. Milena Angelova
Department of Computer Science

Faculty of Computing
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

ABSTRACT

Background: Modern automotive workshops, production, and aftermarket operations increasingly depend on advanced off-board Diagnostic Communication Platform (DCP) to perform Electronic Control Unit (ECU) flashing, identify root causes, and guide repairs. Meanwhile, Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) enable the creation of troubleshooting assistants that analyse logs and use existing diagnostic knowledge. That said, it is not yet clear which machine learning techniques and configurations provide accurate and reliable support in this specialised setting.

Objectives: The thesis benchmarks machine learning techniques for a generative AI-based troubleshooting assistant within an automotive diagnostic communication platform. Two research questions are addressed in this study: (i) which state-of-the-art techniques are applicable when using diagnostic knowledge bases and tool usage logs, and (ii) how these techniques compare for retrieval effectiveness and response quality.

Methods: We focus on two primary tasks: identifying root causes from Unified Diagnostic Services (UDS) Negative Response Codes (NRCs) and performing question-answering using resolved incidents. It integrates domain expertise through layout-aware parsing of ISO 14229-1 NRC tables and structured incident descriptions that include problem, symptoms, root cause, and solution. A Structured and Domain-Adaptive RAG retriever, we combine different query representations and a cross-encoder reranking. The evaluation includes two base models - TinyLlama and an OpenAI model - and we evaluate in three configurations: plain or baseline LLM, LLM with Structured Domain-Adaptive RAG, and LLM with Structured Domain-Adaptive RAG and reranking. Retrieval effectiveness we measure using Top-k accuracy, and response quality is measured using cosine similarity, ROUGE-L, BLEU, Jaccard, BERTScore, and statistical tests such as one-way ANOVA and Tukey's HSD.

Results: In the root-cause analysis, the highest Top-10 retrieval accuracy of 66.15% we obtain when NRC entries are encoded using concise semantic fields such as service name, service ID, NRC, and error type. More formal byte-level or key-value encodings underperform. For question-answering, applying structured query expansion to resolved incidents results in over 96% accuracy for Top-5 retrieval, with reranking primarily improving the top-ranked results. Regarding TinyLlama, Structured Domain-Adaptive RAG and reranking significantly enhance both lexical and semantic similarity to synthetic target values. For the OpenAI model, Structured RAG consistently increases lexical overlap and semantic coverage, while additional reranking has a limited impact. ANOVA analysis rejects the null hypothesis of equal performance across all the metrics for OpenAI model.

Conclusions: The thesis suggests that the most statistically significant decisions for a generative AI troubleshooting assistant are Structured Domain-Adaptive RAG and query expansion, rather than relying solely on the baseline LLMs. Smaller local models can perform well when combined with strong retrieval and reranking techniques, while larger, hosted models mainly benefit from structured grounding. These results offer practical guidance for design and an evaluation framework for future research on incorporating generative AI troubleshooting assistants into diagnostic communication platforms. This thesis also emphasises the importance of using real workshop, production and aftermarket data, along with human evaluation methods.

Keywords: Automotive diagnostics, root cause analysis, troubleshooting assistant, retrieval-augmented generation (RAG), large language models (LLMs)

ACKNOWLEDGEMENTS

This thesis was carried out at a heavy truck and bus manufacturing company in Sweden, in collaboration with the Department of Computer Science at Blekinge Institute of Technology. I sincerely thank my academic advisor, Dr Milena Angelova, for her valuable guidance and support throughout this work. I am also very grateful to my industrial advisor, Divyansh Khunteta, for his continuous support, positivity, and motivation, and to my group manager, Rezvan Orangpoor, for believing in me and supporting this thesis. This work would not have been possible without the expertise and dedication of Dr Milena Angelova and Divyansh Khunteta; I truly appreciate their contributions. I am also deeply thankful to my previous group manager, Patrik Lorbeck, for considering my interest in pursuing the thesis at the company. Additionally, I want to thank my colleagues for their cooperation, helpful discussions, and support throughout this work journey.

I would also like to express my heartfelt gratitude to my husband and son for their unwavering support, patience, and encouragement throughout this journey. My heartfelt gratitude goes to my parents and family for their constant love, motivation, and belief in me, even from a distance. I also appreciate my friends for their support, understanding, and encouragement during the challenging periods of this work.

During the preparation of this work, I used Grammarly for proofreading and improving readability. Afterwards, I reviewed and revised the content myself and took full responsibility for the final version.

Gazi Samia Ahmed, Stockholm, January 2026

CONTENTS

ABSTRACT ..... **II**

ACKNOWLEDGEMENTS ..... **III**

CONTENTS ..... **IV**

LIST OF TABLES ..... **VI**

LIST OF FIGURES ..... **VII**

ACRONYMS ..... **VIII**

1 INTRODUCTION **1**

1.1 PROBLEM STATEMENT 1
1.2 AIMS AND OBJECTIVES 2
1.3 RESEARCH QUESTIONS 2
1.4 SCOPE OF THE THESIS 3
1.4.1 Scope of the Thesis *3*
1.4.2 Out of Scope *3*
1.5 OUTLINE 3

2 BACKGROUND **4**

2.1 WHY ARE LLMS AND RAG RELEVANT TO AUTOMOTIVE TROUBLESHOOTING? 4
2.2 RELIABILITY GAPS TO ACHIEVE ACCURATE ANSWERS? 4
2.3 WHY IS BENCHMARKING AND EVALUATION NECESSARY TO MEASURE PERFORMANCE? 5
2.4 FUNDAMENTAL CONCEPTS AND TERMINOLOGY 5
2.4.1 Automotive Domain-Specific Terminologies *5*
2.4.2 Technical Terminologies *7*

3 RELATED WORK **8**

3.1 LLM AND RAG IN AUTOMOTIVE TROUBLESHOOTING 8
3.2 IMPROVING RAG PIPELINES FOR HIGHER RESPONSE ACCURACY 8
3.3 KNOWLEDGE-GRAPH AND GRAPH-BASED RAG FOR STRUCTURED TECHNICAL KNOWLEDGE 8
3.4 DOMAIN ADAPTATION AND SELF-IMPROVING RAG 9
3.5 BENCHMARKING AND EVALUATION FRAMEWORKS FOR LLM AND RAG TROUBLESHOOTING ASSISTANTS 9
3.6 HALLUCINATION AND FAITHFULNESS IN TROUBLESHOOTING ASSISTANTS 10
3.7 IDENTIFIED RESEARCH GAPS 10
3.8 SURVEY OF PUBLISHED ACADEMIC LITERATURE TO ADDRESS RQ1 10
3.8.1 Survey to Address RQ1, LLMs used *10*
3.8.2 Survey to Address RQ1, LLMs and RAG *12*
3.8.3 Survey to Address RQ1, LLMs and Structured RAG *16*
3.8.4 Survey to Address RQ1, LLMs and Domain-Adaptive RAG *18*

4 METHOD **20**

4.1 DEFINING RESEARCH QUESTIONS 20
4.2 RESEARCH METHODS 20
4.2.1 Literature Survey to Address RQ1 ..... *20*
4.2.2 Benchmarking Experiment to Address RQ2 ..... *21*

5 RESULTS AND ANALYSIS **28**

5.1 DESCRIPTIVE STATISTICS OF THE DATASET 28
5.1.1 Populations *28*
5.1.2 Samples *28*
5.1.3 Data Structure *28*
5.2 RETRIEVAL QUALITY EVALUATION OF DOMAIN-ADAPTIVE AND STRUCTURED RAG FOR ROOT-CAUSE ANALYSIS 28

5.2.1 Evaluation of Baseline, Query Expansion and Reranking ... *29*
5.2.2 Evaluation with More Query Expansion and Reranking ... *30*
5.2.3 Evaluation with Even More Query Expansion and Reranking ... *31*
5.3 Retrieval Quality Evaluation of Structured RAG for Question-Answering ... 32
5.4 Similarity Evaluation - Retrieval of Root-Cause Analysis ... 34
5.5 Similarity Evaluation - TinyLlama for Question-Answering ... 36
5.5.1 TinyLlama without RAG ... *37*
5.5.2 TinyLlama with Structured RAG ... *39*
5.5.3 TinyLlama with Structured RAG and Reranking ... *41*
5.5.4 Hypothesis Testing ... *44*
5.6 Similarity Evaluation - OpenAI for Question-Answering ... 45
5.6.1 OpenAI without RAG ... *45*
5.6.2 OpenAI with Structured RAG ... *47*
5.6.3 OpenAI with Structured RAG and Reranking ... *49*
5.6.4 Hypothesis Testing ... *51*
5.7 Benchmarking Highlights ... 52
5.7.1 Benchmarking Highlights on Retrieval Quality Evaluation ... *52*
5.7.2 Benchmarking Highlights on Similarity Evaluation of Question-Answering ... *53*
5.8 Cost Effectiveness ... 54
5.8.1 Cost Structure ... *54*
5.8.2 Latency Analysis ... *54*
5.8.3 Throughput Analysis ... *55*
5.8.4 Latency and Throughput Comparison - OpenAI vs TinyLlama ... *56*
5.8.5 Cost Analysis ... *57*
5.8.6 Cost Comparison - OpenAI vs TinyLlama ... *57*
5.8.7 Cost Comparison for Vector Store Choices ... *58*

6 DISCUSSION ... 59

6.1 Addressing RQ1: State-of-the-art Techniques and Their Applicability ... 59
6.2 Addressing RQ2: Comparative Performance and Hypothesis Testing ... 63
6.3 Patterns and Unexpected Findings ... 66
6.4 Limitations ... 66
6.4.1 Data Availability and Quality ... *66*
6.4.2 Evaluation Design ... *66*
6.4.3 Model and Compute Constraints ... *66*
6.4.4 Retrieval and Document Processing ... *67*
6.4.5 Organisational and Environmental Constraints ... *67*
6.5 Impact on Validity and Reliability ... 67
6.5.1 External Validity ... *67*
6.5.2 Internal Validity ... *67*
6.5.3 Construct Validity ... *67*
6.6 Implications for Research and Practice ... 68
6.6.1 Implications for Research ... *68*
6.6.2 Implications for Practice ... *68*
6.7 Ethical, Societal, and Sustainability Aspects ... 69
6.7.1 Ethical Aspect ... *69*
6.7.2 Societal Aspect ... *69*
6.7.3 Sustainability Aspect ... *69*

7 CONCLUSION AND FUTURE WORK ... 70

7.1 Future Work ... 70

8 REFERENCES ... 71

9 APPENDIX ... 2

LIST OF TABLES

Table 3.1: Survey to address RQ1, LLMs used 12
Table 3.2: Survey to address RQ1, LLMs and RAG 16
Table 3.3: Survey to address RQ1, LLMs and Structured RAG 17
Table 3.4: Survey to address RQ1, LLMs and Domain-Adaptive RAG 18
Table 4.1: Device specifications 21
Table 4.2: Technologies used 22
Table 4.3: Evaluation metrics 25
Table 5.1: Top-10 retrieval evaluation with query expansion and reranking 29
Table 5.2: Top-10 retrieval evaluation with more query expansion and reranking 31
Table 5.3: Top-10 retrieval evaluation with even more query expansion and reranking 32
Table 5.4: Top-5 retrieval evaluation with query expansion and reranking 33
Table 5.5: Descriptive statistics of similarity metrics for retrieved contexts of root cause 34
Table 5.6: Descriptive statistics of similarity metrics for responses generated by TinyLlama without RAG 37
Table 5.7: Descriptive statistics of similarity metrics for responses generated by TinyLlama with Structured RAG 39
Table 5.8: Descriptive statistics of similarity metrics for responses generated by TinyLlama with Structured RAG and Reranking 42
Table 5.9: One-Way ANOVA results across Plain TinyLlama, Structured RAG-based TinyLlama, Structured RAG-based TinyLlama with Reranking 44
Table 5.10: Descriptive statistics of similarity metrics for responses generated by OpenAI 45
Table 5.11: Descriptive statistics of similarity metrics for responses generated by OpenAI with Structured RAG 48
Table 5.12: Descriptive statistics of similarity metrics for responses generated by OpenAI with Structured RAG and Reranking 50
Table 5.13: One-Way ANOVA results across Plain OpenAI, Structured RAG-based OpenAI, Structured RAG-based OpenAI with Reranking 52
Table 5.14: Benchmarking highlights on retrieval quality evaluation 53
Table 5.15: Benchmarking highlights on similarity evaluation of question-answering 53
Table 5.16: Descriptive statistics for OpenAI response generation time (seconds) 55
Table 5.17: Descriptive statistics for TinyLlama response generation time (seconds) 55
Table 5.18: Descriptive statistics for token usage in OpenAI 56
Table 5.19: Descriptive statistics for token usage in TinyLlama 56
Table 5.20: Latency and throughput comparison (tokens/sec) 56
Table 5.21: Descriptive statistics for OpenAI cost per query (USD) 57
Table 5.22: Cost Summary comparison between OpenAI and TinyLlama 57
Table 5.23: Cost comparison for vector store choices (RAG integration) 58
Table 6.1: Identified techniques in the survey to address RQ1 59
Table 6.2: Techniques considered for empirical study 60
Table 6.3: Results in RQ2 64
Table 9.1: Top-20 retrieval evaluation with query expansion and reranking (Sub-section 5.2.1) 2
Table 9.2: Top-20 retrieval evaluation with more query expansion and reranking (Sub-section 5.2.2) . 2
Table 9.3: Top-20 retrieval evaluation with more query expansion and reranking (Sub-section 5.2.3) . 3
Table 9.4: Top-10 retrieval evaluation with query expansion and reranking (Sub-section 5.3) 3
Table 9.5: Tukey's HSD post-hoc test results across Plain OpenAI, Structured RAG-based OpenAI, Reranked Structured RAG-based OpenAI (Sub-sections 5.6.4) 5

LIST OF FIGURES

Figure 1.1: Problem to address within an automotive off-board diagnostic communication platform ... 2
Figure 4.1: Supported NRC table structure in the ISO 14229-1 ... 23
Figure 4.2: Feature extraction for RAG ... 24
Figure 4.3: Pipeline for LLM without RAG ... 25
Figure 4.4: Pipeline for LLM with RAG ... 26
Figure 4.5: Pipeline for LLM with RAG and Reranking ... 27
Figure 5.1: Distribution of similarity metrics for retrieved contexts of root cause ... 36
Figure 5.2: Distribution of similarity metrics for responses generated by TinyLlama without RAG ... 39
Figure 5.3: Distribution of similarity metrics for responses generated by TinyLlama with RAG ... 41
Figure 5.4: Distribution of similarity metrics for responses generated by TinyLlama with RAG and Reranking ... 44
Figure 5.5: Distribution of similarity metrics for responses generated by OpenAI without RAG ... 47
Figure 5.6: Distribution of similarity metrics for responses generated by OpenAI with RAG ... 49
Figure 5.7: Distribution of similarity metrics for responses generated by OpenAI with RAG and Reranking ... 51
Figure 6.1: Proposed an architecture of Generative AI-powered troubleshooting assistant using AWS for deployment ... 68

ACRONYMS

ACC	Accuracy
AI	Artificial Intelligence
ANN	Approximate Nearest Neighbour
ANOVA	Analysis Of Variance
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy
DCP	Diagnostic Communication Platform
DL	Deep Learning
ECU	Electronic Control Unit
EM	Exact Match
EXAI/XAI	Explainable AI
HSD	Tukey's Honestly Significant Difference
ISO	International Organization for Standardization
KB	Knowledge Base
KG	Knowledge Graph
LLM	Large Language Model
ML	Machine Learning
MNED	Mean Normalized Edit Distance
nDCG	Normalized Discounted Cumulative Gain
NLG	Natural Language Generation
NLI	Natural Language Inference
NLP	Natural Language Processing
NN	Neural Network
NRC	Negative Response Code
OCR	Optical Character Recognition
QA	Question-Answer
RAG	Retrieval-Augmented Generation
RAGAS	Retrieval-Augmented Generation Assessment Suite
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
TLM	Tiny Language Model
UDS	Unified Diagnostic Service
UI	User Interface

1 INTRODUCTION

Modern vehicles are increasingly software-defined, and their ECUs rely on complex diagnostic communication for tasks such as software updates, root cause identification, and incident handling. In practice, workshops, production, and the aftermarket use off-board diagnostic toolchains and communication platforms to interpret UDS messages, analyse NRCs, and determine appropriate repair actions. As the number of ECUs, software versions, and diagnostic procedures increases, it becomes challenging for technicians to integrate raw log data, ISO standards, and historical incidents into a consistent troubleshooting strategy.

Recent advances in LLMs and RAG enable the development of troubleshooting assistants that can understand natural-language queries, interpret diagnostic messages, and access relevant information from existing knowledge bases to support technicians with ECU flashing and root-cause analysis. That said, simply connecting a generic LLM to workshop data is insufficient. Without careful design, LLMs might hallucinate, overlook domain knowledge, or offer incomplete recommendations. As a result, it is not yet clear which specific machine learning techniques and configurations are best suited for this specialised application, or how they compare systematically to benchmarks.

We address these challenges by benchmarking machine learning techniques for a generative AI troubleshooting assistant within an automotive diagnostic communication platform. We focus on two core capabilities of the assistant: first, root-cause analysis using UDS NRCs and ISO 14229-1 standards; second, question-answering using a structured knowledge base of resolved ECU flashing incidents. A key insight throughout the thesis is that the performance of the generative assistant's responses, such as accuracy, latency, and cost, depends not only on the base LLM but also on how domain knowledge is structured, retrieved, and supplied to the model, a point revisited later in the discussion and conclusion.

1.1 Problem Statement

In the modern automotive industry, workshops, production, and aftermarket often need support for vehicle ECU flashing or software updates due to a range of issues. As vehicles become increasingly software-dependent, the need for efficient and accurate software updates and troubleshooting grows. These processes are frequently slowed by unexpected problems that require expert-level assistance at the right time. Delays in responding to user queries and fixing issues can lead to substantial costs from system outages or production downtime.

Currently, support teams handle user queries via email, chat, and other channels, using a manual problem-solving approach. Relevant information is often scattered across multiple sources, such as extensive documentation, tool logs, and email threads. However, this data is frequently unstructured, making it difficult to access quickly and delaying solutions to common or recurring problems during troubleshooting on the diagnostic communication platform. While AI tools and health monitoring systems in the automotive industry primarily focus on sensor data and structured vehicle telemetry, they often overlook valuable insights in textual descriptions, user reports, and support team feedback. This represents a notable gap, particularly in workshops, production, and aftermarket support, where textual problem descriptions and logs offer rich, actionable information.

The thesis addresses the challenge of using diagnostic logs for both failure and non-failure cases, and of answering user queries using textual problem descriptions to produce automated, relevant responses during ECU flashing for automotive diagnostic troubleshooting on an off-board communication platform. It specifically examines failures occurring in the off-board diagnostic communication platform during software flashing to ECUs with diagnostic tools, aiming to address these issues by using knowledge bases and logs within the automotive diagnostic environment. When the software on this platform has defects or deviations between expected and actual outputs during ECU flashing, it can cause failures or other problems.

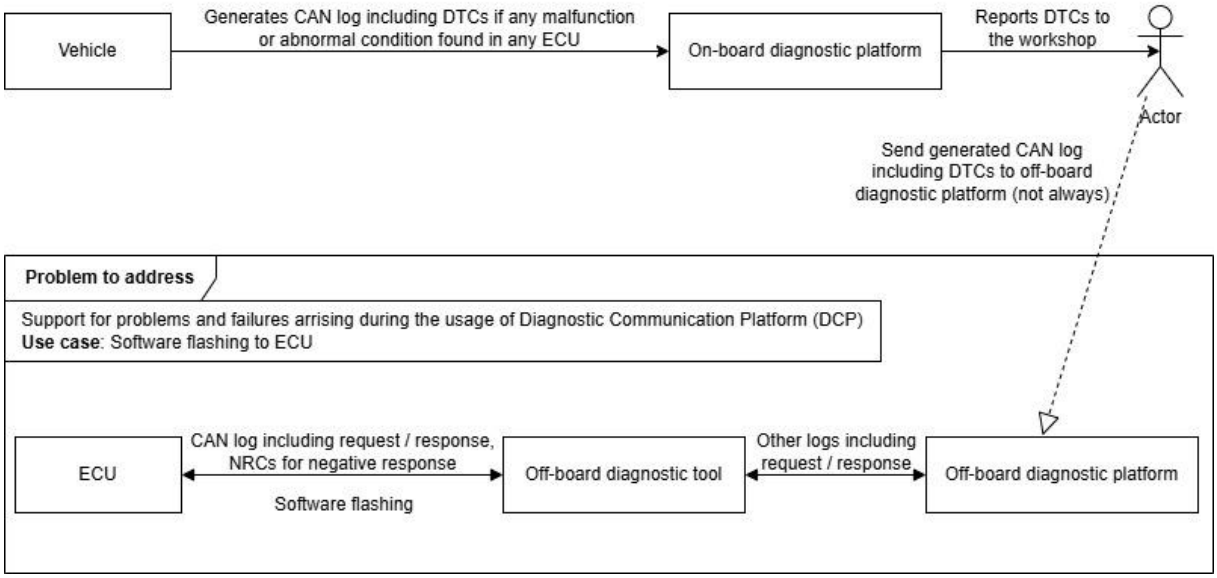


Figure 1.1: Problem to address within an automotive off-board diagnostic communication platform

1.2 Aims and Objectives

The thesis aims to benchmark machine learning techniques that produce the most accurate responses to user queries for a generative AI troubleshooting assistant, using tool usage logs for both failure and non-failure cases, and knowledge base data, including ISO standards and resolved incidents, within an automotive off-board diagnostic communication platform.

To achieve this aim, the following objectives are formulated:

Objective 1: To identify the state-of-the-art machine learning techniques to generate accurate responses in a generative AI-powered troubleshooting assistant for user queries, considering knowledge base data and tool usage logs within the automotive off-board diagnostic communication platform domain.

Objective 2: To evaluate and compare the performance of the identified state-of-the-art machine learning techniques, focusing on their ability to generate accurate responses to user queries by utilizing knowledge base data and tool usage logs, including both failure and non-failure cases, for a generative AI-powered troubleshooting assistant within the automotive off-board diagnostic communication platform domain.

1.3 Research Questions

The research questions are formulated as follows to achieve the objectives of the thesis:

RQ1: What are the state-of-the-art machine learning techniques used to provide accurate responses in a generative AI-powered troubleshooting assistant to user queries, considering the knowledge base data and tool usage logs within the automotive off-board diagnostic communication platform? Which techniques can be applied among the techniques identified?

RQ2: How do different machine learning techniques, identified in RQ1, compare in generating accurate responses to user queries in terms of performance by utilizing knowledge base data and tool usage logs, including both failure and non-failure cases, for a generative AI-powered troubleshooting assistant within the automotive off-board diagnostic communication platform?

To address RQ2, the following hypotheses are formulated:

H0 (null hypothesis): There is no significant difference in performance among the machine learning techniques identified in RQ1 for achieving the most accurate responses.

H1 (alternative hypothesis): There is a significant difference in performance among the machine learning techniques identified in RQ1 for achieving the most accurate responses.

1.4 Scope of the Thesis

1.4.1 Scope of the Thesis

We focus on benchmarking of NLP components essential for a RAG-based troubleshooting assistant in automotive diagnostics. It uses UDS logs in accordance with ISO 14229-1 and incorporates knowledge sources to evaluate how effectively the pipeline retrieves relevant information and delivers reliable diagnostic solutions. The research investigates sentence-similarity techniques to explicitly extract retrieval features, re-ranking methods to enhance retrieval accuracy and minimise hallucinations, question-answering systems for interactive diagnostics, and RAG-based root-cause analysis. The root-cause analysis involves a simplified, template-driven interpretation of UDS logs aligned with ISO 14229-1, followed by a natural-language explanation that clearly references the retrieved evidence and log data.

1.4.2 Out of Scope

We do not aim to develop a general-purpose language agent or address the full range of NLP and machine learning challenges related to automotive diagnostic data. NLP tasks, such as log summarisation, translation, dialogue management, content creation, personalisation, and broader language understanding, are intentionally excluded. The thesis also excludes capabilities that require extensive reasoning beyond the retrieved evidence, such as math-intensive reasoning or code generation. It avoids standalone examinations of ethics, safety, bias, and toxicity. Regarding machine learning, the thesis does not include anomaly detection, pattern recognition, or classification tasks, such as intent recognition, success or failure prediction, or the prediction of UDS message or error types (e.g., NRC or service name) from raw log data. Sentiment analysis is also outside its scope.

1.5 Outline

The remainder of this thesis is organised to progress from context and research grounding to method, evaluation, and conclusions:

- **Chapter 2 - Background**: Covers the relevant research background and defines fundamental concepts and terminology used throughout the thesis.
- **Chapter 3 - Related Work**: Summarises relevant research and identifies the research gap addressed by this thesis.
- **Chapter 4 - Method**: Explains how RQ1 (literature survey) and RQ2 (benchmarking experiment) are conducted in this thesis.
- **Chapter 5 - Results and Analysis**: Presents and analyses findings from the survey and experiments.
- **Chapter 6 - Discussion**: Interprets results, discusses limitations and implications, and addresses ethical, societal, and sustainability aspects, including validity considerations.
- **Chapter 7 - Conclusion and Future Work**: Concludes the thesis and proposes future research directions.

2 BACKGROUND

2.1 Why are LLMs and RAG relevant to automotive troubleshooting?

Modern vehicle troubleshooting and software update workflows are more complex and time-sensitive. A recent study argues that LLMs can support this process by enabling natural-language interaction with diagnostic knowledge and providing assistance in interpretation and guidance during automotive diagnostics and troubleshooting [1]. The same research also highlights practical constraints, such as accuracy and reliability issues, that need to be addressed before LLMs can be confidently integrated into real diagnostics scenario [1]. Off-board diagnostics and ECU flashing face notable challenges in the industrial sector. The challenges go beyond the vehicle, as the off-board diagnostic platform's reliability, covering configuration, timing, compatibility, and software quality, significantly influences success.

A study emphasises that LLMs can generate helpful natural-language answers from textual problem descriptions, but alone are not reliable enough for "knowledge-intensive" technical domains [2]. A key challenge is hallucination, which involves confident but incorrect model outputs [1], [2]. In diagnostic assistants, incorrect troubleshooting advice can mislead users, delay workshops, production, and aftermarket troubleshooting, and weaken trust in the toolchain. To minimise hallucinations and enhance factual accuracy, RAG has become a popular architecture [3], where the system retrieves relevant documents, such as technical manuals, error-code descriptions, and related conditions. In a troubleshooting-focused RAG workflow, it is crucial to prepare both the knowledge documents and the user query [4]. Embedding-based retrieval enables matching user queries with contextually relevant documents, especially when the assistant needs to infer information beyond basic keyword overlap [4]. A study highlights that RAG is an effective method for providing LLMs with specific context, but this approach needs further evaluation and improvement, and outcomes depend heavily on query quality and structure [1]. In automotive diagnostics, knowledge is spread across diverse sources and is frequently updated, so RAG is essential in this context.

2.2 Reliability gaps to achieve accurate answers?

Research on knowledge-graph-based RAG suggests that RAG systems can still deliver inaccurate answers even when the required factual knowledge is present [5]. This often results from retrieval and reasoning errors, especially with complex questions [5]. Thus, it is crucial to carefully evaluate each component of the pipeline individually, rather than considering RAG as a single, unified approach [5]. This is directly related to diagnostic communication platform queries, where user questions often lack context, describe ambiguous symptoms, and suggest constraints, such as ECU type, flashing step, or session state. These observations motivate this thesis, which benchmarks state-of-the-art ML techniques, such as LLMs, RAG variants, and grounding/evaluation strategies to generate accurate, evidence-supported troubleshooting responses using knowledge bases and tool usage logs within an automotive diagnostic communication platform.

Also, automotive adoption faces constraints related to confidentiality and practical deployment [6]. Engineering sectors often require access to sensitive documents, such as technical details, internal standards, and trade secrets, which increases the risks associated with cloud-hosted models [6]. Alternatively, local deployment can be used, but it can be expensive and may require smaller models that might not always deliver sufficient performance [6]. RAG is essential in our context because accurate responses often rely on context, including private internal documents, standards, and tool-specific behaviours that are not included in public training datasets.

2.3 Why is benchmarking and evaluation necessary to measure performance?

Since system quality relies on several interconnected components, such as retrieval, prompting, and generation, benchmarking and evaluation are crucial for building a reliable troubleshooting assistant. A survey on LLM evaluation highlights that assessment generally involves (i) automatic metrics, (ii) model-based judging, and (iii) human evaluation protocols, with the selected method depending on the task type and possible failure modes [7]. In RAG-focused benchmarks, CRAG's human evaluation guidelines label major hallucinations as incorrect responses and use graded accuracy scales to differentiate between missing, inaccurate, acceptable, and accurate answers [8].

An automotive industry case study on PDF chatbot deployment focuses on enhancing RAG techniques for complex automotive documents [9]. These include multi-column layouts and technical specifications [9]. The study proposes optimising PDF processing, retrieval strategies, and context compression [9]. The study evaluates an optimised RAG model and a self-RAG agent compared to a baseline, showing improvements in retrieval and generation metrics, such as context precision and recall, answer relevance, and faithfulness [9]. These improvements are particularly notable on a dataset of automotive industry documents [9]. Similarly, an industrial assistant architecture (IDAS) describes a RAG pipeline and highlights detailed metrics, such as context precision and recall, to evaluate the quality of the generated output and how well it aligns with the retrieved information sources [10]. These metrics support the thesis goal of benchmarking techniques that deliver accurate responses using an automotive diagnostic knowledge base.

Selecting an AI troubleshooting method requires careful consideration, given the complexities of diagnostic communication and the risks of incorrect advice. Options such as baseline LLMs, RAG pipelines, reranking, domain adaptation, or log-aware retrieval can perform differently depending on query type, log quality, and documentation. It is crucial to evaluate effectiveness separately for normal and complex cases to ensure accurate diagnosis. The current literature review supports a benchmarking strategy aligned with research aims: first, selecting state-of-the-art techniques for a diagnostic communication platform (RQ1); second, testing these methods with consistent datasets and metrics focused on accuracy and evidence-based reasoning (RQ2). The goal is to establish an evidence-based foundation for deploying a trustworthy, efficient generative AI troubleshooting assistant in an automotive diagnostics platform.

2.4 Fundamental Concepts and Terminology

2.4.1 Automotive Domain-Specific Terminologies

2.4.1.1 Toolchain

In the automotive industry, a toolchain refers to a set of software tools and processes utilised in autonomous vehicles, embedded systems, and Electronic Control Units (ECUs). It encompasses software development, testing and validation, security and safety measures, connectivity and telematics, along with workshops, production, aftermarket and diagnostics procedures.

2.4.1.2 Diagnostic Communication Platform (DCP)

The Diagnostic Communication Platform is a collection of tools for vehicle communication. It facilitates interaction between engineers and service technicians with vehicle ECUs, aiding troubleshooting and diagnostics.

2.4.1.3 On-Board Diagnostics (OBD)

On-board diagnostics refer to a vehicle's self-diagnostic and reporting capability, provided by the sensors and systems installed within the vehicle, to continuously monitor the vehicle's performance. The reported errors are recorded in the error memory as DTCs. On-board vehicle diagnostic protocols are OBD/OBD2.

2.4.1.4 Off-Board Diagnostics

Off-board diagnostics require connecting external tools or software to the vehicle's electronic systems to perform in-depth analysis rather than relying on standard fault codes. Off-board vehicle diagnostics protocols are UDS, KWP, etc.

2.4.1.5 Electronic Control Unit (ECU)

An Electronic Control Unit (ECU) is an embedded system within a vehicle that controls one or more electrical systems.

Examples of ECUs in modern vehicles - Engine Control Module/Unit (ECM/ECU), Transmission Control Module/Unit (TCM/TCU), Brake Control Unit (BCU), Climate Control Unit (CCU), Navigation Control Unit (NCU), and Battery Management System (BMS) in electric vehicles as well.

2.4.1.6 International Organization for Standardization (ISO)

ISO stands for the International Organisation for Standardisation, a worldwide federation of national standards bodies.

2.4.1.7 ISO 14229

ISO 14229 defines the common requirements for diagnostic systems in road vehicles and explains how a diagnostic tester tool (client) communicates with a vehicle ECU (server) over different communication protocols, followed by https://www.iso.org/obp/ui/#iso:std:iso:14229:-3:ed-2:v1:en

2.4.1.8 Unified Diagnostic Services (UDS)

UDS is standardised under ISO 14229, and we use it to diagnose and reprogram ECUs (Electronic Control Units).

The basics of the UDS protocol are the SID (Service ID), which identifies the diagnostic function, subfunctions, or parameters, depending on the SID, and responses, including positive responses (SID + 0x40) and negative responses (starting with 0x7F).

2.4.1.9 Diagnostic Trouble Code (DTC)

DTC stands for Diagnostic Trouble Code, also known as fault codes. These codes are generated by a vehicle's on-board diagnostics system when it detects a malfunction or abnormal condition in the engine, transmission, or other systems [11].

Use case: DTC represents specific system or component failures, which helps with fault detection and classification.

2.4.1.10 Negative Response Code (NRC)

A Negative Response Code (NRC) is a byte value that provides information about why the request was rejected or failed, sent by an ECU in response to a diagnostic request message.

Use case: NRC helps with error handling for diagnostic requests to troubleshoot communication errors, not physical faults. This is incredibly helpful for identifying the root cause of issues related to diagnostic tool communication, session control, security access, or message format.

2.4.1.11 Open Systems Interconnection (OSI)

OSI stands for Open Systems Interconnection. The OSI model is a conceptual framework. It has seven layers, each independent of the others.

The layers are: physical layer (1), data link layer (2), network layer (3), transport layer (4), session layer (5), presentation layer (6), and application layer (7).

2.4.2 Technical Terminologies

2.4.2.1 Generative AI

Generative AI is a branch of AI that generates new content, such as text, audio, and video, using ML models and algorithms.

2.4.2.2 Large Language Model (LLM)

Large language models are AI systems that can understand and generate human language by analysing extensive text data [3].

2.4.2.3 Retrieval-Augmented Generation (RAG)

RAG is an architecture that improves AI model performance by connecting them with external knowledge sources. This approach enables LLMs to generate more accurate and contextually appropriate responses.

2.4.2.4 Embeddings

Numerical vector representations of text that capture semantic meaning and enable similarity-based retrieval, matching user queries to relevant knowledge base passages [3], [4].

2.4.2.5 Vector Database/Vector Store

A storage and retrieval component optimised for high-dimensional embeddings, supporting fast similarity search over knowledge base chunks [3], [12].

2.4.2.6 Retriever and Re-ranker

Many RAG systems use a two-step retrieval strategy: (1) fetch candidates using embedding similarity, then (2) apply a re-ranking model to reorder and filter candidates for higher relevance [3], [13].

2.4.2.7 Hallucination

Undesirable LLM outputs that are irrelevant, duplicated, unclear, or unsupported by the underlying source information often motivate validation steps and stronger grounding [14].

2.4.2.8 RAG Evaluation Metrics (automatic)

A commonly used set of RAG-focused metrics includes context precision (relevance of retrieved context), context recall (extent of needed evidence covered), answer relevance, and faithfulness (alignment with evidence support) [15].

2.4.2.9 Semantic Similarity

Semantic similarity measures how two pieces of text convey similar meanings, regardless of their specific wording or phrasing used. Semantic comparison involves assessing the similarity in meaning between texts, focusing on conceptual understanding rather than exact word matches. It captures deeper relationships and underlying intent beyond surface-level keywords.

2.4.2.10 Cosine Similarity

Cosine similarity measures the closeness between two embedding vectors by calculating the angle between them, reflecting their directional similarity in a high-dimensional space.

2.4.2.11 Feature Extraction

Feature extraction involves deriving important parameters from raw data to identify the most relevant and essential information for classification and further processing [11].

3 RELATED WORK

3.1 LLM and RAG in automotive troubleshooting

Dhillon and Torresin explore the use of LLMS in vehicle diagnostics, highlighting their potential to assist with fault understanding and troubleshooting [1]. They also emphasise practical concerns around reliability and the risk of incorrect outputs, and the necessity for thorough evaluation in diagnostic settings [1]. Narimani and Klarmann similarly focus on real-time vehicle diagnostic scenarios and motivate integrating LLMs to interpret diagnostic information and support decision-making, while recognising operational constraints that require careful system design and validation [4]. A key theme in automotive studies is that LLMs alone cannot offer effective diagnostics. They require integration with retrieval systems and domain sources to enhance accuracy and reduce hallucinations. This is why RAG has become the primary architectural pattern in many recent approaches to systems.

Han et al. introduce a RAG-based Q&A system focused on ship regulations and use generative techniques, such as GPT-4o, to synthesise training queries, aiming to reduce the need for labelled data, and it is beneficial in fields where annotated diagnostic Q&A pairs are limited [16]. This is conceptually similar to the thesis settings, where past resolved incidents are present but may not be marked as clean question–answer training data.

These studies support using LLMs and RAG for automotive industry troubleshooting assistance, but do not explicitly benchmark techniques using tool usage logs that cover both successful and failed ECU flashing cases. We focus on off-board diagnostic communication platforms and benchmark techniques within those constraints.

3.2 Improving RAG pipelines for higher response accuracy

Ampazis proposes practical improvements for RAG quality, including reranking and retrieval setup, and explicitly discusses the trade-offs between quality and computational cost or latency, which are crucial considerations for workshops, production, and aftermarket environments where response time is critical [17]. Liu et al. (Automotive Industry) evaluate several RAG variants for enterprise settings by comparing retrieval and indexing strategies, including chunking, retrieval options, and reranking, and report measurable differences in retrieval performance and subsequent answer quality [9]. The same author emphasises that pipeline choices, such as retrieval settings and reranking, can significantly influence performance [9]. This emphasises the need for systematic benchmarking rather than relying on a standard RAG configuration to achieve better response accuracy.

Agrawal et al. (Mindful-RAG) investigate the reasons behind RAG failures across the entire pipeline, including retrieval, context, and generation [5]. They suggest that accuracy issues often arise from retrieval mismatch, insufficient evidence, or incorrect context use [5]. These are common challenges found in automotive diagnostics, where errors can be subtle and difficult to identify, especially in context-heavy situations. This benchmarking closely aligns with this approach; we can evaluate retriever and re-ranker choices, chunking strategies, and context filtering against the knowledge base and tool logs of the diagnostic communication platform.

3.3 Knowledge-graph and graph-based RAG for structured technical knowledge

Sarmah et al. (HybridRAG) integrate knowledge graphs with vector retrieval, introducing a hybrid approach to enhance factual accuracy and retrieval performance beyond relying solely on embedding-based search [15]. Akbari's review of GraphRAG methods provides a taxonomy and highlights the importance of GraphRAG, as naive RAG may overlook structure and relationships that are essential for complex technical questions [2].

Jia et al. (StructRAG) propose a structure-aware framework that uses a scholarly knowledge graph and outline future research directions focused on evaluation metrics for structured retrieval, such as

diversity dimensions, illustrating that evaluation methods are still developing for KG-based RAG [18]. Rani et al. introduce graph-based retrieval augmentation techniques and evaluate generation quality using text similarity metrics such as ROUGE variants, reflecting a common yet occasionally limited approach to evaluating graph-augmented models [19].

GraphRAG is promising for ISO standards, diagnostic communication concepts, and incident knowledge. Most graph-RAG research benchmarks against generic QA; this thesis can assess whether graph structures improve log and knowledge base queries over vector RAG.

3.4 Domain adaptation and self-improving RAG

Siriwardhana et al. explore methods to enhance domain adaptation for RAG and show that adjusting both the retrieval and generation components can lead to significant improvements in domain-shifted QA tasks, often measured with EM/F1 [20]. Mao et al. (RAG-Studio) introduce an in-domain adaptation approach for RAG pipelines, focusing on systematic adaptation instead of depending on out-of-domain retrievers or generators [21].

Xu et al. introduce SimRAG, a self-enhancing RAG approach that gradually refines its components through feedback loops, reflecting a broader trend toward continuous improvement rather than static pipelines [22]. Lee et al. (tRAG) introduce term-level retrieval techniques that enhance the capture of domain-specific vocabulary, essential in scenarios where diagnostic logs and standards contain specialised tokens and abbreviations [23].

These studies support evaluating domain adaptation strategies, such as fine-tuning retrievers, synthetic query generation, and vocabulary-aware retrieval, as benchmark candidates for a diagnostics troubleshooting assistant. The data, consisting of logs and resolved incidents, is particularly appropriate for domain adaptation.

3.5 Benchmarking and evaluation frameworks for LLM and RAG troubleshooting assistants

Dholakia et al. argue that benchmarking generative AI demands a comprehensive approach that goes beyond basic metrics, focusing on aspects like factual correctness, bias, robustness, and user-centric usefulness; they also highlight the complexity of evaluating generative AI, which often depends on proxy measures and mixed evaluation methods [24]. They emphasise the importance of including operational metrics such as latency and robustness analysis for real-world systems [24].

Djeddal et al. introduce an evaluation framework for attributed information seeking and compare common architectures, such as Generate, Retrieve-Then-Generate, and Generate-Then-Retrieve, measuring both answer correctness and attribution quality through various metrics, including NLI-based methods that align with human judgments [25]. This directly corresponds to troubleshooting assistants who need to reference evidence from logs, standards, and previous incident records.

Sánchez-Navalón et al. focus on evaluating the performance and trustworthiness of RAG systems and explicitly discuss assessing hallucinations and faithfulness using multiple metrics, highlighting the growing importance of trustworthiness evaluation over simply generating answer similarity [26]. Yang et al. introduce CRAG as a comprehensive benchmark framework for RAG, highlighting the shift towards standardised RAG evaluation rather than isolated case studies [8].

These works provide a methodological grounding for RQ2, suggesting that the benchmark should include task-level correctness, faithfulness and evidence attribution, and operational constraints such as latency and cost. That said, what remains absent is a benchmark specifically designed for automotive diagnostic communication.

3.6 Hallucination and faithfulness in troubleshooting assistants

Hallucination poses a significant risk in automotive troubleshooting, as providing incorrect procedural guidance can lead to vehicle downtime or incorrect ECU flashing steps. Huang et al. survey hallucination in LLMs and define it as outputs that conflict with source knowledge or facts, highlighting both the taxonomy and mitigation strategies, such as retrieval grounding and verification [3]. Dholakia et al. similarly highlight hallucinations as a key concern in responsible AI and suggest that benchmarking should evaluate both the extent and harmfulness of these issues, although they note that this is challenging due to their context-dependence [24]. The thesis focuses on a knowledge base and tool logs as foundational sources, specifically addressing hallucination mitigation.

3.7 Identified Research Gaps

The literature consistently highlights RAG as the preferred architecture for trustworthy technical assistants, with ongoing research focusing on (i) optimising retrieval and reranking techniques [9], (ii) structured retrieval methods [18], (iii) domain adaptation strategies [23], and (iv) evaluation frameworks that emphasise faithfulness and attribution rather than solely similarity metrics [26]. This thesis seeks to fill this gap by benchmarking state-of-the-art RAG variants, including structured and adaptation approaches, on diagnostic communication platform data, and evaluating both accuracy and trustworthiness using a comprehensive set of metrics.

3.8 Survey of Published Academic Literature to Address RQ1

This section presents the survey of published academic literature to address RQ1 for LLMs without RAG, LLMs with RAG, LLMs with Structured RAG, and LLMs with Domain-Adaptive RAG.

3.8.1 Survey to Address RQ1, LLMs used

Study and Domain	**Methodology/LLMs used**	**Limitations/research gaps/open questions**
[27], Complex QA attribution with knowledge graphs (CAQA)	Benchmark many LLMs such as GPT-4/ChatGPT and open-source like LLaMA2/Vicuna/Mistral/Mixtral	Calls out that robust automatic evaluation of attribution is still open; motivates improved evaluators and broader coverage; future work points to improving evaluators/coverage
[28], Biomedical yes/no QA with model selection	Self-supervised strategy using linguistic and embedding similarity between questions and passages to select among LMs such as PubMedBERT, etc.	Future work: broader linguistic categories and improved oracle-style selection/transfer to other tasks
[7], Survey: evaluation of LLMs across tasks/capabilities/trustworthiness	Systematises evaluation methods (benchmarks, human eval, automatic metrics) + dimensions like robustness/safety/bias	Major gaps: lack of standardisation, reproducibility issues, and weak evaluation for open-ended generation; calls for more reliable evaluation practices
[29], Benchmarking LLM QA/explanations with automated and human eval	Multi-LLM benchmark; combines automated similarity scoring and human rating dimensions	Discusses evaluation constraints and need for broader/robust evaluation; acknowledges limitations in scope/generalizability

[30], Operations/supply chain benchmarking (conceptual framework)	Conceptual/theoretical benchmarking framework; not focused on specific LLM implementations	Calls for theory-driven empirical validation of the framework
[31], QA over historical newspapers (noisy OCR text)	Creates a large-scale QA dataset and benchmarks Transformer QA (BERT/RoBERTa/T5 variants) + instruction-tuned LLMs (e.g., LLaMA2 7B/13B/70B, Mixtral 8×7B, Mistral 7B)	OCR noise causes major degradation; EM/F1 can be misleading for verbose generative answers; calls for more robust methods and better evaluation for noisy sources
[32], General benchmarking/evaluation of LLMs across tasks	Multi-model evaluation; emphasises prompt effects; compares automatic and human evaluation methods	Notes sensitivity to prompt design and domain-knowledge limits; highlights need for stronger/more standardised evaluation across tasks
[33], Industrial troubleshooting / predictive maintenance support with a hybrid AI and LLM agent.	Hybrid approach using LangChain agent + GPT-3.5 (OpenAI API) and app integration, such as Streamlit	Prototype/experimental; depends on external LLM services and data quality; needs broader validation and robustness checks
[34], Tiny/edge language models for automation/control under resource constraints	Discusses Tiny Language Models (TLMs), deployment constraints, and comparisons vs larger LLMs for control contexts	Challenges: domain adaptation, safety/verification, tight compute/memory; calls for systematic evaluation in control settings
[35], Chart question-answering by converting chart images to text for LLM reasoning	Modality conversion pipeline + prompting strategies using GPT-3.5-turbo (zero-shot, CoT, self-consistency, PoT, rephrasing), compared to a finetuned baseline	Sensitive to chart extraction quality, LLM output variance, dataset question variety, and robustness issues remain
[14], Synthetic dialogue benchmark generation grounded in knowledge graphs	Generates multi-turn dialogues from a KG using an LLM-driven pipeline with validation/post-processing to enforce structure/grounding	Quality depends on KG coverage and generation constraints; it needs stronger grounding and broader validation to avoid synthetic artifacts
[36], Cross-domain, including healthcare/transport: AI development & deployment challenges, explainability focus	Narrative review/discussion; covers XAI (LIME, SHAP); mentions GenAI/LLMs (GPT, LLaMA, Claude) but says main focus is traditional AI	Notes interpretability methods can be hard to interpret/validate, and highlight deployment issues like system compatibility/integration constraints
[37], Explainable AI for Industry 5.0 / human-centric industrial CPS	Survey and reference architecture; discusses EXAI/XAI and human-in-the-loop transparency needs	Notes trade-offs between accuracy and interpretability, and adoption challenges (complexity/overhead/data quality); need domain expertise and trustworthy explanations

[38], Conversational QA over multi-doc instructional and troubleshooting guides; benchmark and evaluator	Builds InsCoQA; uses GPT-4 to generate multi-turn conversational history and translation; evaluates Mistral-Instruct v0.3, WizardLM-2, Llama-3.1-Instruct, GPT-4; evaluator ablation uses Qwen2-7B-Instruct vs GPT-4	Motivates a key gap: traditional reference-based recall metrics fall short for open-ended procedural answers; evaluator bias/domain transfer remain open concern implied by the need for LLM-evaluator and domain-specific benchmark

Table 3.1: Survey to address RQ1, LLMs used

3.8.1.1 Relevance

These studies are relevant for benchmarking evidence-grounded troubleshooting assistants, particularly for measuring faithfulness and attribution when answers are justified by KB or log evidence [27]. Several papers enhance the evaluation framework for the thesis by motivating multi-metric, tool-aware benchmarking for faithfulness and tool-use reliability, and by providing practical templates that combine automatic metrics with human or LLM-assisted judging to assess troubleshooting quality [7], [29], [32]. Others contribute design insights relevant to automotive diagnostics, including dynamic routing across models and tools for different query types, robustness to noisy or incomplete evidence, and hybrid structured reasoning with generative explanations for maintenance workflows [28], [31], [33]. Finally, synthetic data generation and procedural troubleshooting benchmarks, especially those using ontologies, knowledge graphs, or LLM-assisted evaluation, can be useful for constructing and validating automotive-focused benchmarks when real diagnostic dialogues or labelled QA are insufficient [14], [38].

3.8.1.2 Evaluation metrics used

This survey evaluation spans attribution quality, task performance, and operational feasibility. Several papers measure attribution/grounding using classification metrics, including precision, recall, accuracy, and F1, including macro-F1 over fine-grained labels, such as supportive vs contradictory evidence, and compare automated and human judgments [27]. Task success is measured by accuracy/EM/F1, edit-distance variants such as MNED, relaxed accuracy, sometimes augmented with LLM-oriented measures such as token recall or answer-string containment, and often by improvements against oracle or baseline selections [28], [31], [35]. For generative outputs, studies use automatic similarity metrics (ROUGE-L, BLEU, BERTScore, BLEURT) and consistency, combined with human ratings of correctness, comprehensiveness, and relevance, or judge-based scores such as JudgeScore and task completion rate/recall [29], [32], [38]. Multiple works emphasise pipeline- and deployment-level KPIs: success rates, validation/parsing errors, and efficiency measures such as processing time, latency, and token/cost, highlighting evaluation dimensions relevant to benchmarking diagnostic assistants integrated with tools and logs [14], [33], [34].

3.8.2 Survey to Address RQ1, LLMs and RAG

Study and Domain	**Methodology/LLMs used**	**Limitations/research gaps/open questions**
[39], RAG evaluation: can LLMs judge utility of retrieved passages?	Studies prompting/selection strategies for utility judgment (k-sampling, listwise); evaluates LLM judges, including GPT-3.5-turbo and open models (Llama2, Vicuna)	Notes: LLM judgment can be distorted by internal knowledge; semantic retrievers miss non-paraphrases; listwise is costlier and sensitive to passage ordering

[13], Goal-oriented dialogue / conversational QA with multi-document retrieval	RAG-style pipeline using LlamaIndex and retrieval components; experiments with retrieval/prompting to improve faithfulness	Highlights latency/cost bottlenecks from LLM calls and retrieval accuracy trade-offs; calls for optimisations to reduce LLM computation and retrieval time
[40], RAG faithfulness: evidence contextualization and attribution for knowledge-grounded QA / hallucination risk	Proposes evidence contextualization (uses full-document context) and counterfactual attribution; uses GPT-4o as the main LLM; also evaluates Llama 3.1-8B	Points out gaps in prior work: ignoring full-doc context; attribution not truly causal. Bottlenecks include question completion and entity/pronoun resolution
[41], Practical implementation of a generative AI service (system architecture)	Implements a RAG-style stack (e.g., LangChain, Chroma/FAISS) using GPT-3.5-turbo / GPT4All; embeddings via OpenAI embeddings; focuses on service workflow	Highlights need to improve responsiveness, accuracy, and deployment concerns (privacy/security), and imply a need for more rigorous evaluation in real use
[3], Hallucination (factuality/faithfulness), detection & mitigation (incl. RAG).	Survey taxonomy, causes and detection/mitigation approaches; discusses hallucination in RAG/knowledge-intensive settings.	Points to open problems, including hallucination for VLMs and identifying knowledge boundaries; discusses challenges in RAG, such as retrieval timing/ambiguity and failures
[25], Evaluating attribution/grounding in LLM outputs (RAG-relevant)	Framework combining classical NLG metrics with attribution-focused checks (AutoAIS / NLI-style precision/recall, citation overlap measures)	Shows standard similarity metrics are insufficient for attribution; motivates better faithfulness/citation evaluation
[1], Automotive vehicle diagnostics with LLM assistance	Applies LLMs for diagnostic support and recommends grounding via RAG to reduce hallucinations	Emphasizes hallucination/trust issues and the need for grounded methods and stronger evaluation in real workflows
[42], Text-only textbook QA	Paragraph-level RAG (OpenAI ada-002 embeddings + Pinecone + Cohere rerank) + Llama2 SFT + LoRA/PEFT	Limited to textual part (ignores diagrams); retrieval can distract; future work: multimodal integration + better retrieval
[17], RAG quality improvement (general)	Improves RAG pipeline quality via retrieval/reranking/query refinement (topic-aware improvements). Uses multiple LLMs (varies across experiments).	Still sensitive to embedding/retrieval failures and noisy corpora; evaluation often depends on proxy metrics and/or dataset assumptions; generalisation to other domains/tools can be unclear.
[24], LLM benchmarking/evaluation methodology	Framework/taxonomy style paper: how to benchmark GenAI systems across dimensions (quality, safety, efficiency, etc.).	Highlights lack of standardisation and comparability across benchmarks; benchmarks can become stale; human eval is expensive; metric gaming.

[10], Automotive/driver assistance	RAG-based assistant for driving support; compares different LLM backends (e.g., GPT-family vs others depending on setup). Uses retrieval + generation; often evaluated with RAG evaluation tooling.	Often limited by dataset realism, safety validation, and real-world deployment constraints (privacy, edge conditions).
[43], Healthcare / medical QA reliability	RAG pipeline with classical + neural retrieval (e.g., BM25 + embeddings) and reranking; compares multiple open LLMs (e.g., Llama2-class + others such as Zephyr/SOLAR depending on config).	Heavy reliance on expert labelling; small-scale evaluations can limit statistical confidence; domain shift and evidence quality remain open.
[12], Automotive software engineering	Uses LLMs to support software development tasks (requirements/code/tests/docs). LLMs include GPT-family and/or comparable models, depending on the experiment.	Limited samples/case studies; reproducibility sensitive to prompts/tooling; confidentiality and integration into real dev workflows remain open.
[44], Education/tutoring	RAG tutor architecture (e.g., LlamaIndex + vector DB like Chroma) with an LLM backend (ChatGPT/GPT-3.5 style).	Few expert raters; limited user study evidence; cost and deployment practicality; generalisation beyond one course/content set.
[45], Prompting / cognitive attention modelling	Prompt-tuning methods to shape "attention-like" behaviour; evaluates LLM behaviour changes (GPT-4 / Claude-class mentioned).	Prompt-tuning effects can be task-dependent; unclear transfer to real industrial troubleshooting; limited grounding/retrieval focus.
[4], Real-time assistance / industrial troubleshooting	Conceptual/system framework: combines LLMs with RAG/fine-tuning/tool integration for real-time support.	Often not a full empirical benchmark; highlights integration/data-quality issues, transparency, scalability, safety.
[46], RAG robustness	Retrieval-side improvements (e.g., better query formulation/retrieval/reranking) to make RAG less brittle.	Robustness tests depend on perturbation design; may not cover domain-specific failure modes (e.g., diagnostic logs); compute cost.
[47], Automotive failure analysis/knowledge management	LLM-assisted summarisation/knowledge structuring for failure analysis; includes GPT-4 usage in the described pipeline.	ROUGE may not reflect factual correctness; limited dataset diversity; grounding/citation, and causal correctness remain open.
[48], Cybersecurity QA	RAG chatbot pipeline (retrieval + LLM generation) over a cybersecurity knowledge base.	Domain constraints; coverage depends on the corpus; limited real-world adversarial testing.
[40], Evidence-grounded conversational QA	"RAGonite"-style system: evidence contextualization, counterfactual attribution, hybrid retrieval; compares LLM backends (e.g., GPT-4o + Llama 3.1 class).	Bottlenecks in question completion/retrieval quality; explanation scoring remains difficult; dependence on corpus structure.
[26], RAG evaluation/trustworthiness	Proposes or compares evaluation approaches for RAG trustworthiness;	Ground truth for "faithfulness" is hard; automatic metrics can

	uses common RAG metrics + methodological guidance.	disagree with humans; domain portability is open.
[15], RAG and knowledge graphs	Hybrid retrieval: combines KG-based structured retrieval with vector similarity retrieval, then LLM generation.	KG construction/maintenance overhead; entity linking errors; scalability to changing corpora; integration complexity.
[6], Automotive design support	Applies RAG to vehicle design knowledge bases; LLM generates design guidance grounded in retrieved docs.	Early-stage evaluation; limited large-scale benchmarking; unclear generalisation to other vehicle domains and proprietary KBs.
[49], Enterprise / internal knowledge search & RAG QA (software/product documentation)	Proposes an enterprise RAG pipeline emphasising modularity and "human-in-the-loop/lead" evaluation; model-agnostic approach (can try different LLMs), retrieval grounded in enterprise sources (search/topic-based retrieval rather than only classic offline benchmarks).	Open issue: reliable evaluation for questions with no single ground-truth; cost/effort of human evaluation; need for monitoring over time (drift, content changes), and better practical metrics for the enterprise.
[50], Safety engineering / automated safety requirement derivation (text generation in safety-critical context)	Uses an LLM-based pipeline to derive safety requirements from inputs; explicitly uses GPT-3.5 (generator + LLM-as-judge) in evaluation loops.	Heavy dependence on LLM-as-judge reliability; sensitivity to prompts/format; generalizability to other safety domains and requirement styles; evaluation remains hard when multiple "acceptable" requirements exist (needs stronger ground-truthing / expert baselines).
[51], Intelligent transportation / autonomous driving; explainability + RAG	Builds a RAG-based explainability approach around a behaviour-prediction setting; uses GPT-4 for generated explanations; uses all-MiniLM-L6-v2 embeddings and Chroma vector store for retrieval.	Open questions: robustness of explanations under imperfect retrieval; dependence on the knowledge base coverage; faithfulness of generated explanations to retrieved evidence; transferability across road-user contexts/datasets.
[9], Automotive documentation QA; RAG optimisation	Practical RAG system work for automotive documents; mentions local deployment using Ollama, orchestration with LangChain, agentic/self-RAG elements (e.g., LangGraph-style flows) and retrieval improvements for PDFs.	Notes gaps around optimising Ollama + function calling / callbacks for RAG; deployment in low-resource environments; need for robust/secure retrieval as doc volume grows; more optimisation needed for automotive document processing.
[8], RAG benchmarking across multiple domains/tasks	Builds a benchmark with a standardised scoring system and evaluation pipeline for RAG systems, including automated judgment of answer quality/truthfulness. Includes system-level measures such as latency and	Notes that many benchmarks do not directly evaluate the first-stage candidate pool and calls the missing evaluation of early retrieval quality.

	identifies coverage gaps in the evaluation.	

Table 3.2: Survey to address RQ1, LLMs and RAG

3.8.2.1 Relevance

This related work is highly relevant, as it aligns with the core needs of an automotive diagnostic assistant: evidence-based answers, traceability, and rigorous benchmarking. Studies highlight that the usefulness and accuracy of retrieved evidence, beyond lexical overlap, are vital for multi-turn troubleshooting, and they provide metrics and protocols to evaluate grounding, hallucination, and RAG failures [13], [25], [39]. Several works provide end-to-end patterns for integrating KB and log retrieval into assistant UIs, producing stepwise, attributable responses that cite evidence behind each decision, directly matching diagnostic communication platform needs [40], [41]. The literature supports the benchmarking methodology by motivating multi-metric evaluation, covering quality, reliability, robustness, and efficiency, with frameworks such as RAGAS-style approaches that can be reused or adapted for automotive RAG evaluation [4], [9], [24], [26]. Finally, domain-adjacent reliability research, such as medical, safety workflows, and automotive-specific RAG evaluations, offers strategies for reducing hallucinations, handling noisy or partial evidence, and rubric-based judging, which are important when diagnostic ground truth is incomplete, and correctness must be defensible [10], [43], [46], [47], [49], [50], [51].

3.8.2.2 Evaluation metrics used

Across the related work, evaluation generally occurs across three levels: retrieval quality, answer quality, and trustworthiness or operational performance. Retrieval effectiveness is assessed using IR metrics like precision, recall, F1, and ranking metrics such as nDCG@k, MRR/MAP, and Recall@k, including P@1 and recall@1/@2, which are often associated with improvements in downstream QA tasks, such as F1 score enhancements [13], [17], [39], [40], [51]. Answer quality is assessed using task success and QA metrics, such as accuracy and EM/F1, with breakdowns by question type, and sometimes overlap or semantic similarity measures, such as BLEU, ROUGE-L/ROUGE-F1, and BERTScore, or custom similarity scores like NASS [1], [15], [42], [47], [50]. Trustworthiness is evaluated through a combination of measures including faithfulness and factual accuracy, using NLI/QA-based checks, citation overlap metrics, AutoAIS-style methods, and uncertainty-aware scoring, and LLM-as-judge or expert Likert ratings, while broader frameworks also emphasise monitoring toxicity, bias, robustness, latency, cost, and user satisfaction; several studies implement these assessments with RAGAS-style metrics, which cover context precision and recall, answer relevance, faithfulness, and specific scores for hallucination, missing information, and truthfulness, as well as response time [3], [8], [9], [24], [25], [26], [43], [49].

3.8.3 Survey to Address RQ1, LLMs and Structured RAG

Study and Domain	**Methodology/LLMs used**	**Limitations/research gaps/open questions**
[19], Healthcare (diabetes care and control QA)	Builds a diabetes knowledge graph and integrates keyword search + graph search + vector embeddings to retrieve context for an LLM. Graph construction uses GPT-3.5-turbo. Uses human annotator answers as a benchmark.	Long/summary-style generative answers hurt overlap metrics; authors highlight summarisation of long answers as a key improvement area. Suggests future enhancement by adding property graphs.
[18], Scholarly/academic QA with scholarly knowledge graph	Inference-time structurization: query analysis → entity recognition (uses GPT-4) → retrieve papers → iterative segmentation into sub-questions/subgraphs. Answer generation	Failure modes: incorrect/insufficient entity recognition can lead to missing structure; the dataset uses paper

	uses LLaMA3-70B-Instruct; evaluation uses Claude 3 Sonnet as the LLM judge.	abstracts rather than full papers, limiting the available information.
[5], General RAG systems (failure analysis and evaluation)	Decomposes RAG pipeline and identifies failure points across indexing/retrieval/generation; proposes automated evaluation and a Mindful-RAG evaluation suite. Uses LLMs, including ChatGPT, for evaluation and comparison.	Focuses on detecting failures rather than guaranteeing fixes; highlights that different pipeline stages can break and need targeted mitigation.
[52], Survey/review of RAG techniques and trends	Reviews classic RAG components: retriever (e.g., DPR, BERT) + generator (e.g., BART); covers structured integration approaches like GraphRAG (LLM-based entity/relationship extraction) and modular/domain adaptation via benchmarking environments (e.g., RaLLe).	Points out risks such as bias/misinformation in retrieved content, and the need for selective retrieval; also emphasises domain-specific challenges.
[53], Knowledge graph completion (link prediction/relation inference)	Proposes KRIS: converts into NL and uses RAG over Wikipedia dump + Common Crawl to support reasoning. Uses Phi-3-medium-4k-instruct for NL conversion/prompting. Improves prompting by retrieving candidate relations and refining answers.	Authors' note: LLMs can hallucinate and have limited historical data; KRIS mitigates this with retrieval, but still depends on data availability/quality.
[2], Graph RAG methods + comparative benchmarking (technical + narrative datasets)	Identifies the “structural gap” motivating Graph RAG; benchmarks GraphRAG/LightRAG/StructRAG variants and emphasises trade-offs between quality vs cost. Uses evaluation covering efficiency and quality, including formal definitions of latency and token cost.	Explicitly states a research gap: frameworks are evaluated in isolation and lack “apples-to-apples” comparisons, and literature often ignores operational cost. Open question: how to balance efficacy/robustness/efficiency in practice.

Table 3.3: Survey to address RQ1, LLMs and Structured RAG

3.8.3.1 Relevance

These works demonstrate how RAG can be made more reliable, explainable, and measurable in automotive troubleshooting. Studies show that combining structured knowledge bases with hybrid retrieval improves answer quality and traceability, both essential for grounding recommendations in diagnostic evidence [19]. This is particularly important for diverse automotive diagnostic knowledge, such as logs, tables, and procedures, where approaches are needed to select or construct a structure during inference to align with actual troubleshooting workflows [18]. Several papers provide frameworks for benchmarking RAG failure modes, such as retrieval issues, redundancy, unsafe outputs, and factual errors, and key strategies, such as modular pipelines, hybrid retrieval, and domain adaptation [5], [52], which are relevant to diagnostic KBs and logs. Research on structured reasoning highlights linking symptoms, such as causes, and repairs, indicating that retrieval and prompting strategies can improve automotive knowledge reasoning [53]. Evaluation work provides templates for evaluating RAG systems on quality, robustness (hallucination), and efficiency (latency/token cost), which are crucial for benchmarking troubleshooting assistants with KBs and logs [2].

3.8.3.2 Evaluation metrics used

The reviewed literature assesses the quality of RAG system's answers and their efficiency. Some studies compare generated answers to human references using overlap-based metrics such as ROUGE-

1 and ROUGE-L when contrasting Graph RAG with traditional or vector RAG approaches [19]. By contrast, others report QA accuracy with response diversity, latency, and output length [18]. Retrieval-centric benchmarks assess whether the right evidence is found and how clean the retrieved context is, using measures such as retrieval hit rate, context relevance, context redundancy, toxicity, and factuality [5], and structured-knowledge settings often use ranking metrics such as Hits@1/3/10 [53]. Finally, benchmarking work proposes complete suites that combine end-to-end latency, token usage, and semantic similarity, such as BERTScore, hallucination detection, and LLM-as-judge quality scoring [2], whereas some review papers survey available tooling, such as RAGAS, without presenting a unified metric suite [52].

3.8.4 Survey to Address RQ1, LLMs and Domain-Adaptive RAG

Study and Domain	**Methodology/LLMs used**	**Limitations/research gaps/open questions**
[16], Maritime/ship-regulations QA (domain-specific doc retrieval + answer generation)	RAG pipeline retrieving regulation clauses via embedding similarity + ranking; answer generation with a LLaMA 3.1-based module; includes an LLM-as-a-judge scoring module	Heavy reliance on judge-based scoring and domain-specific tests; robustness depends on retrieval quality & terminology alignment; limited evidence of generalisation beyond ship regulations
[54], Survey/conceptual overview of adaptive RAG	Reviews adaptive retrieval strategies (reranking/query refinement, dynamic context selection, efficiency-focused enhancements); not tied to one fixed LLM	Primarily a survey (limited/no new experiments); highlights the lack of standard benchmarks and robust faithfulness/grounding evaluation as open needs
[20], Domain adaptation for RAG under domain shift (QA)	Two DA methods: masked-context reconstruction (teach model to ignore out-of-domain contexts) + auxiliary task to contextualise queries; applied to RAG-Sequence and RAG-End2End	Focused on certain datasets/shifts; may require domain-specific tuning; does not directly address tool-use/log reasoning typical in industrial troubleshooting
[21], In-domain adaptation of RAG (enterprise/specialised domains)	RAG adaptation pipeline using synthetic/self-training signals to adapt the retriever and generator for a target domain; integrates LLM-based evaluation in the workflow	Quality depends on synthetic data (risk of error propagation); compute/training cost; limited coverage of tool-use integration & operational constraints in eval
[22], Self-improving/self-training RAG (low/no labels)	Unsupervised dual-stage improvement: learn retrieval-oriented behaviour, then specialise on domain corpus via pseudo-labelled QA; includes judge-model usage in evaluation for some tasks	Feedback-loop risk (self-generated labels can reinforce mistakes); compute cost; depends strongly on initial retriever quality and corpus coverage
[23], Retrieval and RAG focused on rare/unseen terms (term-level recall)	Term-level RAG: retrieve candidate keywords, verify/refine with an LLM; uses GPT-3.5-Turbo; keyword retriever uses sentence-transformers/msmarco-distilbert-base-v2; supports query refinement/expansion	Multiple LLM calls increase cost/latency; depends on keyword retriever and verification quality; may add little if term coverage is already high

Table 3.4: Survey to address RQ1, LLMs and Domain-Adaptive RAG

3.8.4.1 Relevance

This set of studies is highly relevant because it outlines repeatable patterns for building and benchmarking domain-specific assistants in an automotive diagnostics setting, combining KB retrieval, generation, and multi-angle evaluation, including retrieval effectiveness, latency, and quality scores across varied knowledge and context [16]. The literature helps frame the design and benchmarking space, including adaptive retrieval, reranking, query refinement, and latency trade-offs, providing concrete evaluation dimensions for this thesis [54]. Several works motivate robustness and adaptation in shifting domains, such as new software releases and fault patterns, suggesting self-improving loops that use proprietary KBs and historical diagnostic logs when labelled QA data is limited [20], [21], [22]. Finally, techniques for refining queries at the term level are directly applicable, and enhancing query formulations can significantly improve retrieval quality and end-to-end troubleshooting accuracy [23].

3.8.4.2 Evaluation metrics used

Related work focuses on retrieval quality, answer quality, and system performance. Multiple studies incorporate rank-sensitive retrieval metrics, such as custom mean accuracy using rank position, nDCG@k, Recall@k/Top-k recall, and latency or elapsed time, to assess real-world performance, including semantic similarity scores, and sometimes extend retrieval evaluation by analysing term-level recall for seen vs unseen terminology, as well as by comparing query-expansion [16], [20], [23]. For assessing answer quality, standard measures include Exact Match and F1 for QA, as well as overlap and semantic-similarity metrics such as ROUGE-L and related scores. Some studies also report BLEU or Evidence F1 (E-F1), depending on the specific task setup [21], [54]. Finally, several studies complement automatic metrics with LLM-as-judge ratings on clarity, accuracy, and completeness, using a 1-5 scale, including GPT-4 as a judge to better evaluate the qualitative features of generated troubleshooting guidance [16], [21], [22].

4 METHOD

4.1 Defining Research Questions

RQ1: What are the state-of-the-art machine learning techniques used to provide accurate responses in a generative AI-powered troubleshooting assistant to user queries, considering the knowledge base data and tool usage logs within the automotive off-board diagnostic communication platform? Which techniques can be applied among the techniques identified?

RQ2: How do different machine learning techniques, identified in RQ1, compare in generating accurate responses to user queries in terms of performance by utilizing knowledge base data and tool usage logs, including both failure and non-failure cases, for a generative AI-powered troubleshooting assistant within the automotive off-board diagnostic communication platform?

To address RQ2, the following hypotheses are formulated:

H0 (null hypothesis): There is no significant difference in performance among the machine learning techniques identified in RQ1 for achieving the most accurate responses.

H1 (alternative hypothesis): There is a significant difference in performance among the machine learning techniques identified in RQ1 for achieving the most accurate responses.

4.2 Research Methods

Quantitative methods were chosen for this thesis to test hypotheses rather than generate insights, using a survey and experimental analysis. To address RQ1, a survey of the published literature synthesises methods, maps the current state-of-the-art, and highlights gaps, helping select suitable machine learning techniques. For RQ2, two hypotheses were developed, and the experimental analysis produced numerical results and performance metrics, including accuracy, precision, and recall, to evaluate effectiveness. A case study is not appropriate here, as it is mainly suited to qualitative analysis, while this study focuses on testing hypotheses.

4.2.1 Literature Survey to Address RQ1

We examined published academic literature to answer RQ1. This involved collecting relevant publications using the search keywords from the search terminology section. We then analyse the collected literature to identify the state-of-the-art machine learning techniques and research gaps. Based on these techniques, we will further benchmark their performance against the formulated hypothesis to provide a comprehensive answer to RQ2.

The survey of published academic literature addressing RQ1 is presented in Sub-section 3.8 of the Related Work section. It is organised into four categories: LLMs used, LLMs and RAG, LLMs and Structured RAG, and LLMs and Domain-Adaptive RAG. It also discusses the relevance and application of evaluation metrics for each category.

4.2.1.1 Search Terminology

We conduct a Systematic Literature Review (SLR) by systematically collecting relevant scientific publications and creating a structured search strategy. Our search covers leading international digital libraries, including Google Scholar, Scopus, ScienceDirect, and IEEE Xplore.

4.2.1.2 Search Keywords

Automotive diagnostics, root cause analysis, troubleshooting assistant, RAG, LLM, structured RAG, and domain adaptive RAG.

4.2.1.3 Inclusion Criteria

- Articles published in journals
- Full-length articles from international conferences
- Articles from the last five years, from 2020 to 2025
- Articles available in English

4.2.1.4 Exclusion Criteria

- Short papers from conferences, unless they are closely related to the research area
- Book chapters
- Articles that are too old

4.2.2 Benchmarking Experiment to Address RQ2

We conduct an experimental analysis to compare the identified state-of-the-art machine learning techniques, focusing on their ability to generate accurate responses to user queries using a knowledge base and tool usage logs, and to obtain insights to address RQ2. RQ2 is addressed by confirming or rejecting the null hypothesis H0 and the alternative hypothesis H1 defined in the introduction.

4.2.2.1 Independent and Dependent Variables

Independent variables: Several advanced machine learning techniques were employed during the experiment.

Dependent variables: Performance metrics collected from the experiment.

4.2.2.2 Input and Output

Input: User queries, including

- Textual description of problem
- Problematic log lines

Output: Benchmark report on model performance for

- Question-answering
- Root-cause analysis

4.2.2.3 Environment Settings

The device specifications provided below are used during the study:

Processor	11th Gen Intel(R) Core(TM) i7-11850H @ 2.50GHz
Installed RAM	32.0 GB
System type	64-bit operating system, x64-based processor
Operating system	Windows 11 Enterprise

Table 4.1: Device specifications

4.2.2.4 Technologies Used

Programming language	Python
LLMs	OpenAI, TinyLlama
Orchestration library	LangChain
Vector store	FAISS
Embeddings	Pretrained transformer-based embedding models
Libraries	LangChain's TextLoader, VectorStores FAISS, chains RetrievalQA HuggingFacePipeline and HuggingFaceEmbeddings Transformers's AutoTokenizer, AutoModelForCausalLM, pipeline SentenceTransformers's CrossEncoder, SentenceTransformer, util, torch, pandas, csv, time, nltk

sklearn's metrics f1_score, rouge_scorer, bert_score

Table 4.2: Technologies used

4.2.2.5 Data Collection

The data was generated using ChatGPT Enterprise, a licensed product of the company. All prompts and responses have been saved for transparency. After many retries, ChatGPT started giving repetitive incident names during ECU software flashing in response to the prompt “What are the repetitive incidents during software flashing into ECUs?” Initially, it provided seven common incidents. After nine more prompts asking for more incidents, it listed 100, covering common, real-world, advanced, rare, critical, system-level, experimental, ultra-rare, and theoretical incidents. This “master list of 100 incidents" encompassed the entire range of ECU flashing issues, from basic UDS failures to encrypted DoIP gateways, dual-core AI modules, and self-bricking firmware, said ChatGPT Enterprise.

When the list of 100 incident names was generated, the next prompt was to generate a dataset for each incident, containing the incident name, problem description, symptoms, root cause, solution, good log file and problematic log file. Each generated dataset was downloaded as a zip file. The next prompt was “Give me the datasets for the incidents "33. Over-The-Air (OTA) Flash Aborted Midway" during software flashing into ECUs by following the format: repetitive incident, problem description, symptoms, root cause, solution, good log file, problematic log file. Give me all the mentioned points in details.” The prompt was given for each incident name separately 100 times to generate more rich dataset.

A total of 500 sample user queries were generated, with 5 queries per incident, using the prompt “Give me five sample questions based on each repetitive incident during software flashing into ECUs” for retrieval quality evaluation, human evaluation, similarity evaluation, and cost analysis of the LLMs. A total of 130 problematic log lines were generated using ChatGPT Enterprise, with 5 problematic log lines per diagnostic service mentioned in the ISO 14229-1 document for root-cause analysis. The prompt was “Give me 5 problematic log lines for each diagnostic service (0x7F <ServiceID> <NRC> - <ServiceName-ServiceID>, <Error Name>, <Error Description>) based on the list below…” for generating the problematic log lines.

4.2.2.5.1 Data Structure

Knowledge base: Documents that explain the knowledge of resolved incidents, symptoms, root cause and solutions.
Tool usage logs: These logs contain NRCs and provide information about failure responses.

4.2.2.6 Data Preprocessing

4.2.2.6.1 Pdf Parsing and Extraction

The NRC tables from ISO standards serve as the knowledge base for the experiment. Each NRC table follows the same structure shown in the figure below. The NRC is defined in the first column. The error name and conditions for a specific NRC are defined in the description column. This information is important for retrieving log insights from the knowledge base. The diagnostic service name is not present in the table's title. The diagnostic service is extracted from the header-2 level of the ISO 14229-1 PDF document. The supported NRCs for each diagnostic service are extracted from the ISO 14229-1 document's tables. We used both the fitz and pdfplumber libraries for data extraction from the PDF as knowledge base.

Extracting multiple table titles from a page using pdfplumber was not correct. We experimented with regular expression methods (search, match, and findall), but none worked perfectly. It extracted only the first table title, not the second or third, if the page contained multiple tables. This is a limitation of pdfplumber. We then considered an alternative approach. We successfully used the fitz library to extract table titles. The pdf extraction pipeline combines table title extraction using fitz with table

extraction using pdfplumber. Extracting figures was out of scope for pdf extraction. Extracting all the tables from a pdf places a greater load on the retriever to retrieve relevant content. Instead, we extracted only the tables titled “Supported negative response codes” from the ISO 14229-1 standard.

Identifying the correct library for extracting PDF content was vital to retrieval quality. Reviewing the extracted content was important before proceeding to the retriever component. pdfplumber did not correctly identify the table titles. fitz (PyMuPDF) correctly extracted all table titles. pdfplumber failed to extract the second line from the same cell (Table 37 in the ISO 14229-1), indicating that the extracted content was incorrect. pdfplumber also failed to extract the table content from the next page when the table spanned two pages (Table 46 in the ISO 14229-1). pdfplumber correctly detected the NRC tables using coordinates, but it could not follow horizontal lines. We therefore read the text using fitz when we knew the table bounding box from pdfplumber. pdfplumber sometimes could not read the last line of the table because the bottom border was set too low. LangChain's RecursiveCharacterTextSplitter did not extract meaningful content from the PDF as well.

Table 18 — Supported negative response codes

NRC	Description	Mnemonic
XX_{16}	NegativeResponseCode#1 1. condition#1 : m. condition#m	NRC_
:	:	NRC_
XX_{16}	NegativeResponseCode#n 1. condition#1 : k. condition#k	NRC_

Figure 4.1: Supported NRC table structure in the ISO 14229-1

Example corpus chunk from ISO NRC table:
Metadata:
{'iso_standard': ISO 14229-1', 'category': 'Negative Response Code (NRC)', 'diagnostic_service': 'ECUReset-11', 'table_title': 'Table 37 — Supported negative response codes'}
Document:
Diagnostic service: ECUReset-11, NRC: 33, Error: securityAccessDenied, Mnemonic: SAD, Condition: This NRC shall be sent if the requested reset is secured and the server is not in an unlocked state.

4.2.2.6.2 Log Parsing

The problematic log follows a template-based, structured format. The log lines are extracted from the attached problematic log to derive log insights using the knowledge base data. 0x7F is the negative response prefix, so the next two bytes are extracted using a regular expression after 0x7F. These represent the diagnostic service and the error, respectively, according to the ISO NRC table structure. The problematic log lines are in raw format, so we use a simplified format and a log interpretation grounded in ISO 14229-1 to evaluate retrievers and LLMs, since LLMs do not understand raw log data or the NRC format. The conditions, described for each NRC, are extracted for root-cause analysis.

Simplify the log lines using a template:
[Timestamp] [Request/Response] [ServiceID] [SubFunction] [NRC] [Status]

Example raw log for the incident “Security access fails repeatedly”:
[11:05:00.400] -> Tester: 0x27 02 DE F0 (Retry)

[11:05:00.401] <- ECU: 0x7F 27 36 (Exceeded Number of Attempts)

Simplified log interpretation grounded to ISO 14229-1:

- Flashing Aborted: ECU locked due to repeated security access failure.
- 0x7F is the Negative Response prefix
- 0x27 refers to the SecurityAccess service
- NRC 0x36 is exceededNumberOfAttempts

4.2.2.6.3 Feature Extraction in RAG

In the experiment, the core features were extracted using the SentenceTransformer text embedding model "all-MiniLM-L6-v2", which converts user queries and knowledge documents into semantic vector representations. After extracting features from all knowledge documents, they were stored in a vector database, FAISS, which enables efficient nearest-neighbour search by comparing query embeddings with indexed document embeddings. When a user submits a query, we use the same encoder to encode it into a feature vector, ensuring that both queries and knowledge documents exist in the same semantic space.

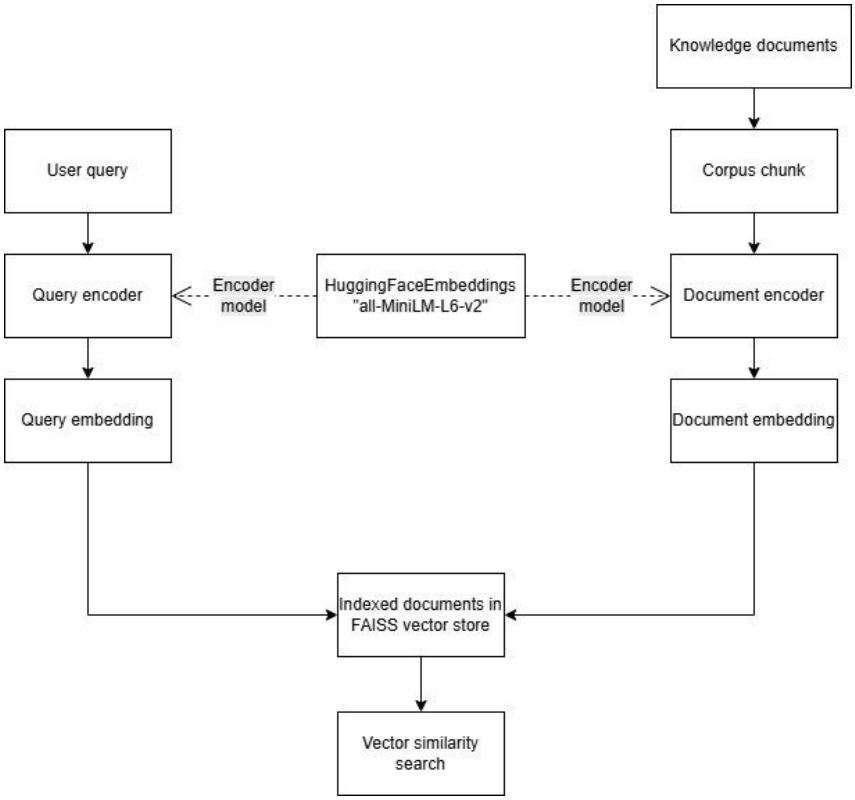


Figure 4.2: Feature extraction for RAG

Example of query and corpus chunk embeddings:
Query: "What causes an ECU to respond with a general programming failure during block transfer?"
Query embedding: [-0.04136392 0.0304158 0.03043108 ... 0.01298911 -0.05017056 -0.02367239]

Corpus chunk: "During the software flashing process, especially when transferring data blocks to the ECU using the UDS service 0x36 (Transfer Data), the ECU intermittently or consistently responds with a negative response: 0x7F 36 72. This indicates a general programming failure. It occurs typically when the ECU is unable to correctly receive, validate, or write a data block into memory. This issue may happen at a specific block or randomly depending on memory conditions, block size, or firmware integrity."
Corpus embedding: [0.0417034 0.00081703 -0.02320578 ... -0.01917244 0.00877253 0.00021307]

4.2.2.7 Pretrained Transformer-Based Embedding Models Used

During the experiment, the HuggingFaceEmbeddings wrapper from the LangChain library was utilized to enable the use of pretrained transformer-based embedding models from HuggingFace for query and corpus embeddings in RAG feature extraction. We employed the popular model "all-MiniLM-L6-v2" from SentenceTransformers for semantic search in document retrieval. Also, we used the Cross-Encoder model "cross-encoder/ms-marco-MiniLM-L-6-v2" from SentenceTransformers to improve retrieval accuracy in the experiment.

4.2.2.8 Experimental Process

For the experimental analysis, we generated a log dataset that closely mirrors the company's real data from the off-board diagnostic communication platform. The dataset was pre-processed and refined to make it suitable for testing. RAG-based models were then applied to the synthetic data until the optimal solution was found. The top-performing model was chosen for further use.

4.2.2.9 Evaluation Metrics

Evaluation criteria	Evaluation metrics
Similarity evaluation of a generative task	Cosine, ROUGE-L, BLEU, Jaccard, BERTScore
Retrieval quality	Relevance of Top-k documents retrieved
Latency	Time to generate a response
Throughput	Token count per second
Cost-effectiveness	Total cost of resources usage

Table 4.3: Evaluation metrics

During the experimental analysis, the evaluation focused on quality and efficiency: the similarity between generated and synthetic target value was measured using cosine similarity, BERTScore for semantic alignment, and ROUGE-L, BLEU, and Jaccard for lexical overlap. Precision and recall assessed how well the generated insights aligned with annotated root causes or known errors. The relevance of the Top-k retrieved documents was examined to evaluate the retrieval effectiveness. System performance metrics included latency (response time), throughput (tokens per second), and cost-effectiveness (total resource use).

4.2.2.10 Experimentation Pipelines

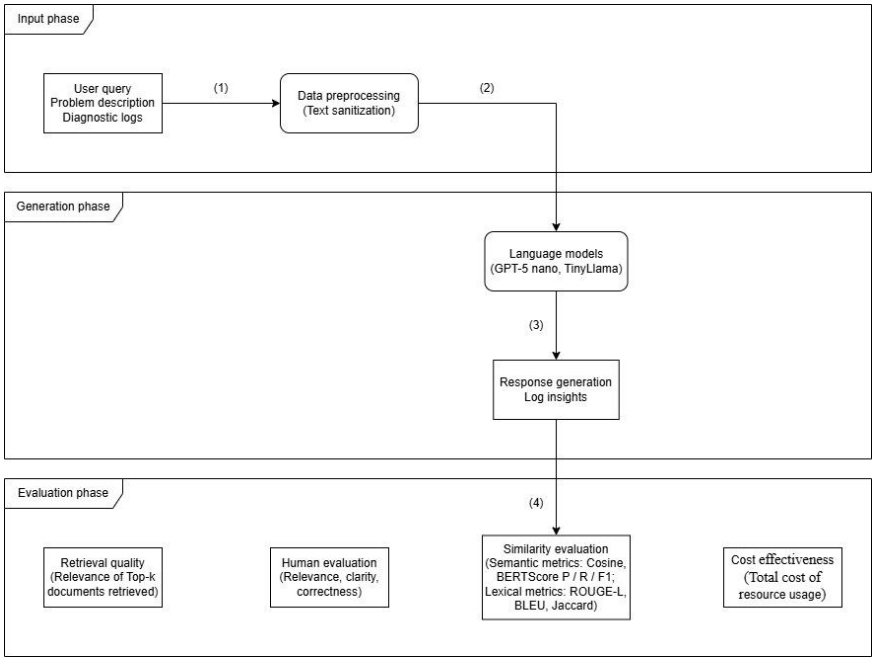


Figure 4.3: Pipeline for LLM without RAG

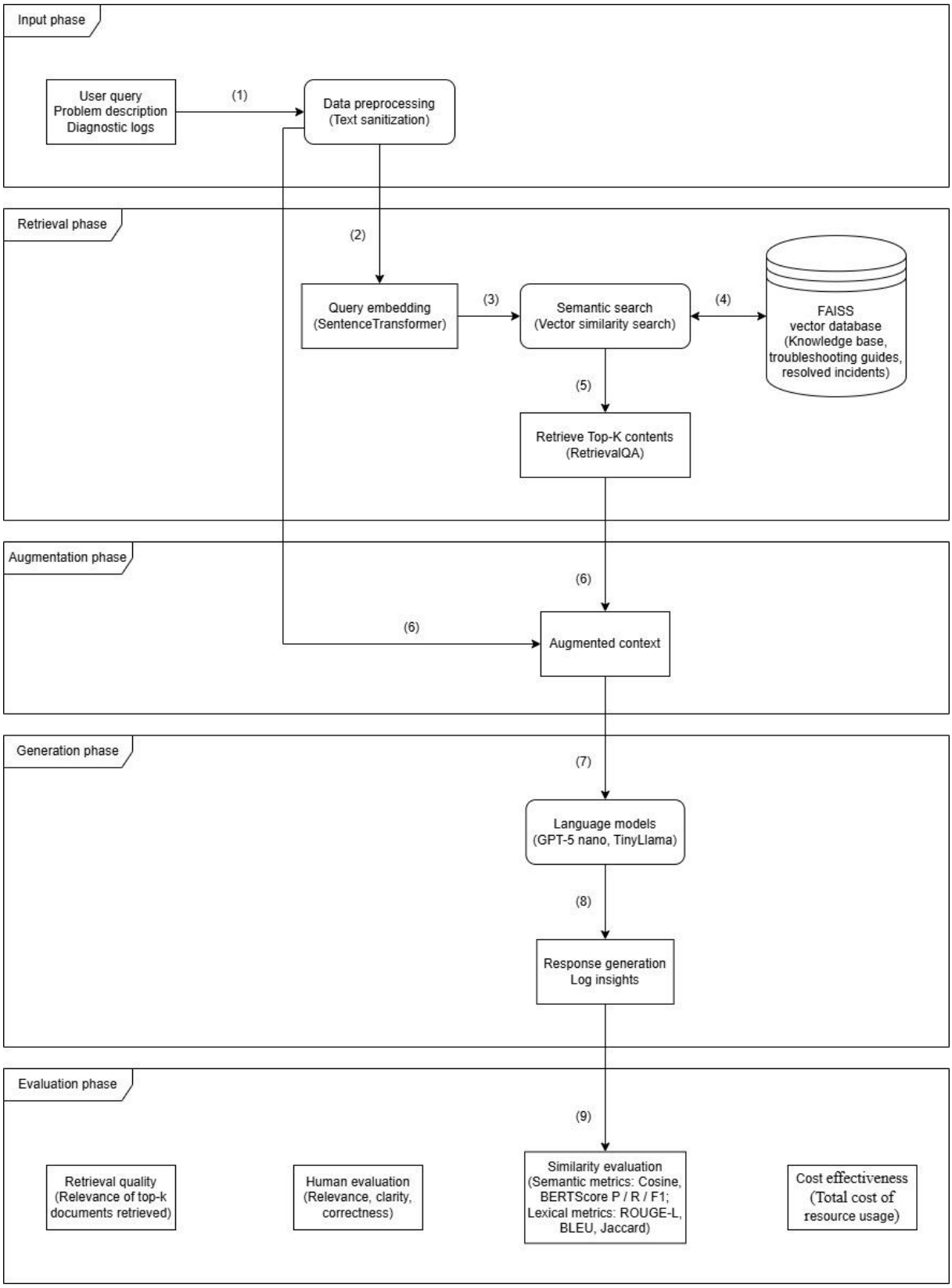


Figure 4.4: Pipeline for LLM with RAG

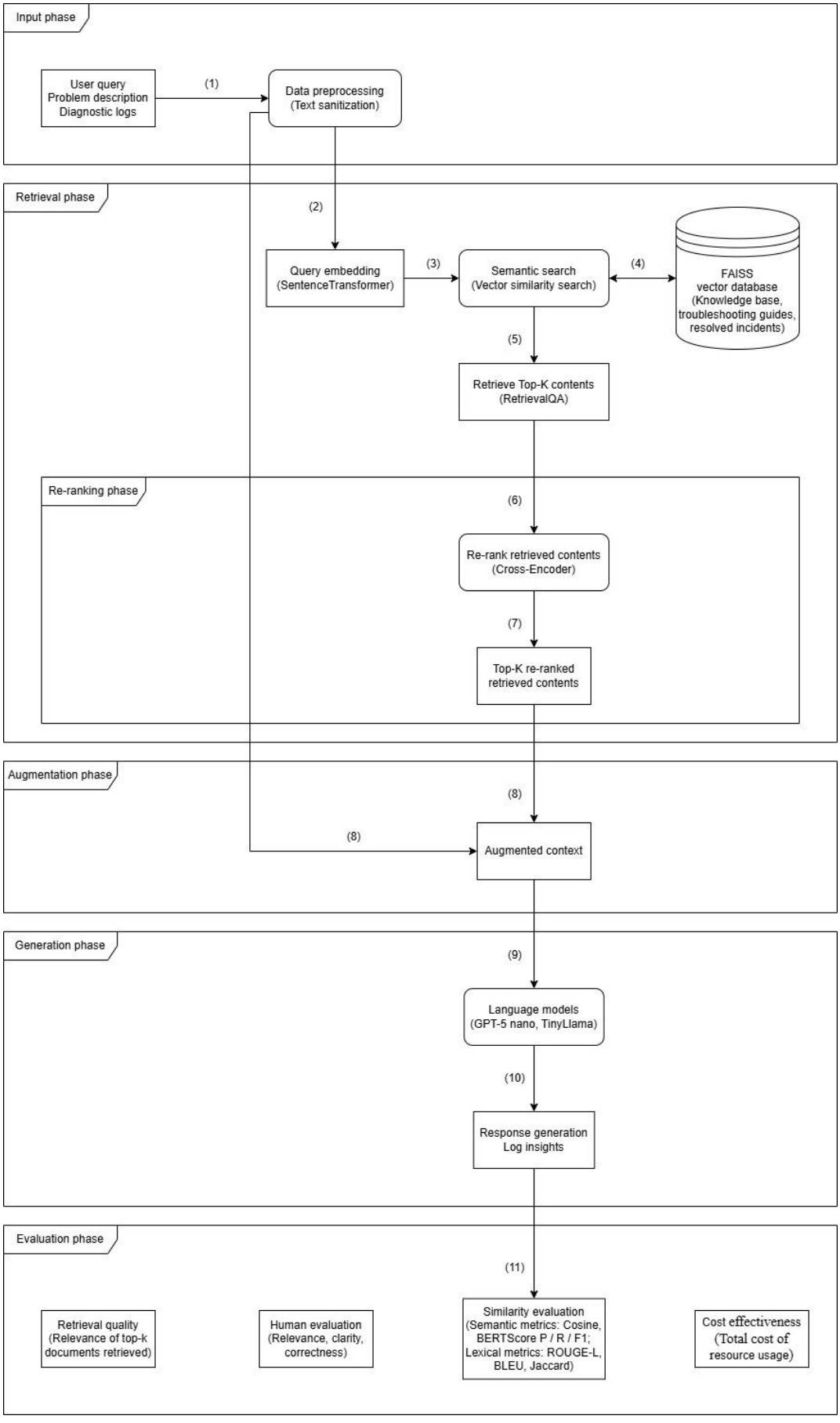


Figure 4.5: Pipeline for LLM with RAG and Reranking

5 RESULTS AND ANALYSIS

5.1 Descriptive Statistics of the Dataset

5.1.1 Populations

This compilation includes all identified repetitive ECU flashing failure modes within the automotive off-board diagnostics platform. It features lists of 100 repetitive and resolved incidents and 130 problematic log lines for each diagnostic service, representing comprehensive data sets for question-answering and root-cause analysis, respectively. All this data is generated using ChatGPT Enterprise for experimental purposes.

5.1.2 Samples

The dataset of 100 documented, unique, resolved incidents represents the complete population, not a sample. They are not extracted from a larger dataset (e.g., 1,000 incidents logged this year), so they are not a sample.

5.1.3 Data Structure

Each incident record includes several text fields. The incident title provides a brief label for the incident. The problem description provides a detailed explanation, including context and the events that occurred. The symptom field details observable issues during flashing, describing what the user or system notices when the problem occurs. The root cause explains the technical source of the issue and its underlying reason. Lastly, the solution field presents recommended fixes or preventive measures to resolve the problem and prevent recurrence.

The following sub-sections provide detailed results and analysis. Retrieval quality for root-cause analysis and question-answering is discussed in Sub-sections 5.2 and 5.3. We also analyse similarity assessments in Sub-sections 5.4, 5.5, and 5.6. The results of the hypothesis tests for TinyLlama and OpenAI are shown in Sub-sections 5.5.4 and 5.6.4, respectively. Finally, cost-effectiveness is discussed in Sub-section 5.7.

5.2 Retrieval Quality Evaluation of Domain-Adaptive and Structured RAG for Root-Cause Analysis

The Retriever component, designed for root-cause analysis, combines Structured RAG with a Domain-Adaptive RAG approach. This novel technique leverages the strengths of RAG without relying on embedding models or vector databases. It uses structured data sources, such as ISO lookup tables and standards content. For instance, ISO 14229-1 contains headings, sub-headings, paragraphs, tables, and figures, with a focus on extracting NRC tables relevant to diagnostic services. Layout-aware parsing of NRC tables from raw knowledge documents enabled us to generate semantic chunks, each representing a single NRC table row as a complete logical unit. The log lines used in our experiments are also in raw form.

We generated five problematic log lines for each of the 26 diagnostic services, following the pattern [0x7F <ServiceID> <NRC> - <ServiceName-ServiceID>, <ErrorName>, <ErrorDescription>], based on ISO 14229-1. We used NRCs, error names, and conditions from ISO 14229-1 for each service. A total of 130 log lines were used to evaluate retrieval quality in three experiments: baseline, query expansion, and reranking (Sub-sections 5.2.1, 5.2.2, 5.2.3). Using LangChain with the “all-MiniLM-L6-v2” embedding model, we embedded NRCs and descriptions and stored them in a FAISS vector store. No metadata filtering was applied, as FAISS does not support it by default.

5.2.1 Evaluation of Baseline, Query Expansion and Reranking

We evaluated three retrieval setups for root-cause analysis across 130 problematic log lines: baseline retrieval (Experiment 1), query expansion (Experiment 2), and query expansion with cross-encoder reranking (Experiment 3), for Top-10 and Top-20. The goal was to determine whether, for each problematic log line, the system could retrieve the correct NRC row or root-cause candidate within the Top-k, how high it ranked, and the latency involved.

Layout-aware parsed document of NRC table's row - Example
Diagnostic service: DiagnosticSessionControl-10, NRC: 12, Error: SubFunctionNotSupported, Mnemonic: SFNS, Condition: This NRC shall be sent if the SubFunction parameter is not supported.

Parsed document pattern
Diagnostic service: <ServiceName>-<ServiceID>, NRC: <NRC>, Error: <ErrorType>, Mnemonic:<Mnemonic>, Condition: <Conditions>

Query pattern
Diagnostic service: <ServiceName>-<ServiceID>, NRC: <NRC>

Query expanded pattern
Diagnostic service: <ServiceName>-<ServiceID>, NRC: <NRC>, Error: <ErrorType>

	Experiment 1 (Top-10)	**Experiment 2 (Top-10, query expanded)**	**Experiment 3 (Top-10, query expanded, cross-encoder reranking)**
Total Queries Evaluated	130	130	130
Found in Top-10	81	86	86
Not Found in Top-10	49	44	44
Retrieval Accuracy	62.31%	66.15%	66.15%
Match Position Breakdown	Matched index 1.0 24 2.0 14 3.0 12 4.0 7 5.0 8 6.0 4 7.0 8 8.0 2 9.0 1 10.0 1	Matched index 1.0 77 2.0 4 3.0 3 4.0 1 8.0 1	Matched index 1.0 86
Average Time	0.007 seconds	0.008 seconds	0.146 seconds

Table 5.1: Top-10 retrieval evaluation with query expansion and reranking

In Top-10 retrieval (Table 5.1), incorporating the error type into the query during Experiment 2 increases the percentage of queries correctly retrieving the NRC row from 62.31% (81 of 130) to 66.15% (86 of 130). This suggests that including the error type enhances the embedding model's ability to differentiate between NRCs that share the same diagnostic service ID and NRC but have different error types. Practically, this means about five additional log lines out of 130 would display the relevant NRC entry within the Top-10 results when query expansion is used.

While the cross-encoder reranking in Experiment 3 maintains a Top-10 accuracy of 66.15%, it significantly improves ranking quality. In Experiment 2, 77 NRCs are ranked first, but in Experiment 3,

all 86 hits are moved to the top position. For a troubleshooting assistant, ensuring the correct NRC appears as the first suggestion is crucial, as it minimizes the effort required to inspect lower-ranked candidates.

Adding query expansion for efficiency slightly increases retrieval time, from 0.007s to 0.008s, which is minimal. That said, employing cross-encoder reranking introduces notable overhead, increasing response time to 0.146 s, roughly 20 times the baseline. The acceptability of this trade-off depends on the platform's performance requirements. Reranking is justified when user interaction speed is less critical than the quality of recommendations.

Moving from Top-10 to Top-20 retrieval (shown in Table 9.1 in the appendix) with query expansion (Experiment 2) and reranking (Experiment 3) results are identical to Top-10 retrieval, with 86 of 130 correct hits. In the baseline (Experiment 1), retrieval accuracy improves slightly from 62.31% to 64.62%, adding just 3 correct hits out of 130 queries. For the query-expanded setups (Experiments 2 and 3), Top-20 accuracy remains at 66.15%, the same as Top-10, indicating all correctly retrievable NRCs are within the Top-10 candidates. This suggests that Top-10 retrieval is sufficient for the troubleshooting assistant, and increasing K mainly results in redundancy rather than expanding coverage.

For reranking, latency approximately doubles as K increases, as expected because cross-encoder complexity scales with the number of candidates scored. The average response time increases from 0.146s for Top-10 to 0.309s for Top-20, roughly doubling as K doubles. Since this extra cost does not enhance retrieval accuracy, Top-10 achieves lower latency while maintaining retrieval effectiveness.

5.2.2 Evaluation with More Query Expansion and Reranking

Compared with the previous setup (Sub-section 5.2.1), we extended the NRC table by adding raw UDS negative response data bytes (0x7F <ServiceID> <NRC>) to both the parsed document and the query, mirroring the raw diagnostic log lines. We aimed to align NRC rows more closely with the actual log errors.

Layout-aware parsed document of NRC table's row - Example
Data: 0x7F 10 12, DiagnosticService: DiagnosticSessionControl-10, NRC: 12, Error: SubFunctionNotSupported, Condition: This NRC shall be sent if the SubFunction parameter is not supported.

Parsed document pattern
Data: 0x7F <ServiceID> <NRC>, Diagnostic service: <ServiceName>-<ServiceID>, NRC: <NRC>, Error: <ErrorType>, Condition: <Conditions>

Query pattern
Data: 0x7F <ServiceID> <NRC>, Diagnostic service: <ServiceName>-<ServiceID>, NRC: <NRC>, Error: <ErrorType>

	Experiment 1 (Top-10, query expanded)	**Experiment 2 (Top-10, query expanded, cross-encoder reranking)**
Total Queries Evaluated	130	130
Found in Top-10	81	81
Not Found in Top-10	49	49
Retrieval Accuracy	62.31%	62.31%
Match Position Breakdown	Matched index 1.0 71 2.0 4	Matched index 1.0 81

	3.0 4 8.0 1 9.0 1	
Average Time	0.009 seconds	0.161 seconds

Table 5.2: Top-10 retrieval evaluation with more query expansion and reranking

In Top-10 retrieval (Table 5.2), using query expansion for reranking does not improve accuracy compared with the non-reranked approach; both retrieve the correct NRC within the Top-10 for 81 out of 130 queries (62.31%). The role of the cross-encoder remains consistent with that described in Sub-section 5.2.1 (Table 5.1); it does not increase the number of successful retrievals. The reranker only reorders the existing candidates, although it improves the ranking quality. Of the 81 matches, 71 are at rank 1 in retrieval-only, and reranking promotes all 81 hits to the top position.

Compared with previous query expansion without Data (query: <ServiceName>-<ServiceID>, <NRC>, <ErrorType>) (Sub-section 5.2.1, Table 5.1), Top-10 accuracy was 66.15% (86 out of 130), with all results at rank 1 after reranking. With more query expansion (adding Data: 0x7F <ServiceID> <NRC>), accuracy drops to 62.31% (81 out of 130). Adding Data slightly reduces accuracy. Reranking increases latency, with response time rising from 0.009s to 0.161s.

For Top-20 evaluation (Table 9.2 in the appendix), increasing k from 10 to 20 adds four additional hits (81 to 85) and accuracy without reranking rises from 62.31% to 65.38%. Reranking keeps the same 85 matches out of 130 but ranks all hits at 1. Latency increases, and response time grows from 0.009s to 0.336s, almost doubling from Top-10 to Top-20 reranking (0.161s to 0.336s).

Compared with the previous setup (Sub-section 5.2.1, Table 5.1), which used simpler query expansion without Data, we achieved 66.15% accuracy at Top-10. With Top-20 and more query expansion, accuracy drops to 65.38%, underperforming the simpler method. This confirms that semantically meaningful fields such as ServiceName, ServiceID, NRC, and ErrorType are valuable, while low-level byte representations do not help and can slightly degrade performance with the embedding model “all-MiniLM-L6-v2”.

5.2.3 Evaluation with Even More Query Expansion and Reranking

In the third configuration, we enhanced the textual structure by representing each NRC row as a key-value pair for documents and queries. Although this appeared more structured and helpful, the results were actually the opposite. The goal was to make UDS fields explicit to the embedding model and to see if a schema-like text would improve retrieval performance.

Layout-aware parsed document of NRC table's row - Example
ServiceID: 10, NegativeResponseCode: 12, ServiceName: DiagnosticSessionControl, ErrorType: SubFunctionNotSupported, Condition: This NRC shall be sent if the SubFunction parameter is not supported.

Parsed document pattern
ServiceID: <ServiceID>, NegativeResponseCode: <NRC>, ServiceName: <ServiceName>, ErrorType: <ErrorType>, Condition: <Conditions>

Query pattern
ServiceID: <ServiceID>, NegativeResponseCode: <NRC>, ServiceName: <ServiceName>, ErrorType: <ErrorType>

	Experiment 1 (Top-10, query expanded)	**Experiment 2 (Top-10, query expanded, cross-encoder reranking)**

Total Queries Evaluated	130	130
Found in Top-10	79	79
Not Found in Top-10	51	51
Retrieval Accuracy	60.77%	60.77%
Match Position Breakdown	Matched index 1.0 36 2.0 19 3.0 7 4.0 2 5.0 3 6.0 3 7.0 4 8.0 1 9.0 3 10.0 1	Matched index 1.0 79
Average Time	0.010 seconds	0.148 seconds

Table 5.3: Top-10 retrieval evaluation with even more query expansion and reranking

For the Top-10 evaluation (Table 5.3), the query-expanded configuration without reranking retrieves 79 of 130 correct NRC entries (60.77%). Reranking does not change this accuracy but improves ranking, with matches distributed across ranks 1-10. Without reranking, only 36 are at rank 1; with reranking, all 79 are promoted to rank 1. Response time increases from 0.010s to 0.148s with reranking.

The Top-20 results (Table 9.3 in the appendix) follow a similar pattern. Increasing K from 10 to 20 slightly increases accuracy from 79 to 81 matches (60.77% to 62.31%). While reranking does not increase the total number of successful queries, it improves the ranking of correct NRCs to top positions. Latency nearly doubles from 0.148s to 0.308s due to the additional cross-encoder processing.

Compared with previous query expansion strategies, structured text using key-value pairs does not improve retrieval performance. The best setup from Sub-section 5.2.1, combining textual descriptions of the diagnostic service ID, NRC, and error type, achieves a Top-10 accuracy of 66.15%. An intermediate configuration from Sub-section 5.2.2, adding the raw Data: 0x7F <ServiceID> <NRC> prefix, reaches 62.31%. The key-value representation performs worst at 60.77%, with a similar ranking for Top-10 accuracy.

These results suggest that for root-cause retrieval in UDS diagnostics, it is better to focus on concise, semantically meaningful fields, such as service name, service ID, NRC, and error type, rather than on raw data attributes and a highly structured key-value style representation.

5.3 Retrieval Quality Evaluation of Structured RAG for Question-Answering

For the question-answering task, the retriever component uses the Structured RAG approach. During software flashing into ECUs, 100 incident types are generated, covering categories such as common, real-world, advanced, critical, system-level, cutting-edge, and experimental. These resolved incidents are stored as structured data in a vector database for analysis. Each incident includes knowledge data such as problem descriptions, symptoms, root causes, and solutions, forming domain-specific knowledge chunks. In total, 400 chunks were created by storing each section with the incident title and embedding them using the “all-MiniLM-L6-v2” model and a FAISS vector index. We used 500 synthetic queries to evaluate retrieval quality, five per incident category.

For Top-5 and Top-10 evaluation in question-answering, Experiment 1 uses the raw user query as the retrieval query. In the Structured RAG setup (Experiment 2), a query expansion step transforms the original question into a pattern that aligns with the incident structure of stored documents in the vector database. The same expanded query is used in Experiment 3, where a cross-encoder reranks the Top-k candidates from the vector database.

Parsed document example
Title: General Programming Failure,
Section: Problem Description,
Content: During the software flashing process, especially when transferring data blocks to the ECU using the UDS service 0x36 (Transfer Data), the ECU intermittently or consistently responds with a negative response: 0x7F 36 72. This indicates a general programming failure. It occurs typically when the ECU is unable to correctly receive, validate, or write a data block into memory.

Parsed document pattern
Column: <ColumnName>, IncidentTitle: <IncidentTitle>, Content: <ProblemDescription>

Query pattern
<QueryString>

Query expanded pattern
IncidentTitle: <IncidentTitle>, Content: <query>

	Experiment 1 (Top-5)	**Experiment 2 (Top-5, query expanded)**	**Experiment 3 (Top-5, query expanded, cross-encoder reranking)**
Total Queries Evaluated	500	500	500
Found in Top-5	356	484	484
Not Found in Top-5	144	16	16
Retrieval Accuracy	71.20%	96.80%	96.80%
Match Position Breakdown	Matched index 1.0 240 2.0 39 3.0 41 4.0 16 5.0 20	Matched index 1.0 471 2.0 9 3.0 3 5.0 1	Matched index 1.0 479 2.0 2 3.0 3
Average Time	0.011 seconds	0.013 seconds	0.121 seconds

Table 5.4: Top-5 retrieval evaluation with query expansion and reranking

The question-answering task results in Top-5 (Table 5.4) and Top-10 (Table 9.4 in the appendix), demonstrating that structured query expansion is the key factor in enhancing retrieval accuracy. Cross-encoder reranking offers only minor improvements at a higher computational expense. In the baseline (Experiment 1), using the raw user query, the retriever correctly identified the relevant incident section in the Top-5 for 356 out of 500 queries (71.20%). Of these, 240 were ranked first, with the remaining across ranks 2-5. At Top-10, success increased to 386 queries (77.20%), spread across ranks 1-10. This suggests that without structured support, the retriever often finds relevant sections but not always at the top, requiring LLMs or users to inspect multiple candidates.

Implementing Structured RAG query expansion in Experiment 2 notably improved Top-5 and Top-10 retrieval results. By reformulating queries using a structured pattern aligned with incident representation, the system successfully retrieved the correct section within the Top-5 for 484 out of 500

queries, achieving 96.80% accuracy, and ranked it first in 471 cases. For Top-10, 486 queries were correctly resolved with 97.20% accuracy, with most results within the Top-5. The small accuracy gap between Top-5 and Top-10 suggests that relevant documents are almost always among the top candidates. This improvement results in a slight increase in latency: response times grow from 0.011s to 0.013 and 0.014s with query expansion, respectively.

In Experiment 3, cross-encoder reranking improves document rankings but does not increase retrieval accuracy. The number of queries where the correct section appears within the top results is similar to Experiment 2, with 484 out of 500 for Top-5 (96.80%) and 486 out of 500 for Top-10 (97.20%). It cannot retrieve queries missed by vector search. The ranking distribution among successful cases shows slight improvements: correct results at rank 1 increase from 471 to 479 for Top-5 and from 471 to 482 for Top-10. These gains are modest considering the higher computational cost, with response times increasing to 0.121s for Top-5 and 0.233s for Top-10, slower than pure vector retrieval with query expansion.

Analysing the tables suggests that structured query expansion is crucial for achieving high retrieval performance in question-answering. When queries align with the incident structure of the knowledge base, it enhances accuracy and top-rank quality with minimal impact on latency. Cross-encoder reranking slightly improves top-rank correctness but does not improve retrieval accuracy and introduces additional latency. From a system perspective, Structured RAG and query rewriting are fundamental components, whereas reranking remains optional and is used when the quality gain justifies the additional computational cost.

Both the Structured RAG and Domain-Adaptive RAG reduce the risk of hallucination in root-cause analysis and question-answering by replacing generic chunking with domain-aware retrieval. By ensuring retrieved content aligns with domain-specific structures, they assist LLMs in understanding problematic log lines based on ISO standards. This is crucial because LLMs cannot reliably interpret raw diagnostic logs or domain-specific incident knowledge required for accurate troubleshooting.

5.4 Similarity Evaluation - Retrieval of Root-Cause Analysis

Similarity evaluation for root-cause analysis retrieval is valid in RAG systems. It allows us to assess how well the embedding model captures domain-specific semantic relationships independently of ranking. That said, it should complement, not replace, retrieval accuracy metrics such as Top-k, as it cannot measure retrieval performance on its own.

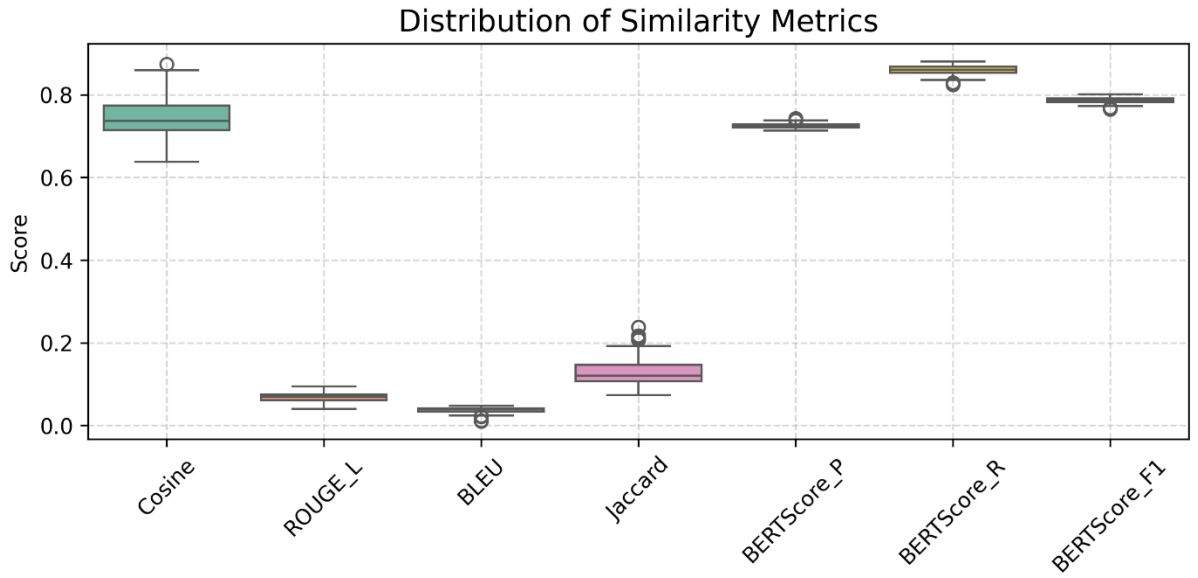
The NRC conditions, based on knowledge documents for problematic log lines using the Structured RAG method to retrieve root-cause information in natural language. The query mimics a knowledge entry structure, and the retriever returns context when this structured query appears. The query serves as target value for the retrieved content. To evaluate retrieval quality, we compared the synthetic target value with the retrieved context using metrics such as cosine similarity for general semantic closeness, ROUGE-L for textual overlap, BLEU for n-gram precision, BERTScore for embedding-based semantic similarity, and Jaccard similarity for token-level overlap.

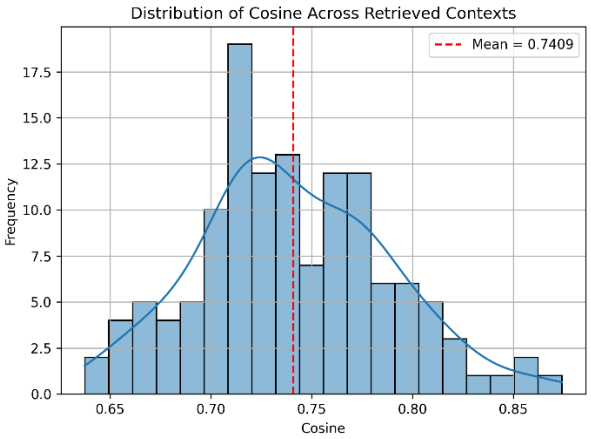
Metric	Mean	Standard Deviation	Minimum	Q1 (25%)	Median (50%)	Q3 (75%)	Maximum
Cosine	0.7409	0.0472	0.6376	0.7136	0.7363	0.7732	0.8741
ROUGE_L	0.0678	0.0104	0.0396	0.0609	0.0702	0.0753	0.0947
BLEU	0.0367	0.0072	0.0106	0.0337	0.0396	0.0411	0.0478
Jaccard	0.1325	0.0385	0.0741	0.1072	0.1205	0.1471	0.2381
BERTScore_P	0.7245	0.0063	0.7127	0.7201	0.7232	0.7272	0.7422
BERTScore_R	0.8595	0.0105	0.8241	0.8527	0.8604	0.8672	0.8794
BERTScore_F1	0.7862	0.0062	0.7644	0.7827	0.7866	0.7907	0.8011

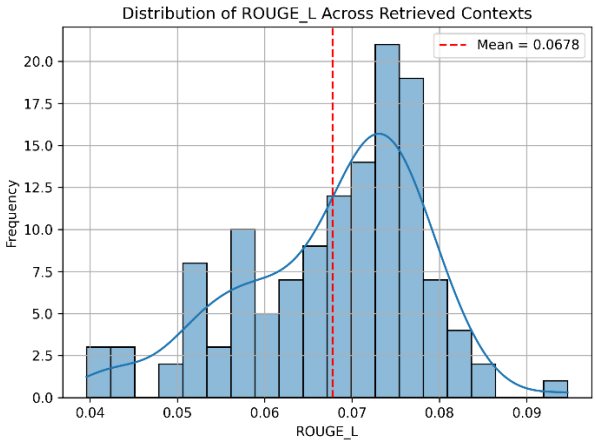
Table 5.5: Descriptive statistics of similarity metrics for retrieved contexts of root cause

Table 5.5 presents descriptive statistics for seven metrics used to evaluate similarity of retrieved NRC conditions. The results show that the Structured RAG retriever consistently retrieves semantically similar context, even with low lexical overlap. Embedding-based metrics, such as cosine similarity and BERTScore are notably high. Cosine similarity has a mean of 0.7409, a standard deviation of 0.0472, and an interquartile range from 0.7136 to 0.7732, with a maximum of 0.8741, indicating close alignment with synthetic target value in embeddings. BERTScore results reveal an average precision of 0.7245, recall of 0.8595, and F1 score of 0.7862, with low standard deviations (0.006-0.011), reflecting consistent, high semantic similarity. The median recall (0.8604) suggests most content words in the synthetic target value NRC conditions are captured, while slightly lower precision indicates some additional, related tokens.

Lexical overlap metrics naturally represent much lower values: ROUGE-L at 0.0678, BLEU at 0.0367, and Jaccard at 0.1325, indicating limited exact token overlap between the synthetic target value and retrieved NRC condition texts. The Jaccard mean of 0.1325, with an interquartile range of 0.1072 to 0.1471, suggests few shared tokens despite important domain terms. This is expected because the synthetic target value is a structured query, whereas the retrieved content is natural language from the ISO knowledge base. Lower BLEU and ROUGE-L scores reflect strict lexical overlap, common when comparing longer retrieved content to short synthetic target values. The combination of high embedding-based similarity and low lexical overlap supports Structured RAG's aim for root-cause analysis, showing that the retriever correctly identifies semantically relevant NRC conditions that reflect the same underlying issue in natural language.







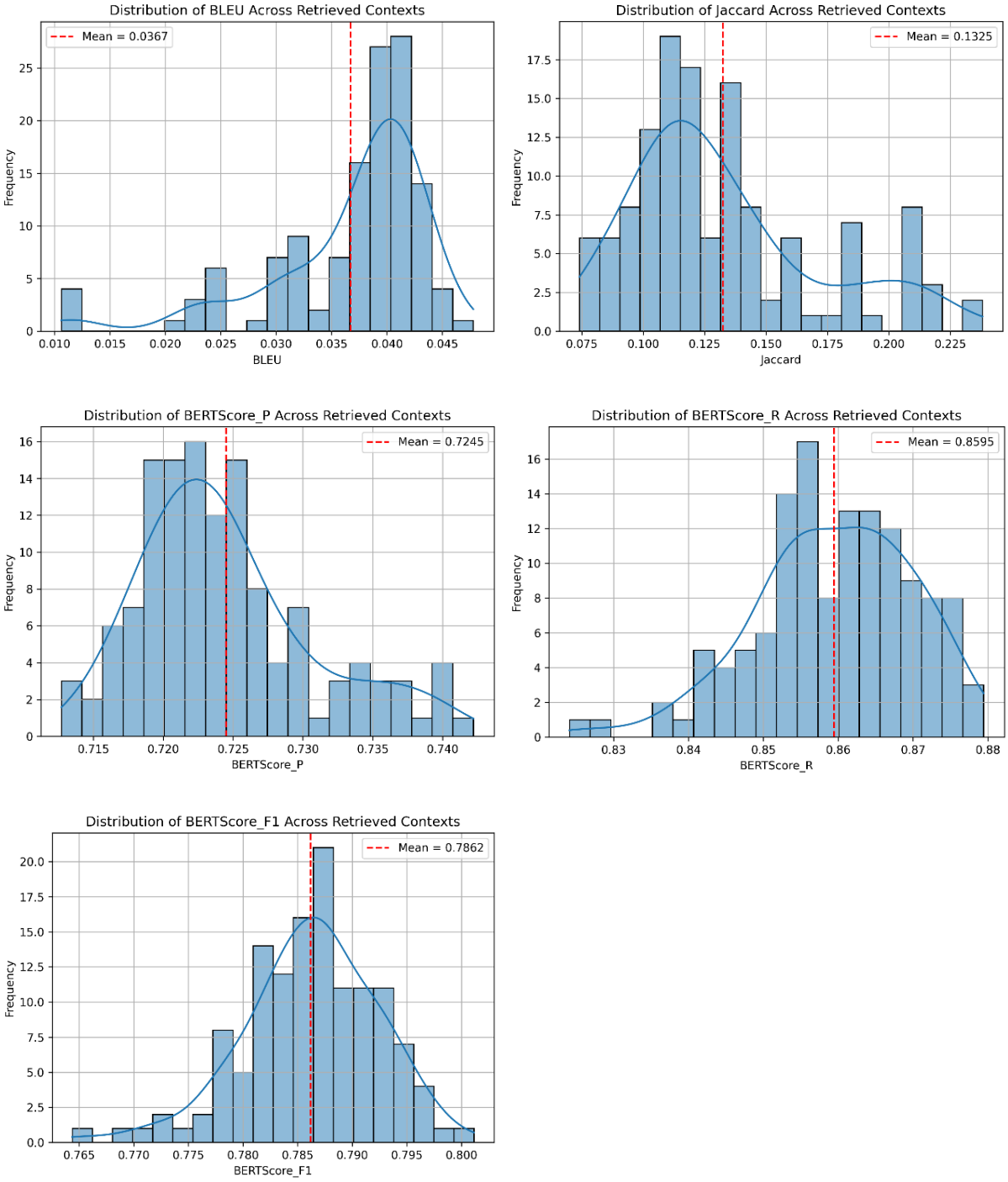


Figure 5.1: Distribution of similarity metrics for retrieved contexts of root cause

5.5 Similarity Evaluation - TinyLlama for Question-Answering

This section evaluates TinyLlama's performance as a troubleshooting assistant across three configurations: without RAG, with Structured RAG, and with Structured RAG and Reranking. The benchmarking involved three different setups and tested 50 queries. Synthetic target values were obtained from reranked documents that matched incident details from a knowledge base for 500 queries. The responses were compared to these synthetic target values using both lexical metrics (ROUGE-L, BLEU, Jaccard) and semantic metrics (Cosine similarity, BERTScore) to measure semantic closeness, text overlap, precision, and token similarity, respectively.

5.5.1 TinyLlama without RAG

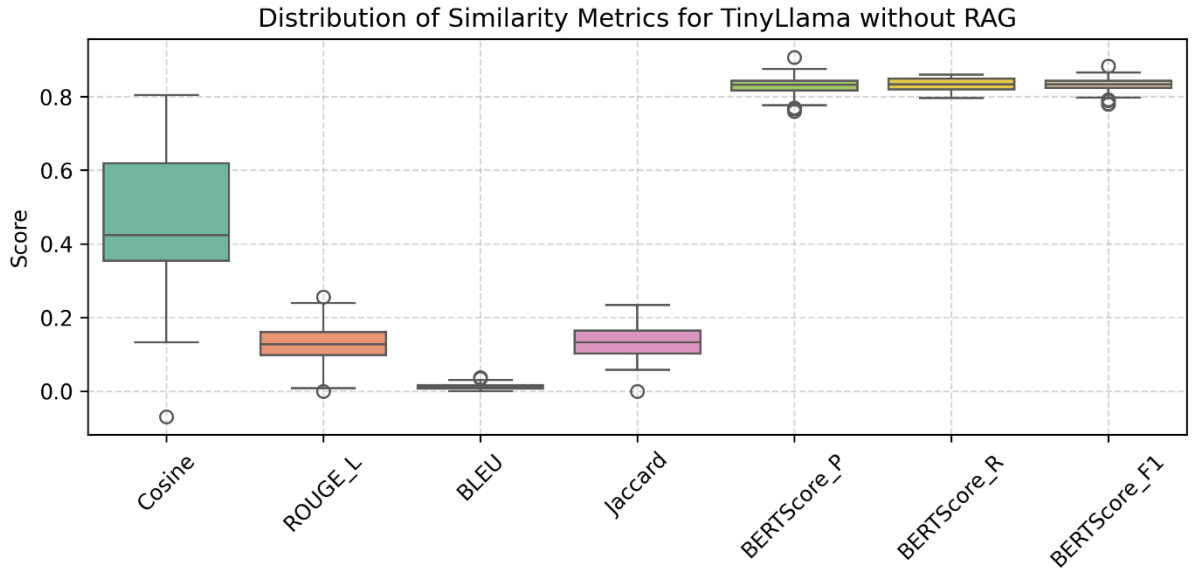
Metric	Mean	Std Dev	Minimum	Q1 (25%)	Median	Q3 (75%)	Maximum
Cosine	0.4586	0.1919	-0.0693	0.3533	0.4234	0.6185	0.8032
ROUGE_L	0.1271	0.0519	0	0.0976	0.1266	0.1598	0.2556
BLEU	0.0129	0.0089	0	0.0068	0.0108	0.0167	0.0368
Jaccard	0.1324	0.0429	0	0.1015	0.1328	0.1636	0.2333
BERTScore_P	0.8285	0.0299	0.7589	0.8158	0.8318	0.8423	0.9069
BERTScore_R	0.8326	0.0172	0.7949	0.8196	0.8332	0.8476	0.8596
BERTScore_F1	0.8303	0.0212	0.7789	0.8226	0.8327	0.842	0.8822

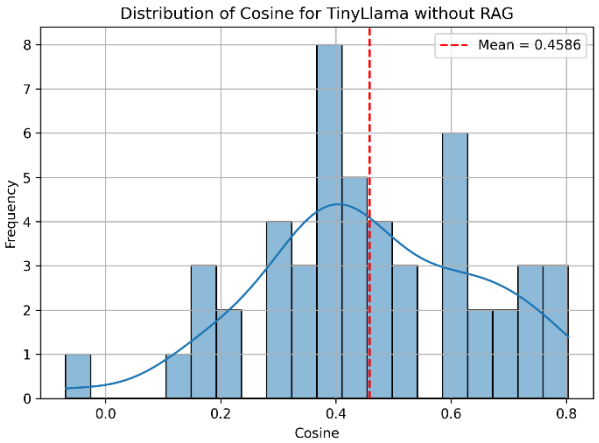
Table 5.6: Descriptive statistics of similarity metrics for responses generated by TinyLlama without RAG

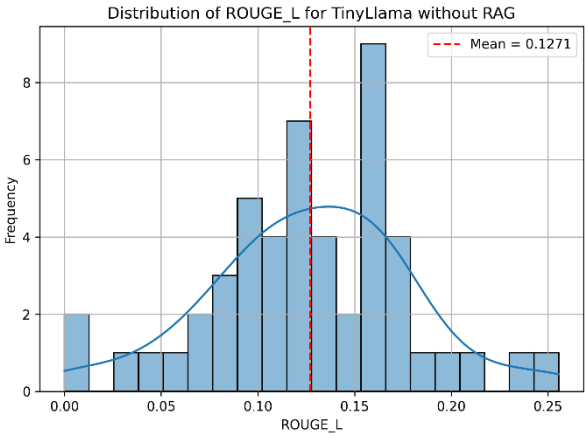
For TinyLlama, Table 5.6's descriptive statistics show mixed responses across the similarity metric. The cosine similarity between generated responses and synthetic target values has a mean of 0.4586, with a standard deviation of 0.1919, ranging from −0.0693 to 0.8032. The median is 0.4234, and the interquartile range spans from 0.3533 to 0.6185. These figures suggest that TinyLlama's global embeddings are only moderately aligned with the synthetic target value, with variability across test cases; some responses are nearly unrelated (cosine near zero or negative), others are close to the synthetic target values. By contrast, lexical overlap metrics like ROUGE-L, BLEU, and Jaccard show low absolute scores: ROUGE-L has a mean of 0.1271 (Q1-Q3 0.098–0.160), BLEU has a mean of 0.0129, and Jaccard has a mean of 0.1324. This suggests the model rarely copies synthetic target value wording, instead producing lexically diverse responses characteristic of paraphrasing models' text.

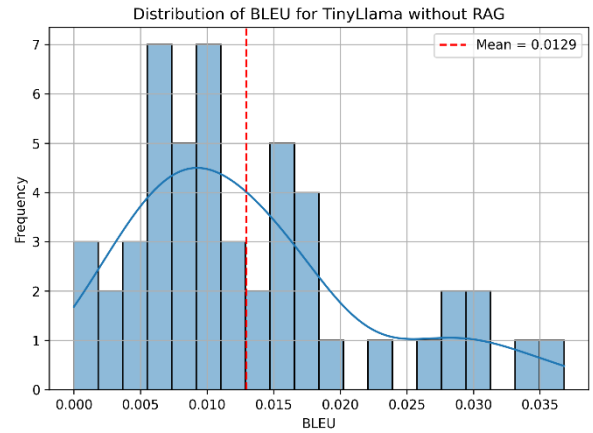
Embedding-based token metrics show high, stable BERTScore precision, recall, and F1 scores, with means of 0.8285, 0.8326, and 0.8303, respectively. Standard deviations are small (0.0172-0.0299), and interquartile ranges are narrow. Minimum values remain above 0.75. These results indicate that TinyLlama's responses are semantically close to synthetic target value and consistently aligned. The higher recall than precision suggests that responses cover most of the synthetic target value, but sometimes include additional tokens or rephrase the synthetic target value.

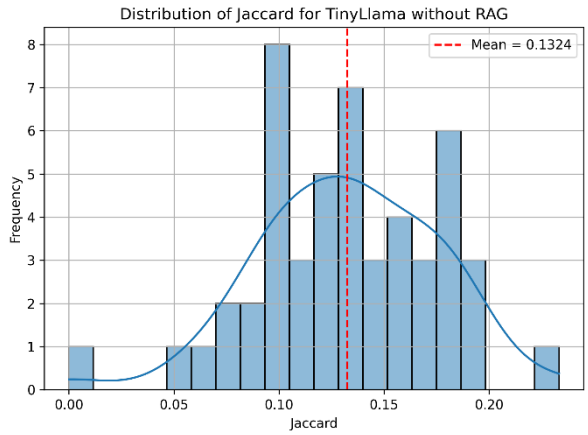
The analysis shows a clear difference between lexical and semantic performance. Lexical metrics show weak but significant overlap, while semantic metrics, especially BERTScore, show strong similarity. Cosine similarity is inconclusive. TinyLlama may not replicate exact wording but generates semantically meaningful responses appropriate for tasks requiring understanding, even if it does not copy the synthetic target value exactly.


Distribution of Similarity Metrics for TinyLlama without RAG
Score
0.0
0.2
0.4
0.6
0.8
Cosine
ROUGE_L
BLEU
Jaccard
BERTScore_P
BERTScore_R
BERTScore_F1


Distribution of Cosine for TinyLlama without RAG
Mean = 0.4586
Frequency
Cosine


Distribution of ROUGE_L for TinyLlama without RAG
Mean = 0.1271
Frequency
ROUGE_L


Distribution of BLEU for TinyLlama without RAG
Mean = 0.0129
Frequency
BLEU


Distribution of Jaccard for TinyLlama without RAG
Mean = 0.1324
Frequency
Jaccard

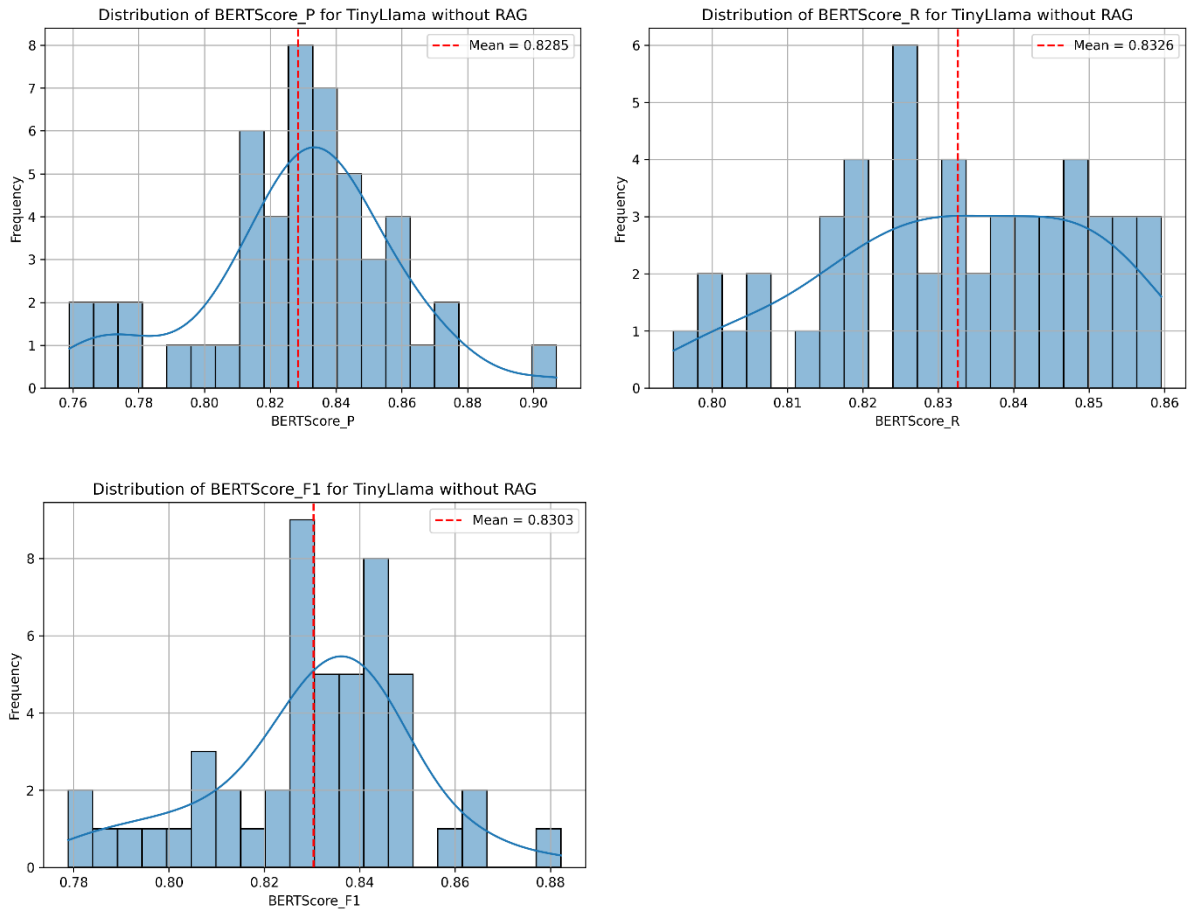


Figure 5.2: Distribution of similarity metrics for responses generated by TinyLlama without RAG

5.5.2 TinyLlama with Structured RAG

Metric	Mean	Std Dev	Minimum	Q1 (25%)	Median	Q3 (75%)	Maximum
Cosine	0.6735	0.173	-0.0659	0.6147	0.7028	0.765	0.9304
ROUGE_L	0.1814	0.155	0	0.1055	0.1481	0.1905	0.8791
BLEU	0.0625	0.1155	0	0.0093	0.0191	0.0597	0.6406
Jaccard	0.2181	0.1298	0	0.1609	0.182	0.2322	0.7674
BERTScore_P	0.8373	0.0443	0.742	0.8073	0.8366	0.8611	0.9676
BERTScore_R	0.8604	0.0315	0.8024	0.8401	0.8568	0.8715	0.9418
BERTScore_F1	0.8482	0.032	0.771	0.8327	0.8465	0.8591	0.9459

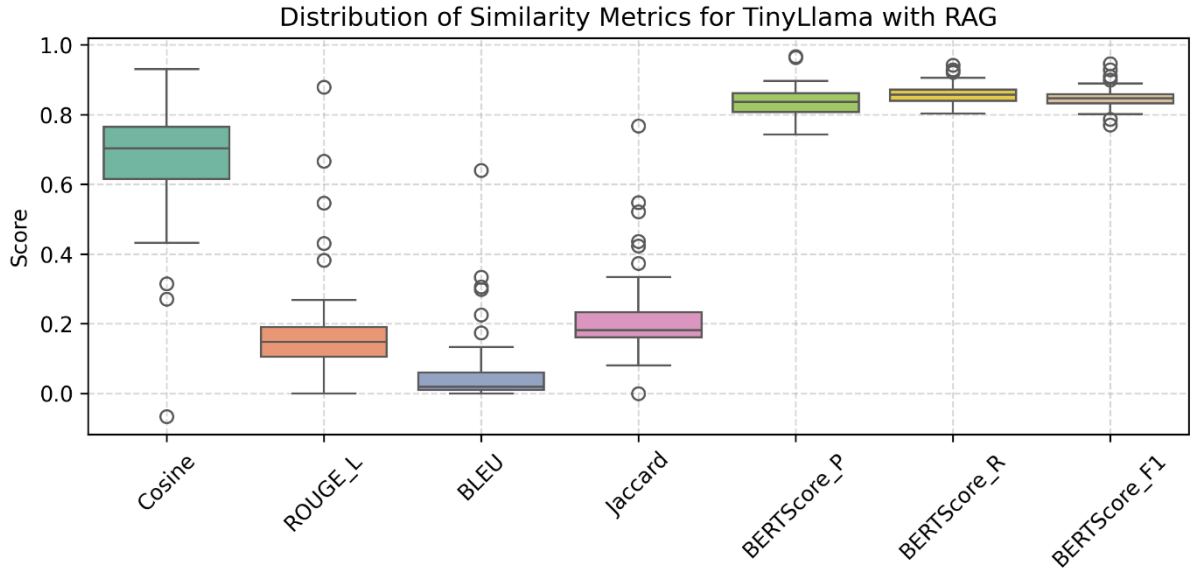
Table 5.7: Descriptive statistics of similarity metrics for responses generated by TinyLlama with Structured RAG

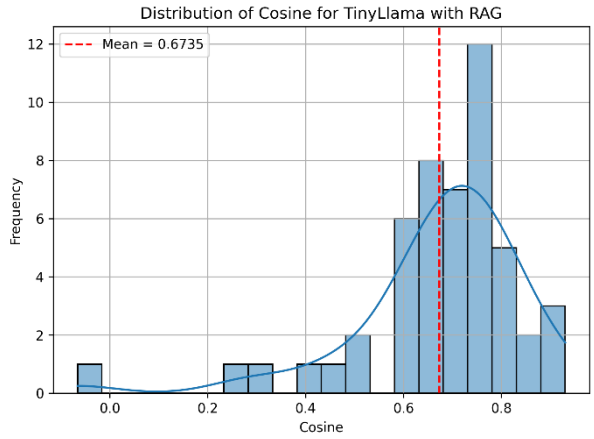
For TinyLlama with Structured RAG, Table 5.7 shows improved semantic similarity between responses and synthetic target values. The mean cosine similarity is 0.6735, with a standard deviation of 0.173, and values ranging from −0.0659 to 0.9304. The median is 0.7028, and the interquartile range spans from 0.6147 to 0.765. Most response embeddings align with the synthetic target value, with many exceeding 0.6. Only a few cases show very low or negative cosine similarity, suggesting that, when grounded in Structured RAG, TinyLlama tends to generate answers that are semantically close to the synthetic target values.

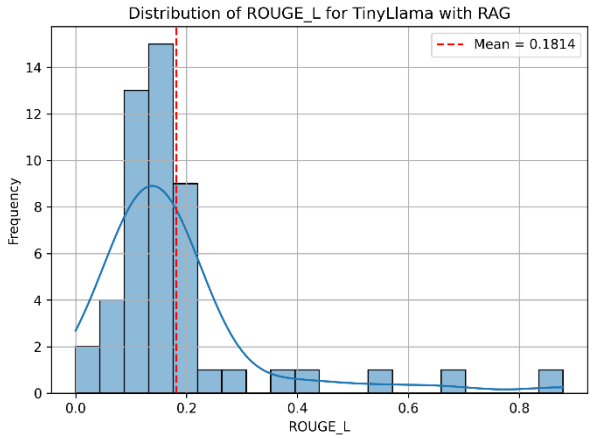
Lexical overlap metrics are modest but higher and more variable than expected for paraphrasing. ROUGE-L has an average of 0.1814 with a standard deviation of 0.155 and a range up to 0.8791;

responses mostly show low to moderate longest common subsequence overlap, though some are very close to the synthetic target values. BLEU has a mean of 0.0625, with a standard deviation of 0.1155, a median of 0.0191, and a maximum of 0.6406, indicating varied phrasing and n-grams, with some responses reusing parts of the synthetic target values. Jaccard token overlap has a mean of 0.2181, standard deviation of 0.1298, a range from 0.1609 to 0.2322, and a maximum of 0.7674. These responses share vocabulary without being basic copies, as expected for retrieved domain-specific data.

The BERTScore metrics for TinyLlama with Structured RAG are consistently high and stable, indicating strong token-level semantic alignment. The mean scores for BERTScore precision, recall, and F1 are 0.8373, 0.8604, and 0.8482, respectively, with standard deviations ranging from 0.0315 to 0.0443. Interquartile ranges are 0.8073–0.8611 for precision, 0.8401–0.8715 for recall, and 0.8327–0.8591 for F1, with all minimum values exceeding 0.74, demonstrating that responses remain semantically close to synthetic target values. The slightly higher recall indicates that the model generally captures most synthetic target value information, although it occasionally introduces related content.







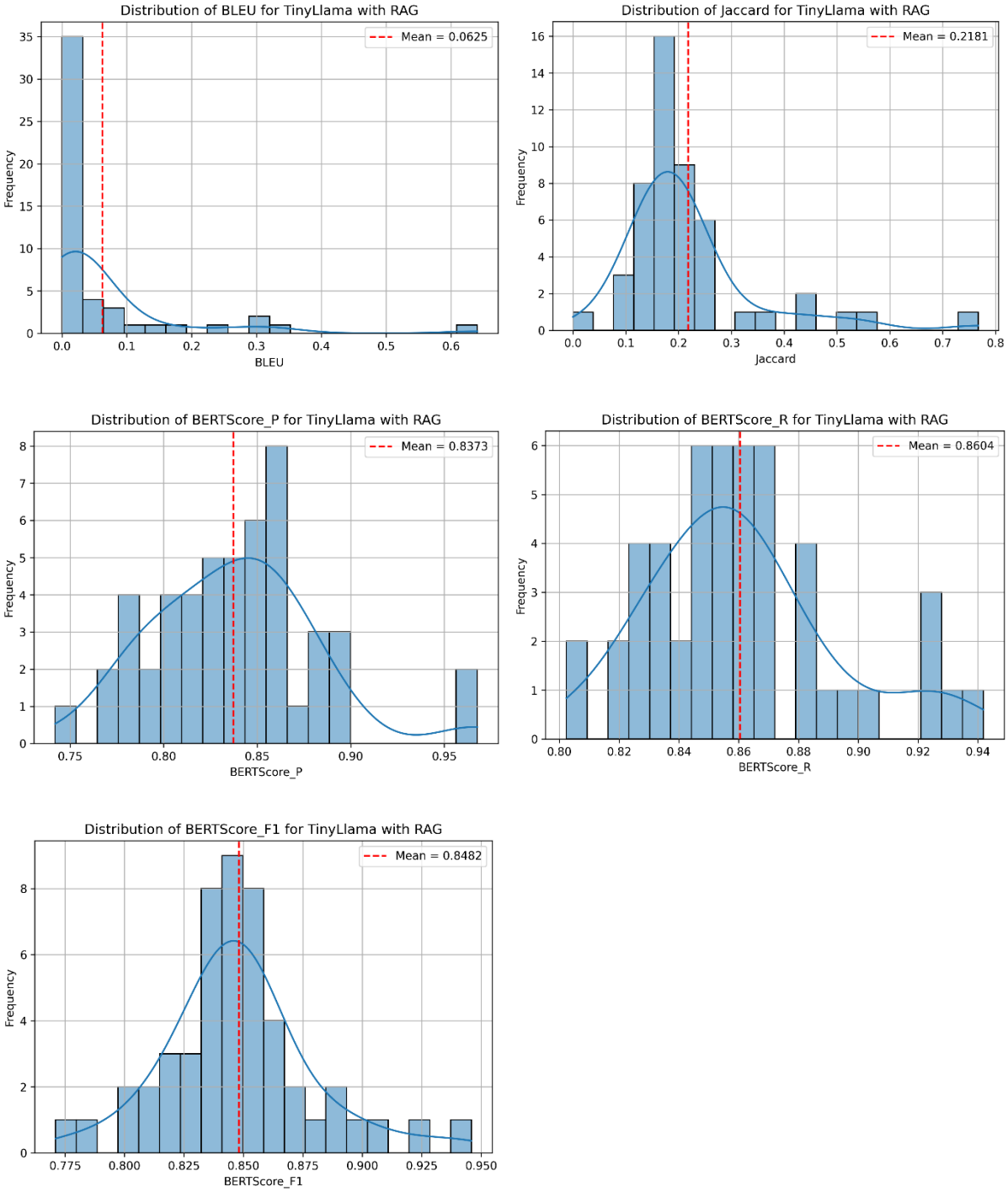


Figure 5.3: Distribution of similarity metrics for responses generated by TinyLlama with RAG

5.5.3 TinyLlama with Structured RAG and Reranking

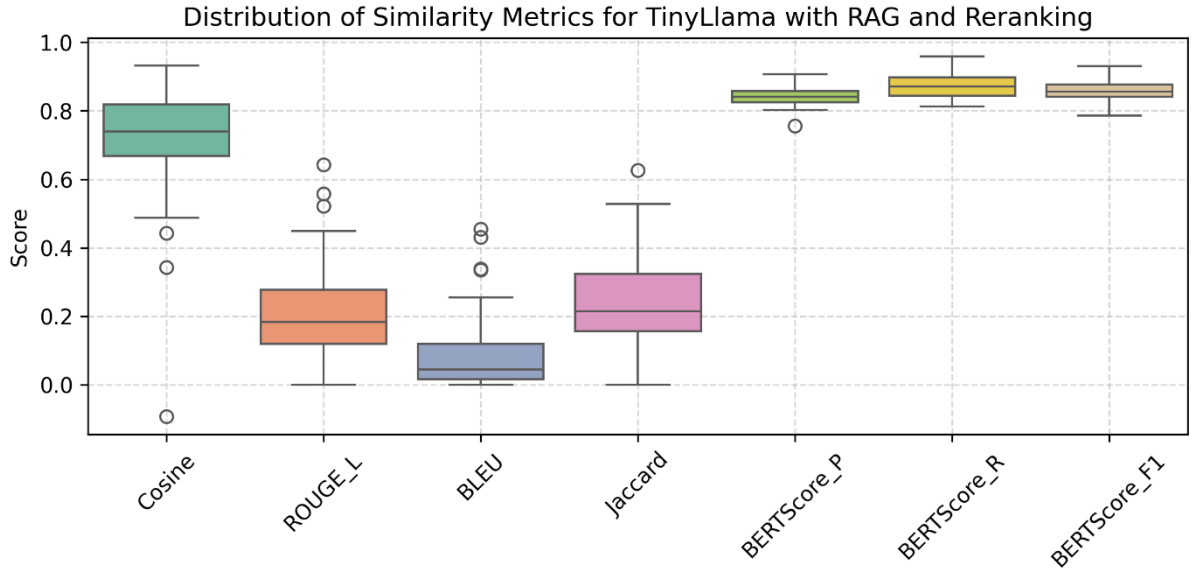
Metric	Mean	Std Dev	Minimum	Q1 (25%)	Median	Q3 (75%)	Maximum
Cosine	0.7172	0.173	-0.0925	0.6685	0.7397	0.8189	0.9325
ROUGE_L	0.2103	0.131	0	0.119	0.1829	0.2771	0.6419
BLEU	0.09	0.1114	0	0.016	0.0439	0.1184	0.455
Jaccard	0.2459	0.1304	0	0.1569	0.2149	0.3226	0.6264
BERTScore_P	0.8434	0.0288	0.7555	0.8248	0.8412	0.8581	0.907
BERTScore_R	0.8734	0.0355	0.8124	0.8444	0.8707	0.8972	0.9596

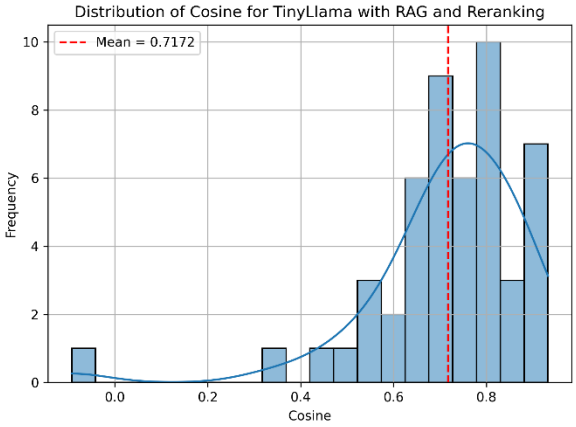
BERTScore_F1	0.858	0.0296	0.7863	0.8404	0.8552	0.8773	0.9309

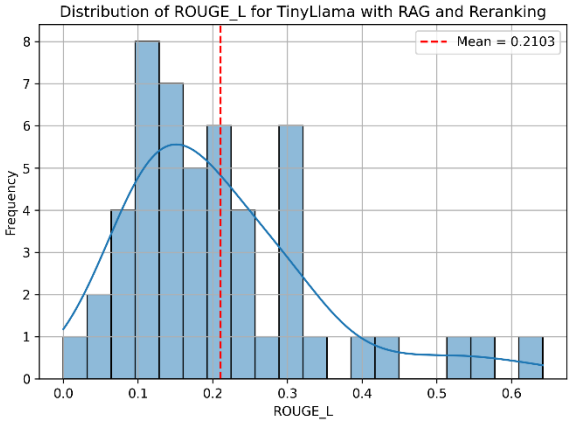
Table 5.8: Descriptive statistics of similarity metrics for responses generated by TinyLlama with Structured RAG and Reranking

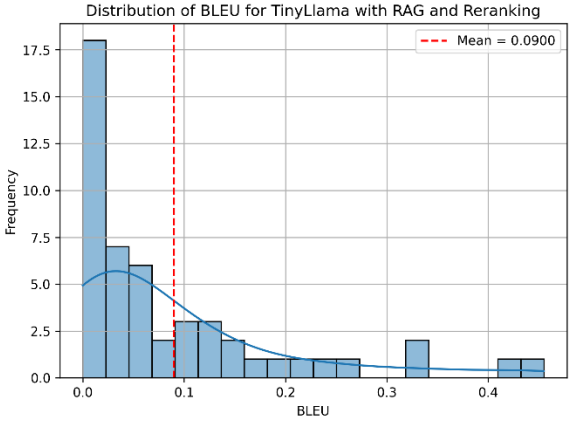
For TinyLlama with Structured RAG and reranking, Table 5.8 suggests that generated responses generally align with synthetic target values at the semantic level, with moderate lexical variation. The average cosine similarity is 0.7172, with a standard deviation of 0.173, ranging from −0.0925 to 0.9325; the median is 0.7397. Most responses score well above 0.65, indicating that sentence-level representations are close to the synthetic target value, with some responses near zero or negative. Lexical metrics vary: ROUGE-L averages 0.2103 (standard deviation 0.131), with a median between 0.1190 and 0.2771, and a maximum of 0.6419, showing low to moderate overlap, with some high values. BLEU median is 0.0439 (mean 0.09, standard deviation 0.1114), with a maximum of 0.455, suggesting responses often differ in local n-grams but sometimes reuse phrases. Jaccard similarity has a mean of 0.2459, a median between 0.1569 and 0.3226, and a maximum of 0.6264, indicating the model shares vocabulary but rephrases retrieved content.

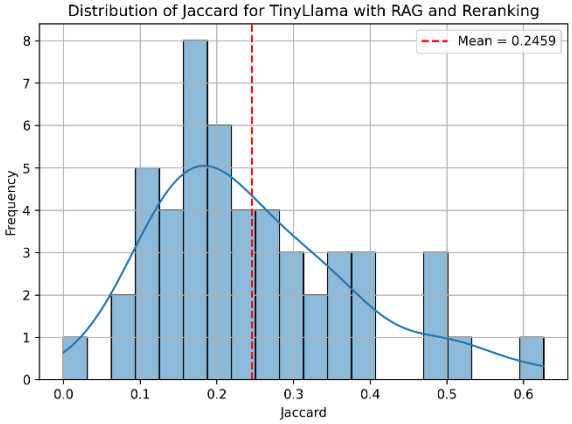
Table 5.8 presents BERTScore metrics, confirming high and consistent semantic similarity at the token level. The average scores for precision, recall, and F1 are 0.8434, 0.8734, and 0.858, respectively, with small standard deviations (0.0288-0.0355) and narrow interquartile ranges. Minimum values exceeding 0.9 indicate that even the worst cases are close to synthetic target values in the embedding space. The slightly higher recall compared with precision suggests that responses tend to include most of the synthetic target value information, sometimes adding semantically related content. This is beneficial for a RAG-based troubleshooting assistant that relies on retrieved knowledge for accurate answers rather than hallucinated content.

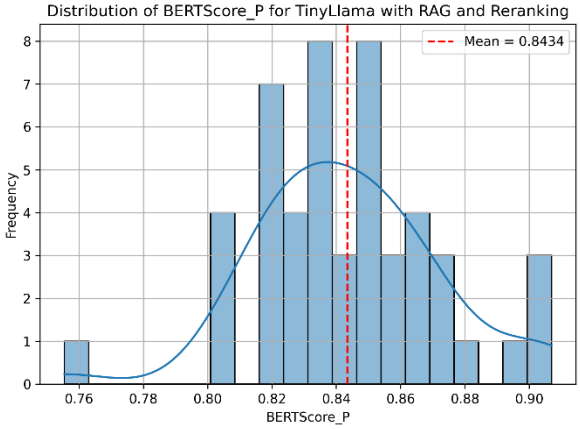


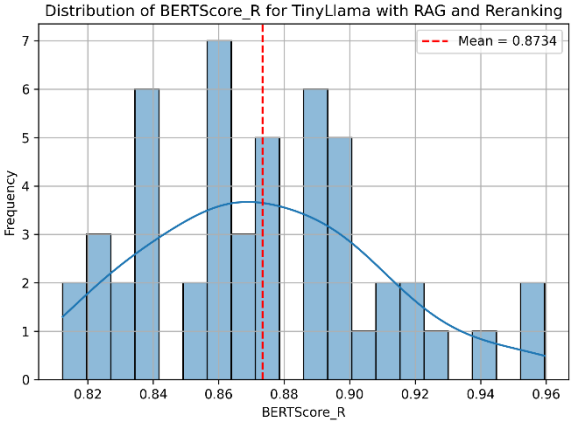

Distribution of Cosine for TinyLlama with RAG and Reranking
Mean = 0.7172
Frequency
Cosine


Distribution of ROUGE_L for TinyLlama with RAG and Reranking
Mean = 0.2103
Frequency
ROUGE_L


Distribution of BLEU for TinyLlama with RAG and Reranking
Mean = 0.0900
Frequency
BLEU


Distribution of Jaccard for TinyLlama with RAG and Reranking
Mean = 0.2459
Frequency
Jaccard


Distribution of BERTScore_P for TinyLlama with RAG and Reranking
Mean = 0.8434
Frequency
BERTScore_P


Distribution of BERTScore_R for TinyLlama with RAG and Reranking
Mean = 0.8734
Frequency
BERTScore_R

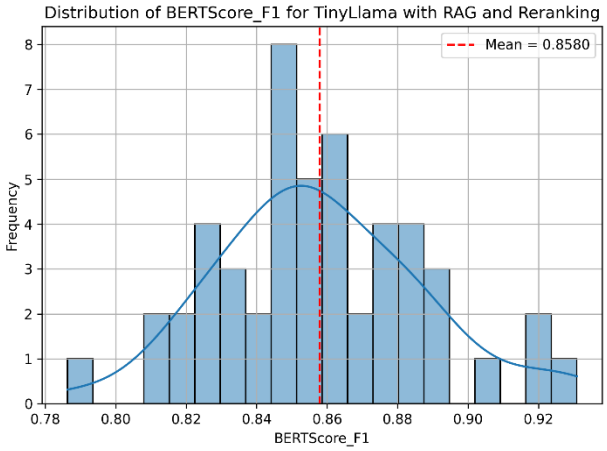


Figure 5.4: Distribution of similarity metrics for responses generated by TinyLlama with RAG and Reranking

5.5.4 Hypothesis Testing

To perform hypothesis testing, we follow a systematic process to assess whether the performance differences among three variants of the same base model: Plain TinyLlama, Structured RAG-based TinyLlama, and Structured RAG-based TinyLlama with Reranking are statistically significant. This involves defining the null hypothesis (H0), the alternative hypothesis (H1), selecting a significance level (α), calculating the F-statistic, deriving the p-value, and making a conclusion. H0 assumes no difference, while H1 suggests there is a difference. We assume the mean similarity score is 0.5 under H0 and differs from 0.5 under H1. The significance level is set at $\alpha = 0.05$. The p-value indicates the probability of obtaining the observed results if H0 is true. If $p < \alpha$, we reject H0, indicating that the results are statistically significant.

To evaluate the performance differences among Plain TinyLlama, Structured RAG-based TinyLlama, and Reranked Structured RAG-based TinyLlama, we conduct a one-way ANOVA on their raw similarity scores for each metric. This analysis assesses whether the mean scores vary across the groups, aligning with our goal for the troubleshooting assistant. For each similarity measure (Cosine, ROUGE-L, BLEU, Jaccard, BERTScore), we examine if all configurations have similar average scores or if at least one differs significantly. If the results are significant ($p < 0.05$), we perform Tukey's HSD post-hoc tests for specific pairs: Plain vs Structured RAG, Plain vs Structured RAG with Reranking, and Structured RAG vs Structured RAG with Reranking. This method helps us understand whether RAG and reranking enhance performance over the baseline and their individual contributions to retrieval-based generation accuracy.

Metric	F-statistic	p-value	Interpretation
Cosine	29.7302	0	Significant difference
ROUGE_L	6.1108	0.002823	Significant difference
BLEU	8.8468	0.000236	Significant difference
Jaccard	14.7148	0.000001	Significant difference
BERTScore_P	2.3091	0.102949	Not statistically significant
BERTScore_R	25.5459	0.000000	Significant difference
BERTScore_F1	12.5558	0.000009	Significant difference

Table 5.9: One-Way ANOVA results across Plain TinyLlama, Structured RAG-based TinyLlama, Structured RAG-based TinyLlama with Reranking

Table 5.9 presents the One-Way ANOVA results comparing similarity metrics across three TinyLlama configurations: Plain TinyLlama, Structured RAG-based TinyLlama, and Reranked

Structured RAG-based TinyLlama. For each metric, the null hypothesis (H0) states that all configurations have equal means, while the alternative hypothesis (H1) suggests that at least one differs.

The F-statistic for cosine similarity is 29.73, with a p-value of 0, indicating a statistically significant difference in mean cosine similarity across the configurations. Similarly, ROUGE-L ($F = 6.11$, $p = 0.0028$), BLEU ($F = 8.85$, $p = 0.00024$), and Jaccard similarity ($F = 14.71$, $p = 1.00E-06$) also show statistically significant differences. These findings suggest that exact n-gram and token overlap metrics vary across configurations. The high F-values for cosine similarity and Jaccard further imply that retrieval and grounding strategies have strong effects on semantic alignment and shared content vocabulary.

BERTScore_P (precision) has an F-statistic of 2.31 and a p-value of approximately 0.103, which exceeds $\alpha = 0.05$, indicating that we cannot reject the null hypothesis. This suggests no significant difference in mean BERTScore precision across configurations, indicating similar levels of semantic alignment in generated responses. On the other hand, BERTScore_R (recall) and BERTScore_F1 show significant differences, with F-statistics of 25.55 and 12.56, and p-values around 0 and 0.000009, respectively. These findings indicate that configurations vary in coverage and overall similarity, with some, such as those using Structured RAG and reranking, more effectively capturing synthetic target values, resulting in higher recall and F1 scores.

The ANOVA results show that most similarity metrics are influenced by retrieval and grounding strategies. Significant differences are observed in Cosine, ROUGE-L, BLEU, Jaccard, BERTScore_R, and BERTScore_F1, while BERTScore_P shows no notable difference. This suggests that structured RAG and reranking primarily enhance synthetic target value coverage and semantic similarity.

To sum up, the Structured RAG-based model and its Reranked variants both significantly outperform the plain model. The Structured RAG-based model with Reranking achieves the best results across most metrics. Semantic faithfulness, measured by BERTScore_P, remains consistently high across all variants, with only minor differences. This suggests that retrieval augmentation, particularly reranking, improves troubleshooting quality with TinyLlama model.

5.6 Similarity Evaluation - OpenAI for Question-Answering

This section evaluates the OpenAI model's performance as a troubleshooting assistant across three setups: the base model without retrieval, with Structured RAG, and with Reranked Structured RAG. It uses both lexical metrics (ROUGE-L, BLEU, Jaccard) and semantic metrics (Cosine similarity, BERTScore) to evaluate output quality. Synthetic target values were derived from retrieved documents aligned with incident titles, covering problem description, symptoms, root cause, and solutions, for 500 queries. Responses from each setup were compared with synthetic target values using Cosine similarity for semantics, ROUGE-L for text overlap, BLEU for n-gram accuracy, BERTScore for embeddings, and Jaccard for token overlap. This evaluation framework highlights the effects of RAG and reranking on response quality.

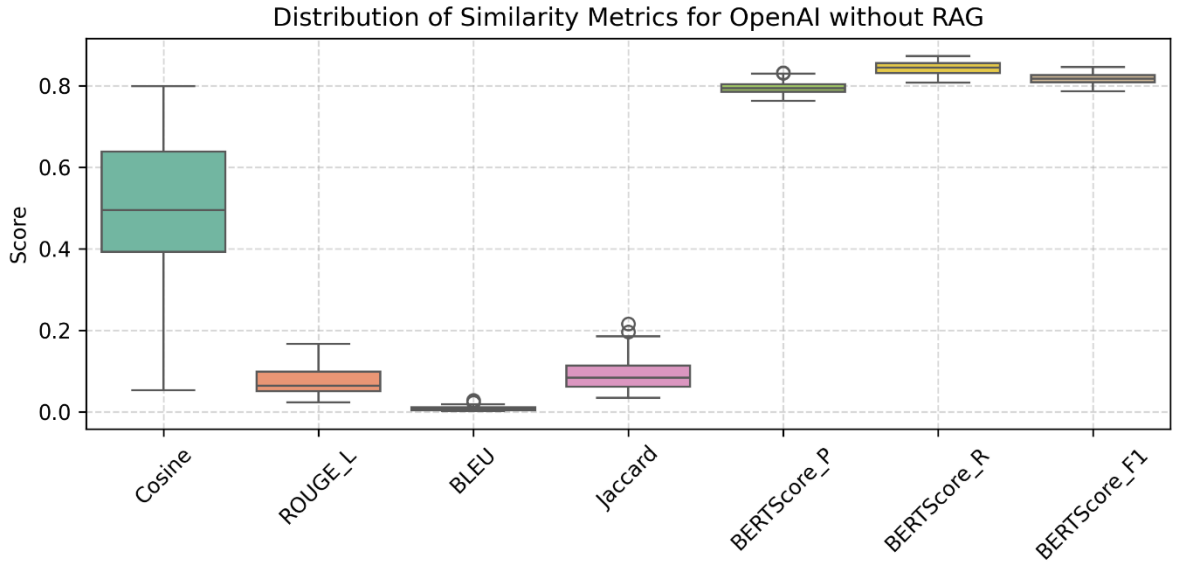
5.6.1 OpenAI without RAG

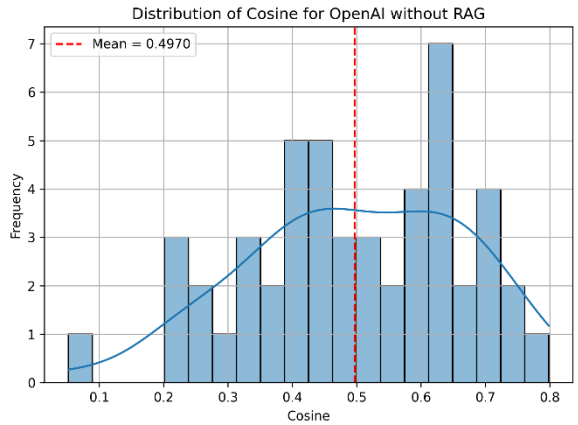
Metric	Mean	Std Dev	Minimum	Q1 (25%)	Median	Q3 (75%)	Maximum
Cosine	0.4970	0.1691	0.0523	0.3923	0.4946	0.6380	0.7985
ROUGE_L	0.0761	0.0369	0.0231	0.0499	0.0637	0.0986	0.1667
BLEU	0.0075	0.0053	0.0015	0.0039	0.0058	0.0101	0.0276
Jaccard	0.0952	0.0436	0.0346	0.0614	0.0841	0.1128	0.2158
BERTScore_P	0.7954	0.0155	0.7628	0.7853	0.7930	0.8035	0.8315
BERTScore_R	0.8418	0.0166	0.8073	0.8303	0.8439	0.8557	0.8723
BERTScore_F1	0.8179	0.0140	0.7868	0.8078	0.8166	0.8261	0.8450

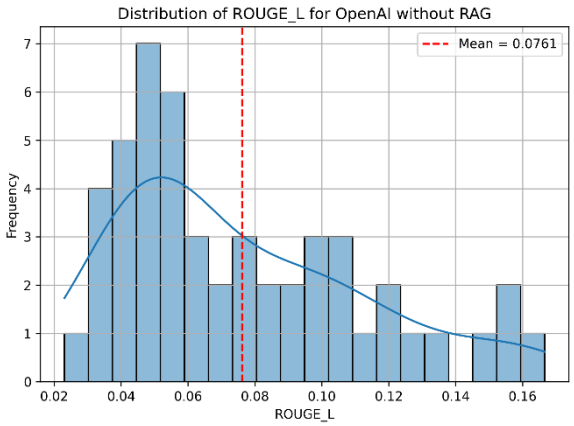
Table 5.10: Descriptive statistics of similarity metrics for responses generated by OpenAI

The OpenAI model without RAG (Table 5.10) generates responses that are semantically similar to the synthetic target values, despite limited word overlap, as it tends to paraphrase rather than copy. The average cosine similarity is 0.4970, with most values above 0.4, indicating that the model effectively captures the intended meanings. ROUGE-L, BLEU, and Jaccard scores are low, suggesting rare exact text reuse, which aligns with natural language generation. BERTScore metrics, precision, recall, and F1, range from 0.79 to 0.84, showing that responses include key content and sometimes add extra words to the text.

The combination of relatively high cosine similarity, low lexical overlap, and strong BERTScore values suggests that the OpenAI model's responses are semantically similar to synthetic target values (high cosine similarity and BERTScore) but differ in exact wording (low ROUGE-L, BLEU, and Jaccard). It reliably provides accurate diagnostic information in natural language, as expected from a generative troubleshooting assistant.







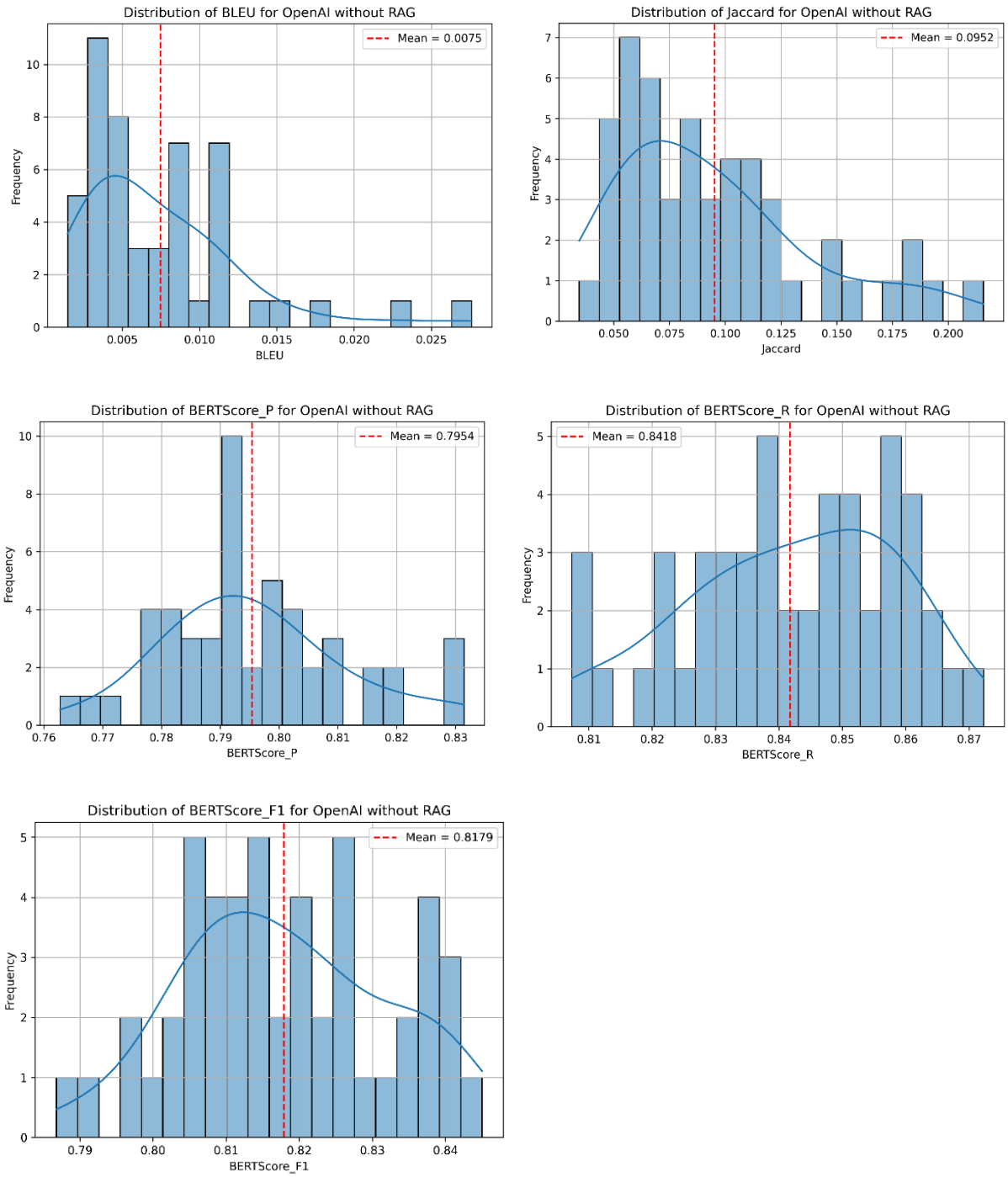


Figure 5.5: Distribution of similarity metrics for responses generated by OpenAI without RAG

5.6.2 OpenAI with Structured RAG

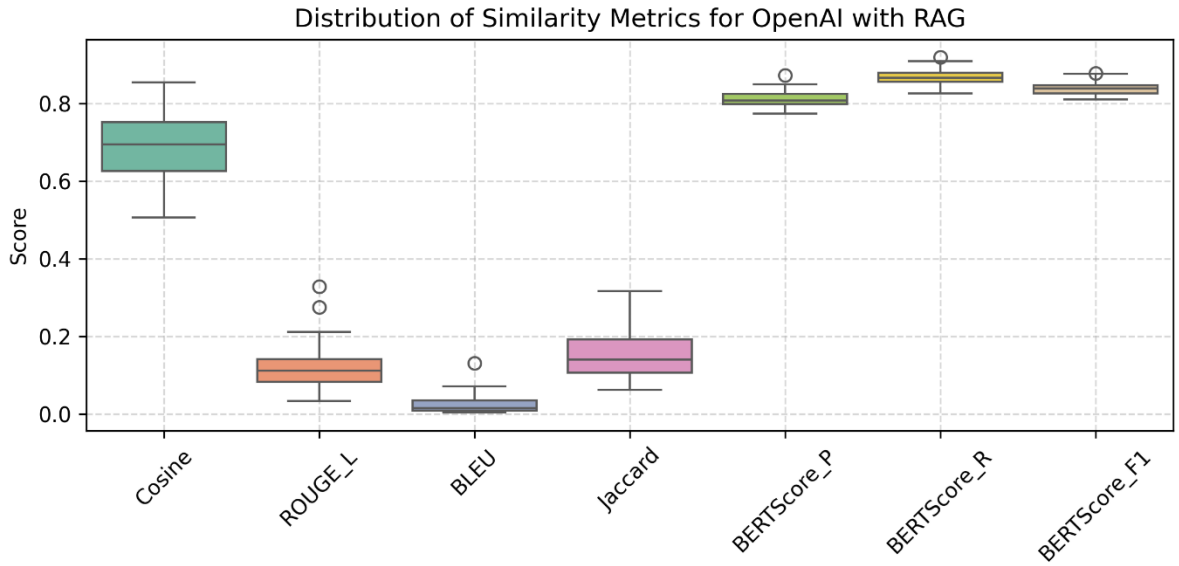
Metric	Mean	Std Dev	Minimum	Q1 (25%)	Median	Q3 (75%)	Maximum
Cosine	0.688	0.0948	0.5058	0.6258	0.694	0.7509	0.8545
ROUGE_L	0.1206	0.0565	0.0346	0.084	0.1121	0.1425	0.3282
BLEU	0.0243	0.0228	0.0038	0.0098	0.016	0.0351	0.1311
Jaccard	0.1537	0.0595	0.0625	0.1071	0.141	0.1928	0.3173
BERTScore_P	0.812	0.0209	0.7739	0.7982	0.8077	0.8242	0.8722
BERTScore_R	0.8682	0.0188	0.8259	0.8556	0.8661	0.8784	0.9184

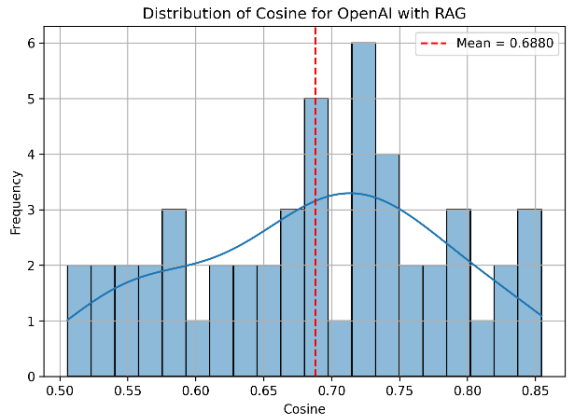
BERTScore_F1	0.839	0.0176	0.8094	0.8254	0.8384	0.846	0.8776

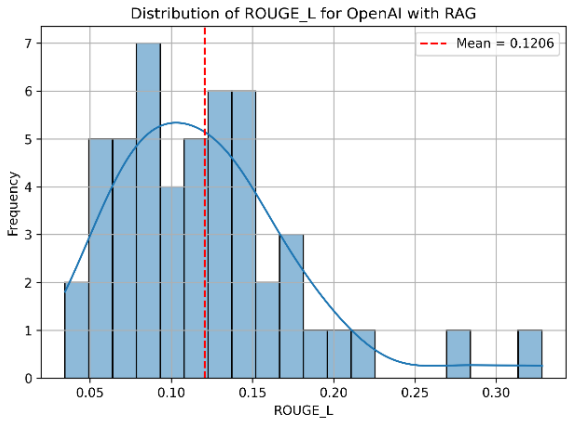
Table 5.11: Descriptive statistics of similarity metrics for responses generated by OpenAI with Structured RAG

Regarding the OpenAI model with Structured RAG (Table 5.11), statistics indicate that responses align well with synthetic target values in terms of meaning, showing moderate lexical overlap. The mean cosine similarity is 0.688, mostly above 0.6, with a median of 0.694 and an interquartile range from 0.6258 to 0.7509, suggesting some responses have lower similarity but generally remain close to the synthetic target value meaning. ROUGE-L averages 0.1206, BLEU 0.0243, and Jaccard 0.1537, showing keyword reuse and rephrasing. Structured RAG incorporates domain-specific terms, enabling diverse language use while maintaining relevance. BERTScore scores (precision, recall, F1) range from 0.81 to 0.87 with low variation, and even lower scores still indicate high semantic overlap. Recall exceeds precision, showing the model captures the most important context but also adds extra details in responses.

These results show that the OpenAI model with Structured RAG delivers answers based on retrieved knowledge, remaining semantically faithful to synthetic target values and sharing moderate similarity in wording. This balance makes it effective for a troubleshooting assistant by combining accuracy with natural language phrasing.







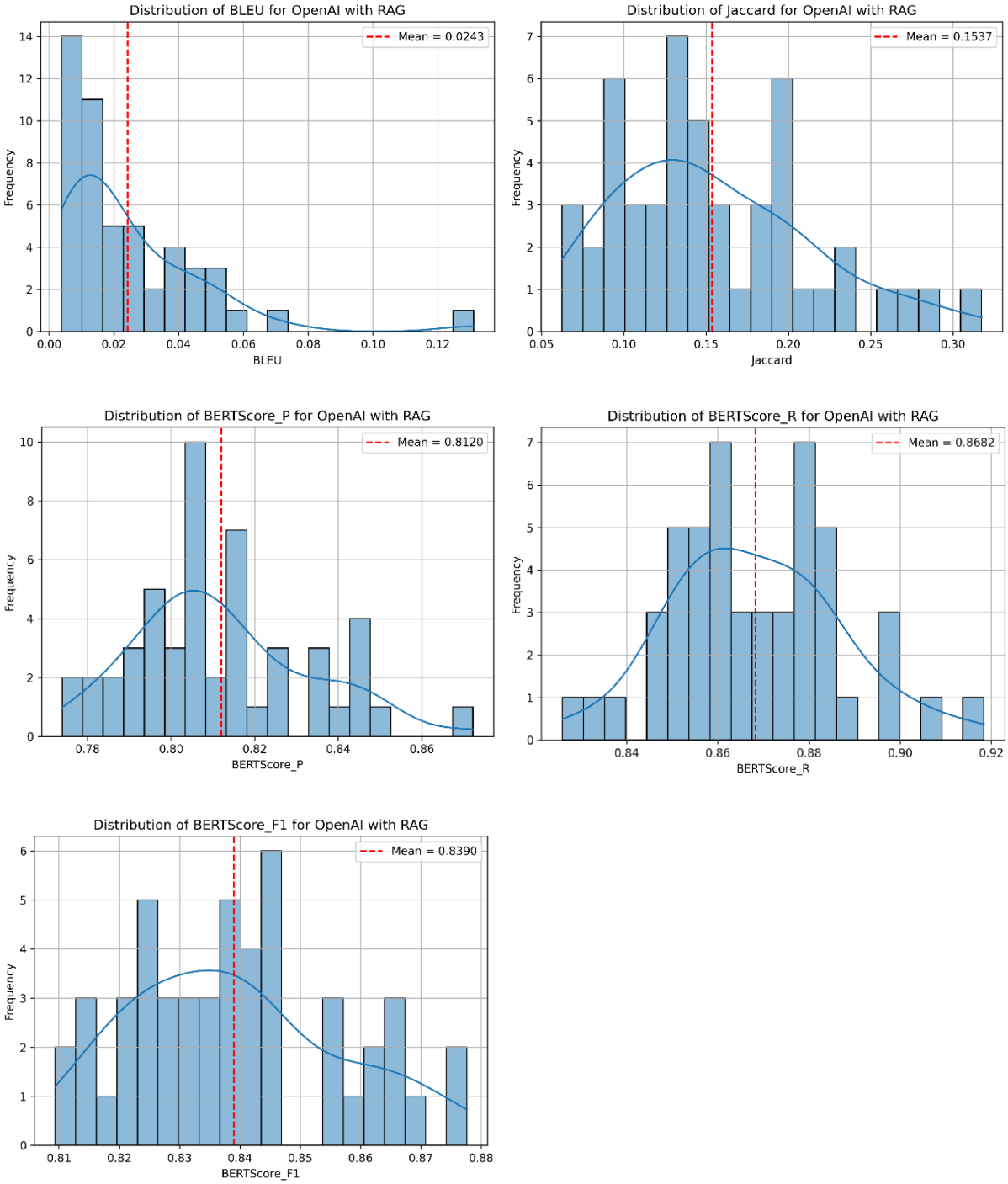


Figure 5.6: Distribution of similarity metrics for responses generated by OpenAI with RAG

5.6.3 OpenAI with Structured RAG and Reranking

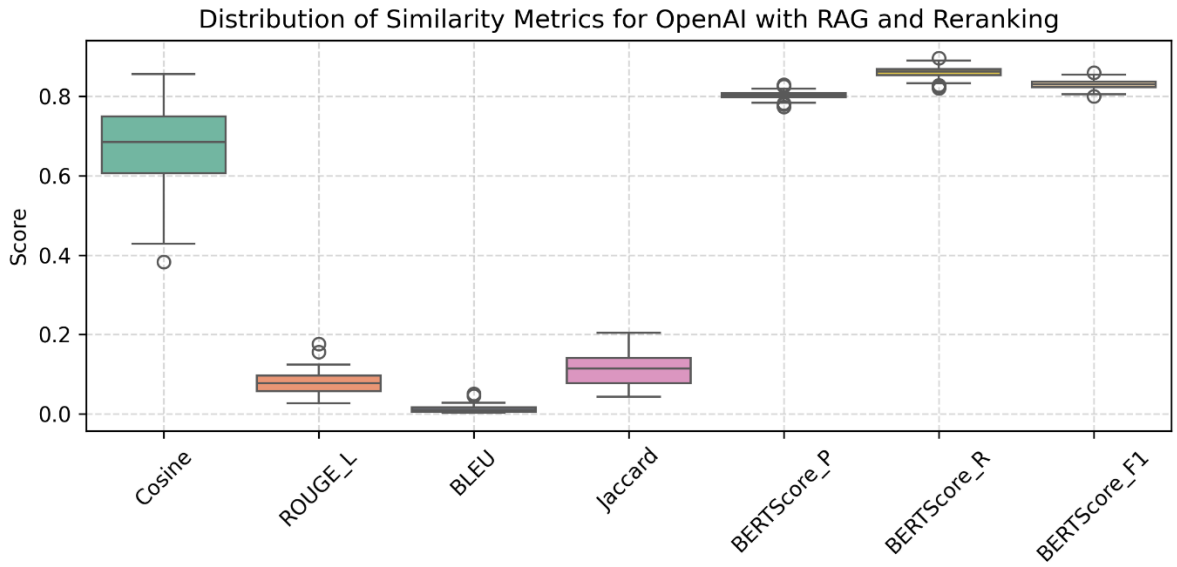
Metric	Mean	Std Dev	Minimum	Q1 (25%)	Median	Q3 (75%)	Maximum
Cosine	0.6661	0.1122	0.3827	0.6059	0.684	0.7493	0.8561
ROUGE_L	0.0787	0.0312	0.027	0.0576	0.0781	0.0968	0.177
BLEU	0.0122	0.0099	0.0024	0.0056	0.01	0.0167	0.0517
Jaccard	0.1117	0.0396	0.0432	0.078	0.1144	0.1408	0.2038
BERTScore_P	0.8025	0.0115	0.773	0.7977	0.8029	0.8079	0.8294
BERTScore_R	0.8601	0.0161	0.8194	0.8526	0.8615	0.8679	0.8957

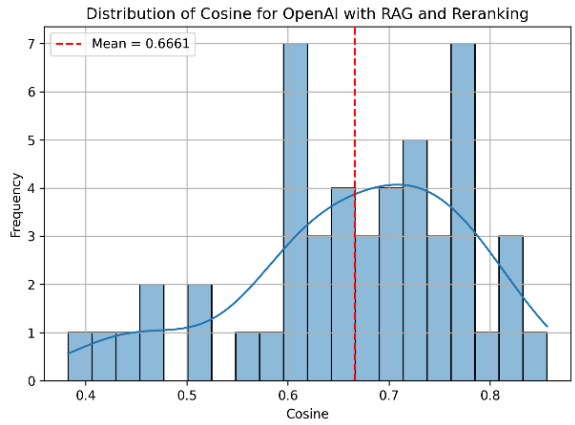
BERTScore_F1	0.8302	0.0124	0.7991	0.8231	0.8297	0.8366	0.859

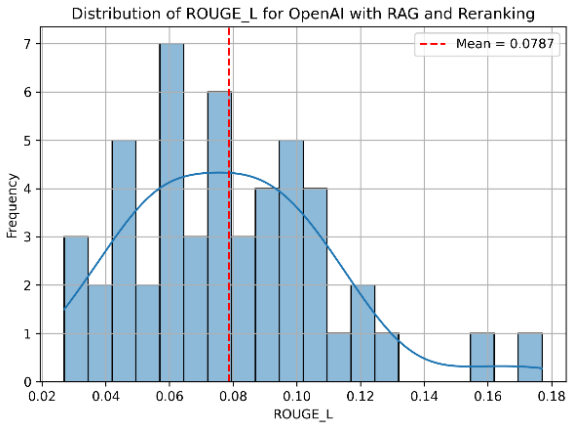
Table 5.12: Descriptive statistics of similarity metrics for responses generated by OpenAI with Structured RAG and Reranking

The OpenAI model with reranked Structured RAG generates responses that align closely in meaning with synthetic target values, even though exact word matches are rare. The average cosine similarity is 0.6661, mostly above 0.6, demonstrating strong semantic similarity. Lexical overlap remains low, with ROUGE-L at 0.0787, BLEU at 0.0122, and Jaccard token overlap at 0.1117, indicating the model paraphrases without copying long sequences. BERTScore metrics are robust, with precision, recall, and F1 scores ranging from 0.80 to 0.86, showing low variability and responses that consistently capture core meanings. The higher recall compared to precision suggests the model covers most relevant content and adds appropriate information details.

The results indicate that the OpenAI model with reranked Structured RAG delivers precise, contextually relevant answers with natural variation in phrasing. It avoids template reuse and stays accurate and grounded. This aligns perfectly with the goals of a generative troubleshooting assistant.







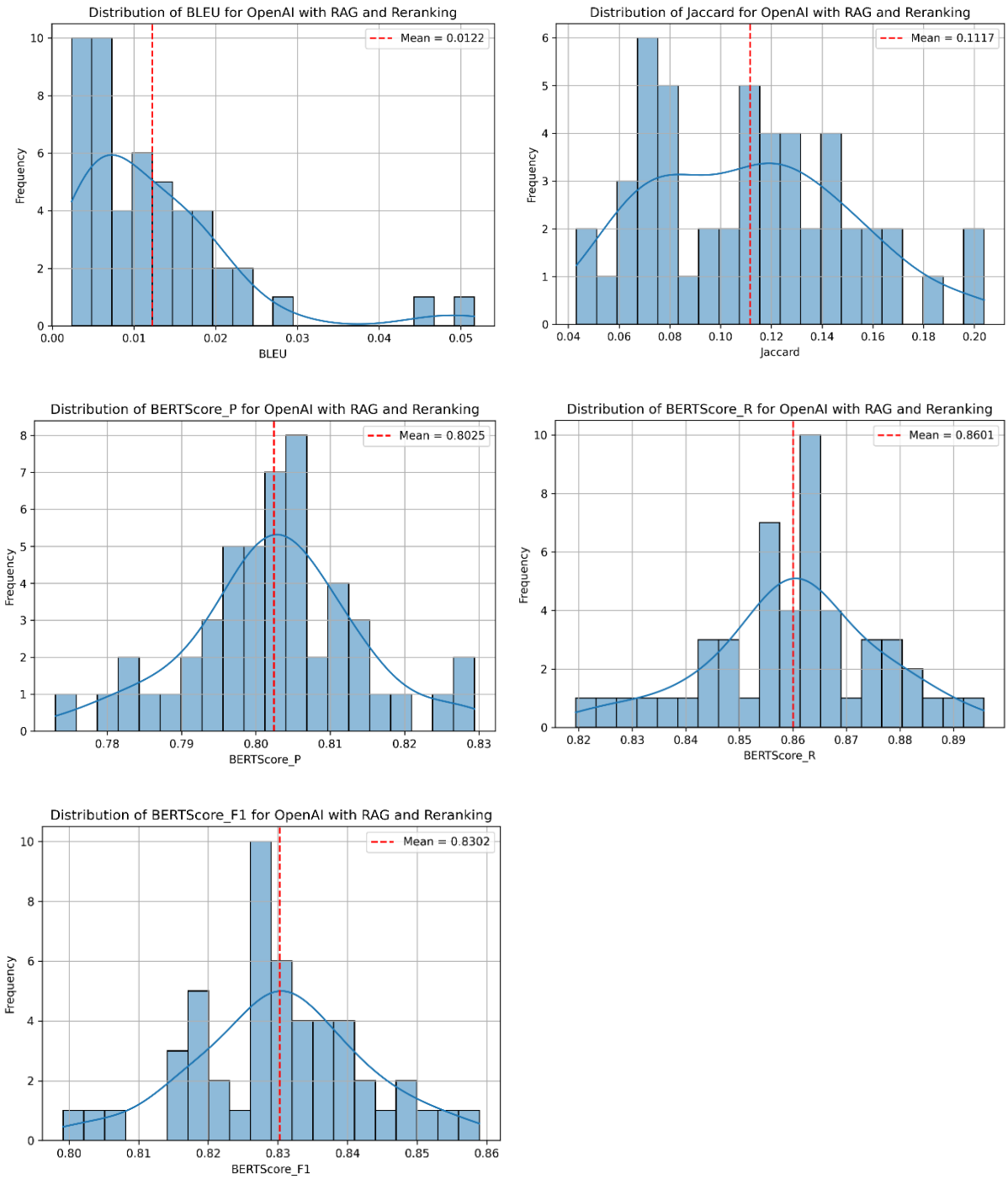


Figure 5.7: Distribution of similarity metrics for responses generated by OpenAI with RAG and Reranking

5.6.4 Hypothesis Testing

Metric	F-statistic	p-value	Interpretation
Cosine	32.7034	0.000000	Significant difference
ROUGE_L	16.9183	0.000000	Significant difference
BLEU	17.4828	0.000000	Significant difference
Jaccard	19.4192	0.000000	Significant difference
BERTScore_P	12.8420	0.000000	Significant difference

BERTScore_R	30.9190	0.000000	Significant difference
BERTScore_F1	25.7253	0.000000	Significant difference

Table 5.13: One-Way ANOVA results across Plain OpenAI, Structured RAG-based OpenAI, Structured RAG-based OpenAI with Reranking

This table (Table 5.13) summarizes a one-way ANOVA comparing similarity scores across three OpenAI configurations: plain OpenAI, OpenAI with Structured RAG, and OpenAI with Structured RAG and Reranking. For each similarity metric, the null hypothesis H0 states that all three setups have equal mean performance, while the alternative hypothesis H1 suggests that at least one differs.

The F-statistic for cosine similarity is 32.7034 with a p-value of 0, indicating statistically significant differences among the three configurations in average sentence-level embedding similarity. Meanwhile, lexical overlap metrics such as ROUGE-L, BLEU, and Jaccard show high F-statistics (16.9183, 17.4828, 19.4192) and p-values of 0, indicating that overlap varies across model types. Embedding-based metrics, including BERTScore precision, recall, and F1, also show statistically significant differences, with high F-values and p-values of 0. This suggests that grounding strategies notably influence semantic quality. Particularly, structured RAG and reranking improve both response coverage and semantic quality compared to plain OpenAI.

The ANOVA results show that although all three OpenAI configurations have similar global embedding similarity, they differ in lexical overlap measures (ROUGE-L, BLEU, Jaccard) and semantic similarity (BERTScore). This suggests that Structured RAG and reranking significantly influence how the OpenAI model produces answers, improving the responses from the knowledge base even though sentence-level similarity remains comparable across the configurations.

An analysis of One-way ANOVA with Tukey's HSD (Table 9.5 in the appendix) indicates no significant differences between the Plain OpenAI and Reranked Structured RAG-based models across all metrics, except for BERTScore_R and BERTScore_F1, as shown in the experimental results. The Cosine similarity metric also shows no significant differences among the variants, implying similar levels of semantic alignment. While no significant difference was found between the structured RAG-based OpenAI and the structured RAG-based models for BERTScore_R, but a significant difference was observed across all variants for a different metric BERTScore_F1.

To sum up, the results show that the Structured RAG-based OpenAI model enhances lexical accuracy without altering semantic similarity. Meanwhile, the Structured RAG-based OpenAI model with Reranking variant does not provide any further improvements in the current setting.

5.7 Benchmarking Highlights

This section presents the benchmarking highlights for both retrieval quality evaluations of question-answering and root cause analysis, and similarity evaluation of question-answering.

5.7.1 Benchmarking Highlights on Retrieval Quality Evaluation

Retrieval Quality Evaluation	**Question-Answering**	**Root Cause Analysis**
Top-K	Top-5 and Top-10	Top-10 and Top-20
Matched index	Top-5 : 1-5 (query expansion) Top-10 : 1-7 (query expansion)	Top-10 : 1 (query expansion + reranking) Top-20 : 1 (query expansion + reranking)
Latency	Top-5 : 0.013 sec Top-10 : 0.014 sec	Top-10 : 0.146 sec Top-20 : 0.309 seconds
Accuracy	Top-5 : 96.80% Top-10 : 97.20%	Top-10 : 66.15% Top-20 : 66.15%

Query expansion	IncidentTitle: <IncidentTitle>, Content: <query>	Diagnostic service: <ServiceName>-<ServiceID>, NRC: <NRC>, Error: <ErrorType>

Table 5.14: Benchmarking highlights on retrieval quality evaluation

Table 5.14 summarises the best-performing retrieval configurations for the two tasks studied in this thesis: question-answering over resolved incidents and root cause analysis over NRC conditions. For question-answering, Structured RAG with query expansion (IncidentTitle: <IncidentTitle>, Content: <query>) achieves very high retrieval accuracy (96.80% for Top-5 and 97.20% for Top-10), with the correct context typically appearing within ranks 1–5 or 1–7 while keeping latency extremely low (around 0.013–0.014 seconds). For root cause analysis, the best configuration combines structured query expansion (Diagnostic service: <ServiceName>-<ServiceID>, NRC: <NRC>, Error: <ErrorType>) with cross-encoder reranking, yielding a Top-10 accuracy of 66.15%, where all correctly retrieved NRC rows are placed at rank 1. Increasing k to 20 does not improve accuracy but roughly doubles latency (from 0.146 to 0.309 seconds), indicating that Top-10 is sufficient, and that reranking mainly serves to concentrate the correct candidate at the top rather than to increase coverage.

5.7.2 Benchmarking Highlights on Similarity Evaluation of Question-Answering

Similarity Evaluation (Question-Answering)	**TinyLlama**	**OpenAI GPT-5 nano**
Semantic metrics: Cosine, BERTScore P / R / F1	No RAG : Good RAG : Good RAG + re-rank : Best	No RAG : Good RAG : Good RAG + re-rank : Best
Lexical metrics: ROUGE-L, BLEU, Jaccard	No RAG : Low RAG : Average RAG + re-rank : Best	No RAG : Low RAG : Moderate RAG + re-rank : Best
Hypothesis testing metrics: ANOVA (Analysis of Variance for No RAG, RAG, and RAG + re-rank)	Fail to reject H0; $p > 0.05$ (BERTScore_P); Not statistically significant	Reject H0; $p < 0.05$ (all metrics); Strongly statistically significant
Post-hoc tests-Tukey's HSD, if significant		RAG: Best

Table 5.15: Benchmarking highlights on similarity evaluation of question-answering

Table 5.15 summarises the similarity-based benchmarking results for TinyLlama and the OpenAI model across three configurations: no RAG, Structured RAG, and Structured RAG with reranking. For both models, semantic metrics (cosine similarity and BERTScore P/R/F1) are already “Good” without RAG, but reach their “Best” levels when retrieval and reranking are enabled, showing that high-quality, ranked context improves semantic alignment with synthetic target values. Lexical metrics (ROUGE-L, BLEU, Jaccard) are low without RAG, increase to average or moderate levels with Structured RAG, and are best in the RAG with reranking setting, indicating more consistent reuse of domain-specific phrasing from the knowledge base. The ANOVA results show that, for TinyLlama, most metrics differ significantly across configurations except BERTScore precision, while for the OpenAI model all metrics show statistically significant differences, leading to rejection of the null hypothesis. Tukey's HSD post-hoc tests further confirm that RAG-based configurations for OpenAI are generally the top-performing setups, highlighting the importance of retrieval for generative question-answering in this domain.

5.8 Cost Effectiveness

Token usage significantly affects costs in usage-based LLM services, as both input and output tokens contribute to expenses. We monitor token usage across three pipelines: plain LLM, structured RAG, and structured RAG with reranking. The plain pipeline generally uses the fewest tokens and does not include retrieved context. RAG pipelines tend to increase input token counts due to retrieved documents, but techniques such as chunking, filtering, and Top-k selection help reduce this extra cost. Token usage in RAG variants is primarily controlled by the following strategies:

Chunking strategy: Larger chunks increase the token footprint of the retrieved context, leading to a higher input token count. Initially, entire README files, including the problem description, symptoms, root cause, and solution, as well as complete NRC tables, were embedded as large segments, resulting in large prompt sizes in RAG. By restructuring the data into smaller, semantically focused chunks, such as NRC-specific tables and incident sections, the retrieved context became more concise and manageable, token-efficient.

Number of retrieved documents (Top-k): Including additional retrieved items extends the prompt length. In structured RAG, fetching the top-10 documents significantly increases the number of input tokens compared with a more selective approach. In the context-customized RAG pipeline, only the top reranked documents are included: the Top-1 for root-cause and the Top-3 for question-answering, which helps reduce prompt length.

Reranking method: LLM-based reranking requires additional token-based model calls, increasing costs for token-priced APIs. To avoid this, a HuggingFace Cross-Encoder reranker was used, which does not charge tokens.

Cost considerations also influenced infrastructure choices. This thesis uses FAISS, an open-source vector store that avoids the per-request charges common with managed vector databases. Since FAISS does not support native metadata filtering, filtering and ranking must be implemented within the system.

5.8.1 Cost Structure

OpenAI's API uses token-based pricing, with costs scaling linearly with the number of tokens. Separate charges apply to input tokens, cached inputs (when applicable), and output tokens. Longer prompts and responses lead to higher expenses. For the GPT-5 nano model referenced in this thesis, prices are specified per million tokens for both input and output. Costs vary by model and token type. Also, OpenAI offers optional features, such as priority and batch processing, which can affect cost and performance.

TinyLlama is a self-hosted, infrastructure-based solution. While it avoids per-token API fees, deployment still involves costs. Major expenses include compute resources, such as renting CPU or GPU instances or using on-premises hardware; energy consumption, particularly for GPUs; engineering efforts, including deployment, optimization, and monitoring; and operational expenses, such as scaling, updates, and maintaining reliability. These costs depend on workload size and can become notable at larger scales if proper infrastructure management is not in place.

Performance criteria for a fair comparison include evaluating both cost-effectiveness and operational performance. This thesis examines latency (end-to-end response time), throughput (tokens processed per second), and availability (reliability). OpenAI provides managed infrastructure with service guarantees that vary by plan and setup, whereas self-hosted TinyLlama's availability depends entirely on the deployment environment and the user's monitoring and redundancy measures.

5.8.2 Latency Analysis

Statistics	Plain OpenAI	Rag-based OpenAI	RAG-based OpenAI with Reranking

count	50	50	50
mean	22.817	21.3366	30.1476
std	13.86902	15.70619	16.87557
min	6.54	7.26	12.41
25%	16.1225	13.2025	19.7975
50%	19.27	17.095	25.875
75%	24.97	21.525	32.75
Max	98.75	87.98	99.75

Table 5.16: Descriptive statistics for OpenAI response generation time (seconds)

The latency analysis (Table 5.14 and Table 5.15), which measures response generation time in seconds, shows distinct differences between OpenAI GPT-5 nano and TinyLlama. OpenAI GPT-5 nano (Table 5.14) consistently maintains low, stable average response times of 22.8s for plain queries, 21.3s for structured RAG, and 30.1s for structured RAG with reranking. The moderate variability, with standard deviations ranging from 13.9s to 16.9s, suggests predictable and reliable performance, with worst-case response times not exceeding roughly 99s.

Statistics	**Plain TinyLlama**	**RAG-based TinyLlama**	**RAG-based TinyLlama with Reranking**
count	50	50	50
mean	43.4556	88.1172	46.7476
std	68.34384	74.39511	14.58346
min	7.37	11.83	24.14
25%	17.66	24.06	36.44
50%	29.54	48.15	45.765
75%	43.135	175.6725	52.7575
max	368.13	221.88	107.18

Table 5.17: Descriptive statistics for TinyLlama response generation time (seconds)

By contrast, TinyLlama (Table 5.15) shows significantly higher average latencies: approximately 43.4s for plain queries, 88.1s for structured RAG, and 46.7s for structured RAG with reranking. Its performance is highly variable, with standard deviations of about 68s for plain and 74s for structured RAG, indicating notable instability affected by prompt complexity and system load. Outliers can reach response times of up to 368s for plain queries, leading to noticeable delays in interactive applications.

OpenAI GPT-5 nano is faster and offers more consistent performance. While TinyLlama can sometimes be competitive in structured RAG with reranking, its high variance and potential for extremely long delays could negatively impact user experience.

5.8.3 Throughput Analysis

Statistics	**Plain OpenAI**	**RAG-based OpenAI**	**RAG-based OpenAI with Reranking**
count	50	50	50
mean	3063.14	3479.6	3918.22
std	849.4526	854.2624	925.9513
min	961	1910	2233
25%	2504.75	2863.5	3156
50%	2936.5	3470	3978.5
75%	3732.25	4061.75	4459.5
max	4980	5853	6163

Table 5.18: Descriptive statistics for token usage in OpenAI

The analysis of token usage per query for OpenAI (Table 5.16) reveals a notable difference between OpenAI GPT-5 nano and TinyLlama in workload and output size. OpenAI GPT-5 nano consistently produces large outputs, with mean token counts of about 3,063 for plain queries, 3,479 for structured RAG, and 3,918 for structured RAG with reranking. Although the token counts vary moderately, from 849 to 925, the size remains high, suggesting that OpenAI is optimized to handle complex queries that require extensive and detailed completions.

Statistics	Plain TinyLlama	RAG-based TinyLlama	RAG-based TinyLlama with Reranking
count	50	50	50
mean	259.76	356.78	280.98
std	370.4601	324.2298	94.64078
min	36	14	133
25%	95.75	87.25	218.25
50%	181	203.5	275
75%	274.25	689	315
max	1988	979	645

Table 5.19: Descriptive statistics for token usage in TinyLlama

Compared with the OpenAI GPT-5 nano model, TinyLlama (Table 5.17) generates significantly shorter outputs, averaging about 260 tokens for simple queries, 356 tokens for structured RAG tasks, and 281 tokens when combined with reranking. Its output size varies widely, with standard deviations sometimes exceeding the mean, such as approximately 370 tokens for plain queries, indicating inconsistent lengths and potential influence from prompt structure or system behaviour. By contrast, OpenAI GPT-5 nano is optimized for long, detailed responses, whereas TinyLlama is designed for shorter, more lightweight outputs. This key difference significantly affects throughput and cost-effectiveness, especially regarding the cost per token for each model.

5.8.4 Latency and Throughput Comparison - OpenAI vs TinyLlama

Scenario	OpenAI Latency (sec)	TinyLlama Latency (sec)	OpenAI Throughput (tokens/sec)	TinyLlama Throughput (tokens/sec)
Plain	22.817	43.456	149.994	6.009
RAG-based	21.337	88.117	202.327	3.682
RAG-based with Reranking	30.148	46.748	152.748	5.969

Table 5.20: Latency and throughput comparison (tokens/sec)

Table 5.18 suggests that OpenAI consistently outperforms TinyLlama in response speed and throughput across several queries because TinyLlama runs locally on a CPU, leading to longer delays. OpenAI is more responsive in interactive settings and achieves higher token rates. Its larger models are more accurate and suitable for high-volume tasks. TinyLlama, being free, relies on a structured RAG with reranking to improve accuracy, making it potentially more efficient for cost-sensitive, high-volume use if infrastructure costs are minimized and reranking is used. OpenAI offers better accuracy, reliability, and scalability management.

5.8.5 Cost Analysis

According to https://openai.com/api/pricing/, OpenAI's GPT-5 nano is the quickest and most affordable GPT-5 version, ideal for summarization and classification tasks compared with other models. The input cost is $0.050 per million tokens, with cached input costing $0.005 and output costing $0.400 per million tokens.

Statistics	Plain OpenAI	RAG-based OpenAI	RAG-based OpenAI with Reranking
count	50	50	50
mean	0.00121	0.001075	0.001472
std	0.00034	0.000336	0.000366
min	0.00037	0.000441	0.000802
25%	0.000987	0.000838	0.001166
50%	0.001161	0.001057	0.0015
75%	0.001478	0.001318	0.001685
max	0.001978	0.001923	0.002345

Table 5.21: Descriptive statistics for OpenAI cost per query (USD)

The cost per query (Table 5.19) suggests that OpenAI GPT-5 nano offers predictable, low-cost performance, with minimal variability, at around $0.0012 for plain queries, $0.00107 for structured RAG, and $0.00147 with reranking, highlighting stable pay-per-token billing. Even at worst, costs stay below $0.00235, ensuring reliable, transparent pricing. TinyLlama has no direct API token costs, being open-source, but infrastructure and energy costs are significant. While it may seem free for TinyLlama, operational costs such as GPU, electricity, and maintenance vary depending on the hardware. OpenAI GPT-5 nano provides consistent, low-cost queries, while TinyLlama avoids token-based billing but incurs indirect infrastructure costs not reflected in simple cost figures.

To connect per-query results to an operational estimate, this thesis assumes 50 queries per day (≈ 1,500 queries/month). Using the observed mean cost per query for OpenAI, the monthly API cost is: Plain OpenAI: 1,500 × 0.00121 ≈ $1.82/month, RAG-based OpenAI: 1,500 × 0.001075 ≈ $1.61/month, and RAG with reranking OpenAI: 1,500 × 0.001472 ≈ $2.21/month. If VAT applies to the billing context, the total would increase proportionally.

5.8.6 Cost Comparison - OpenAI vs TinyLlama

Aspect	OpenAI	TinyLlama
Deployment type	A combination of both self-hosted and OpenAI-managed services	Self-hosted open-source model
Cost model	Both infrastructure and token-based, pay-per-use (input/cached/output tokens)	Infrastructure-based (compute and operations)
Maintenance	Handled by both OpenAI and user-managed in AWS	User-managed for setup, scaling, and optimization in AWS
Data privacy	Handled by the company for OpenAI's Enterprise model	Depending on Meta's compliance for enterprise usage
Embedding model	Yes. But, free to use open-source embedding models	Free to use open-source embedding models
Model fine-tuning (not used in this thesis)	Higher training costs per-token inference	No

Table 5.22: Cost Summary comparison between OpenAI and TinyLlama

Table 5.20 compares OpenAI and TinyLlama: OpenAI is a managed, pay-per-use service with shared maintenance in the DCP context, while TinyLlama is open-source and self-hosted, with costs mainly for compute, deployment, scaling, and optimization. For privacy, OpenAI relies on vendor controls; TinyLlama's privacy depends on hosting and governance. Both support embedding models, including open-source options. Although fine-tuning was not covered due to limited data in this study, OpenAI may have higher token costs, and TinyLlama requires its own compute without vendor fees.

5.8.7 Cost Comparison for Vector Store Choices

Aspect	**AWS vector database**	**FAISS (used in this thesis)**
Vector database	Requires external vector database: FAISS, or enterprise-grade – Amazon OpenSearch Serverless with vector search, Aurora PostgreSQL with pgvector extension, or Kendra	Open-source vector database FAISS (Facebook AI Similarity Search)
Indexing and query cost	Yes. Bills per request in Serverless mode in the AWS vector database	No. Free for FAISS. Local compute for both EC2 instances and on-prem
Indexing speed	Slower for large datasets in the AWS environment	Faster indexing on local GPU
Template-based pdf extraction control	Need to rely on the AWS service. No guarantee for accurate pdf extractions	Complete control over template-based pdf extraction
Retrieval accuracy	Via Approximate Nearest Neighbour (ANN) algorithms, index parameters	More control over accuracy

Table 5.23: Cost comparison for vector store choices (RAG integration)

Table 5.21 compares AWS-managed vector search (the company recommendation) with FAISS (the thesis choice). AWS uses services such as OpenSearch Serverless, Aurora PostgreSQL with pgvector, or Kendra, charging per request in serverless modes. FAISS is free, open-source, and runs on its hardware (EC2 or on-prem). AWS can be slower to index large datasets, while FAISS, especially with GPU support, indexes faster locally. AWS services offer less control over PDF extraction quality, while FAISS pipelines allow full control over template-based extraction. AWS supports ANN retrieval, but FAISS provides more control over index options and parameters, enhancing retrieval control accuracy.

6 DISCUSSION

This chapter critically discusses the study's findings within the broader research context. It revisits the research questions, provides evidence-based answers, compares the findings with previous work, and highlights the contributions and research gaps. The chapter examines data patterns, unexpected results, and conflicting evidence. It reflects on the study's trustworthiness and limitations, including methodological choices. Finally, it explores practical and research implications, as well as societal, ethical, and sustainability aspects.

This thesis benchmarks machine learning techniques for a generative AI troubleshooting assistant within an automotive diagnostic platform. The OpenAI model outperforms TinyLlama for latency and throughput, though TinyLlama remains a cost-effective option. RAG significantly improves performance compared with models that rely solely on internal knowledge. This highlights the importance of grounding generative models in domain-specific diagnostic data rather than relying solely on natural-language capabilities. An interesting and somewhat unexpected finding is that using key-value representations in the query pattern resulted in lower retrieval accuracy in root-cause analysis. This suggests that such representations may not be the most effective during the knowledge-retrieval phase of troubleshooting, which is crucial for DCP.

6.1 Addressing RQ1: State-of-the-art Techniques and Their Applicability

RQ1: What are the state-of-the-art machine learning techniques used to provide accurate responses in a generative AI-powered troubleshooting assistant to user queries, considering the knowledge base data and tool usage logs within the automotive off-board diagnostic communication platform? Which techniques can be applied among the techniques identified?

RQ1 focused on identifying state-of-the-art machine learning techniques for a generative troubleshooting assistant that has access to a knowledge base and tool usage logs in the automotive diagnostic domain. The literature review and empirical study in this thesis highlight several key techniques as follows:

Large Language Models (LLMs)	Using TinyLlama and OpenAI models as the primary generative engines for generating troubleshooting responses.
Structured RAG	Align queries and documents with domain-specific structures, such as using diagnostic service ID, NRC, and error type for root-cause analysis, or incident title, problem description, symptoms, root cause, and solution for question-answering.
Domain-Adaptive RAG	Adjusting retrieval methods to align with UDS diagnostic structures rather than using standard text chunking techniques.
Cross-Encoder reranking	Re-evaluating the Top-k retrieved candidates to enhance the quality of the highest-ranked (rank-1) result.
Similarity-based evaluation	Using metrics such as cosine similarity, ROUGE-L, BLEU, Jaccard, and BERTScore to evaluate content quality, moving beyond simple hit/miss or exact-match assessments.

Table 6.1: Identified techniques in the survey to address RQ1

The industry now employs architectures that combine generative models with retrieval, logging, and domain-specific constraints, moving away from reliance on a single model. Not all techniques are suitable for off-board automotive environments. Approaches that require extensive, clean, labelled

datasets or continuous feedback from diagnostic experts are more difficult to implement because of data availability, privacy, and integration constraints. Building on these constraints and opportunities within the company's diagnostic communication platform, this thesis focuses on three selected techniques for further investigation:

Technique 1	A baseline/plain LLMs without retrieval, using carefully designed prompts in an OpenAI model and a TinyLlama model as purely generative systems.
Technique 2	LLMs with Structured Domain-Adaptive RAG over knowledge base and diagnostic logs.
Technique 3	A hybrid approach that combines the earlier Structured Domain-Adaptive RAG setup with additional re-ranking of candidate answers.

Table 6.2: Techniques considered for empirical study

These techniques were selected because they are appropriate for an automotive off-board diagnostic environment and demonstrate several modelling methods using knowledge base data and tool usage logs. Thus, RQ1 is addressed by (1) providing an overview of the broader state-of-the-art and (2) justifying a subset of techniques that are practical and relevant to the particular industrial context.

The experiments in Section 5.2 (Retrieval Quality Evaluation of Domain-Adaptive and Structured RAG for Root-Cause Analysis) investigated the effectiveness of a Structured RAG and a Domain-Adaptive RAG retriever in accurately identifying the appropriate NRC row as a potential root cause for problematic UDS log lines. They empirically identified which variants are applicable in the automotive diagnostic context. This directly supports RQ1 of the thesis, which examines how to implement state-of-the-art techniques in this domain by evaluating various retrieval configurations and text representations for root-cause analysis. The results indicate that carefully designed structured queries and simple query expansion can significantly enhance retrieval quality. By contrast, incorporating more low-level details, such as raw data bytes or key-value encodings, does not improve performance and may even degrade it.

The first set of experiments (Section 5.2.1) compared three retrieval configurations across 130 synthetic problematic log lines: a baseline embedding retriever using a simple structured pattern, a configuration with query expansion by adding the ErrorType, and a configuration with query expansion and cross-encoder reranking, evaluated for both Top-10 and Top-20 retrieval. The experiments show that a Domain-Adaptive Structured RAG retriever using natural-text fields (Diagnostic service: <ServiceName>-<ServiceID>, NRC: <NRC>, Error: <ErrorType>) achieves the best Top-10 retrieval accuracy (66.15%, 86/130) (Table 5.1) and rank-1 coverage. More structured representations that add raw UDS bytes or key-value schemas reduce accuracy to 62.31% (81/130) (Table 5.2) and 60.77% (79/130) (Table 5.3), respectively. Simple query expansion that includes the ErrorType field in the query helps the embedding model distinguish between rows that share the same service ID and NRC but differ in semantics. From RQ1's perspective, this pattern is directly relevant to RQ1. The applicable evidence-based techniques for this domain are Domain-Adaptive and Structured RAG. They should not only retrieve arbitrary chunks but also use domain semantics (ServiceName, ServiceID, NRC, ErrorType) in both documents and queries. Less appropriate techniques in this context are representations that emphasize raw bytes or heavy key-value structure. This evidence suggests that natural text with semantically meaningful fields performs best in the embedding-based retrieval model, adding raw UDS bytes is not beneficial, and key-value schema text is worse.

The cross-encoder re-ranker maintains Top-10 accuracy at 66.15% while significantly improving rank-1 quality by moving all correct hits to position 1. This is crucial for a troubleshooting assistant, as users typically focus only on the first suggestion. While reranking does not recover missing information, it effectively refines results, making successful cases more immediately usable. That said, this

improvement comes at a latency cost, increasing the average response time from about 0.008 seconds for retrieval only to roughly 0.146 seconds. Whether this trade-off is acceptable depends on system constraints; if a slight increase in latency is permissible for more reliable top suggestions, the added delay can be justified.

Expanding from Top-10 to Top-20 reveals an insightful pattern. With query expansion and reranking, the Top-20 accuracy remains at 66.15%, the same as the Top-10 accuracy. This suggests that all queries with the correct NRC already appear within the Top-10 results. Increasing k to 20 mainly introduces redundant candidates and roughly doubles reranking time (from 0.146 seconds to 0.309 seconds) without improving effectiveness. Thus, in this context, Top-10 provides sufficient coverage and offers a favourable balance between accuracy and efficiency.

The second setup (Section 5.2.2) enhanced the textual representation by including the raw UDS negative response bytes (Data: 0x7F <ServiceID> <NRC>) in both documents and queries. Intuitively, this might seem to improve retrieval since it mimics the exact log-line format, aligning more closely with actual log error appearances. That said, the results indicate the opposite. With this additional Data field, Top-10 accuracy using query expansion and reranking decreases from 66.15% to 62.31% (from 86 to 81 hits), and Top-20 accuracy is 65.38%, still below the 66.15% achieved with the simpler representation. On the one hand, this is an unexpected finding; it contradicts the intuitive assumption that richer structural information does not necessarily benefit the embedding model. Conversely, it could also be argued that all-MiniLM-L6-v2 is trained primarily on natural language, not on hex bytes and protocol-like tokens. As a result, adding sequences such as 0x7F 10 12 may introduce noise that obscures more meaningful fields, such as ServiceName, NRC, and ErrorType.

The third setup (Section 5.2.3) advanced the structure by representing each NRC row as key-value pairs: ServiceID: <ServiceID>, NegativeResponseCode: <NRC>, ServiceName: <ServiceName>, ErrorType: <ErrorType>. While this approach is reasonable from a schema perspective because it explicitly displays each UDS field and could help a model match attributes more easily, the results indicate that this method performs the worst. Top-10 accuracy drops to 60.77% (79/130), and Top-20 accuracy increases only slightly to 62.31%, both lower than the simpler, natural-text-based approach in Section 5.2.1. As with the previous setup, reranking fails to improve retrieval accuracy; it elevates existing hits to rank 1, doubling latency when moving from Top-10 (0.148 s) to Top-20 (0.308 s). This confirms that adding syntactic structure alone is insufficient and may even be counter-productive. Alternatively, key-value patterns might introduce repetitive, non-semantic tokens that dominate the representation, thereby diminishing the emphasis on domain-critical content such as the ErrorType.

Section 5.2 addresses RQ1 by demonstrating that Domain-Adaptive and Structured RAG over natural text with semantically meaningful fields is the most practical and effective state-of-the-art method for root cause retrieval in the automotive diagnostic context. It provides empirical evidence to determine the most valuable representations for this technique, showing that a simple semantic structure is optimal, while raw bytes and key-value schemas are less appropriate.

Section 5.3 (Retrieval Quality Evaluation of Structured RAG for Question-Answering) addresses RQ1 by showing that Domain-Adaptive Structured RAG with query expansion is an effective, state-of-the-art approach for question-answering within the automotive diagnostic communication platform. It presents advanced retrieval methods for question-answering in an automotive diagnostic troubleshooting assistant, using Structured RAG over a domain-specific incident knowledge base. The section compares three retriever configurations: the baseline, which uses the user query directly; Structured RAG with query expansion, where the user query is rewritten to match the incident structure, for example, IncidentTitle: <IncidentTitle>, Content: <query>; and Structured RAG with cross-encoder reranking, which applies reranking to the Top-k candidates using the expanded query.

RQ1 investigates which of the identified techniques can be applied effectively. Section 5.3 provides empirical evidence by demonstrating which retrieval strategies perform best. Using 500 synthetic queries across 100 resolved incidents, the baseline retriever, which relies on raw queries, achieves

71.20% accuracy for Top-5 and 77.20% for Top-10. When Structured RAG query expansion is employed, these figures increase to 96.80% and 97.20%, respectively, with minimal change in latency (~0.011s compared with ~0.013–0.014s). This section suggests that Structured RAG query expansion is a key, applicable technique. Transitioning from raw queries to structured query patterns boosts Top-5 accuracy from 71.20% to 96.80% and Top-10 from 77.20% to 97.20%, with almost no increase in latency (~0.011s to ~0.013–0.014s). Cross-encoder reranking is an optional enhancement. It does not improve Top-k accuracy, as coverage remains at 484/500 and 486/500, but it slightly improves rank-1 focus, increasing rank-1 hits from 471 to 479 for Top-5. That said, it incurs a significant latency increase, from 0.013s to 0.121s, nearly ten times. For RQ1, Section 5.3 suggests that the primary technique for question-answering in this domain is Structured RAG with domain-aligned query expansion. Reranking is also a state-of-the-art method, but it should be considered an optional enhancement, justified only if a slight improvement in rank-1 quality outweighs the additional cost.

Section 5.4 (Similarity Evaluation – Retrieval of Root-Cause Analysis) supports RQ1 by showing that the Structured RAG-based retriever described in Section 5.2 not only finds the correct NRC rows in a Top-k manner but also returns NRC condition texts that are semantically very similar to the synthetic target value descriptions. This suggests that both Structured RAG and Domain-Adaptive retrieval retrieve semantically accurate NRC conditions, rather than just identical rows. This evaluation employs advanced similarity metrics to assess this performance. The retriever from Section 5.2 can be evaluated using modern semantic similarity tools, and the retrieved texts are semantically faithful enough to serve as trustworthy grounding for a generative troubleshooting assistant.

Section 5.4 demonstrates high semantic similarity and low lexical overlap by analysing NRC conditions. It compares the synthetic target value, a structured query string following the Domain-Adaptive RAG pattern, with the retrieved context, the NRC condition text from ISO 14229-1 expressed in natural language. The similarity analysis indicates strong embedding-based closeness, with mean cosine similarity of 0.7409, mean BERTScore_F1 of 0.7862, and mean BERTScore_R of 0.8595, all with low variance. By contrast, lexical metrics such as ROUGE-L, BLEU, and Jaccard scores are low, as expected, because the synthetic target value is a concise, structured query while the retrieved context is a longer, natural-language description. Section 5.2 demonstrated that Structured RAG with domain-aware representations is a promising state-of-the-art technique for root cause retrieval. In Section 5.4, we address RQ1 by showing that the chosen technique, Domain-Adaptive and Structured RAG for NRC root cause retrieval, works as expected. It consistently produces natural-language NRC descriptions that semantically align with the structured diagnostic query, and this alignment can be measured using semantic similarity metrics. As a result, Structured RAG, evaluated with cosine and BERTScore, is a practically viable state-of-the-art technique for grounding a generative assistant in this diagnostic domain.

Sections 5.5 (Similarity Evaluation - TinyLlama for Question Answering) and 5.6 (Similarity Evaluation - OpenAI for Question Answering) discuss TinyLlama and OpenAI as generative engines, respectively, and evaluate them across three setups: Plain LLMs (TinyLlama, OpenAI) without RAG, LLMs with Structured RAG, and LLMs with Structured RAG and reranking. Their performance was measured against synthetic target values for 500 question-answering queries, using similarity metrics. This pipeline, LLMs with a Structured RAG retriever that employs resolved incidents (problem descriptions, symptoms, root causes, and solutions) as structured knowledge and reranking that injects manually reranked context, represents the current state-of-the-art design pattern for troubleshooting assistants in industry.

Section 5.5 directly addresses RQ1 for the TinyLlama-based generative assistant. It suggests that combining TinyLlama with Structured RAG, optionally with reranking, is a practical and state-of-the-art approach: compared with the plain model, cosine similarity improves from 0.4586 to 0.6735 and 0.7172, ROUGE-L from 0.1271 to 0.1814 and 0.2103, and BERTScore_F1 from 0.8303 to 0.8482 and 0.858 across the three setups. While plain TinyLlama can produce meaningful answers, its alignment with precise incident-based synthetic target values is inconsistent, as indicated by the wide cosine range. The combination of TinyLlama with Structured RAG (Section 5.5.2) provides clear evidence for RQ1:

TinyLlama becomes significantly more accurate and reliable as a troubleshooting assistant when combined with Structured RAG. This suggests that structured, domain-specific retrieval is an effective and applicable state-of-the-art technique for this task. According to the scores in the TinyLlama with Structured RAG and reranking setup (Section 5.5.3), reranking on top of Structured RAG produces the best semantic and lexical similarity to synthetic target values. These three configurations illustrate the necessary techniques for TinyLlama in this domain: the plain model alone is insufficient, Structured RAG is crucial, and Structured RAG with reranking delivers the highest question-answering quality within TinyLlama, confirming it as a reliable state-of-the-art setup.

Section 5.6 addresses RQ1 by demonstrating that, for a robust base model such as OpenAI, Structured RAG is an effective and applicable state-of-the-art technique for improving answer quality in diagnostic question-answering. It compares the performance of different configurations, showing that adding Structured RAG to the basic OpenAI model boosts ROUGE-L from 0.0787 to 0.1206, BLEU from 0.0122 to 0.0243, Jaccard from 0.1117 to 0.1537, and BERTScore_F1 from 0.8179 to 0.839, while maintaining high cosine similarity (0.6661-0.688). We note that even without RAG, OpenAI functions as a reasonably accurate, semantically correct assistant, but it remains ungrounded and has weak lexical alignment with incident knowledge. The results in Section 5.6.2 confirm that OpenAI with Structured RAG directly answers RQ1: it is a practical, state-of-the-art technique that significantly enhances lexical accuracy and semantic coverage without compromising the model's natural phrasing. Section 5.6.3 suggests that Structured RAG with reranking improves lexical and semantic similarity compared to the baseline. That said, this benefit is limited because the reranking step does not consistently add value for OpenAI.

6.2 Addressing RQ2: Comparative Performance and Hypothesis Testing

RQ2: How do different machine learning techniques, identified in RQ1, compare in generating accurate responses to user queries in terms of performance by utilizing knowledge base data and tool usage logs, including both failure and non-failure cases, for a generative AI-powered troubleshooting assistant within the automotive off-board diagnostic communication platform?

To address RQ2, the following hypotheses are formulated:

H0 (null hypothesis): There is no significant difference in performance among the machine learning techniques identified in RQ1 for achieving the most accurate responses.

H1 (alternative hypothesis): There is a significant difference in performance among the machine learning techniques identified in RQ1 for achieving the most accurate responses.

RQ2 examined how the selected techniques compare in their ability to produce accurate responses to user queries, using knowledge base data and tool usage logs. The results presented in Chapter 5 show that:

Result 1	Which technique performed best? Domain-Adaptive RAG and Structured RAG-based LLM (Technique 2) achieved the highest accuracy and relevance scores across the evaluation set.
Result 2	Relative differences: LLM (TinyLlama) without retrieval (Technique 1) underperformed in semantic and lexical similarity, producing less accurate responses to user queries.
Result 3	The addition of problematic logs, the use of tool-usage logs (Technique 3), and re-ranking candidate answers were better at proposing realistic next steps than those relying solely on static knowledge base data.
Result 4	Structured RAG and simple query expansion improve retrieval quality.
Result 5	The addition of raw UDS logs and a key-value representation in problematic logs reduces the performance of retrieving relevant root causes from the knowledge base.

Table 6.3: Results in RQ2

In Section 5.2 (Retrieval Quality Evaluation of Domain-Adaptive and Structured RAG for Root-Cause Analysis), retrieval quality is crucial for producing accurate responses, as the generative model's quality depends on the information it receives. This section compares different retrieval configurations, including a baseline (ServiceName-ServiceID, NRC), semantic query expansion (ServiceName-ServiceID, NRC, ErrorType), semantic query expansion with Data (0x7F <ServiceID> <NRC>), and query expansion using a key-value representation. The ranking methods evaluated are pure vector retrieval and vector retrieval with cross-encoder reranking. For each configuration, we report Top-k retrieval accuracy, rank distribution, and latency.

Section 5.2, addressing RQ2, compares retrieval techniques and highlights their varying effectiveness (accuracy, rank-1) and efficiency (latency). Accuracy ranges from 60.77% to 66.15%, depending on the representation. For example, query expansion improves Top-10 accuracy from 62.31% to 66.15%. In Section 5.2.1, reranking boosts successful hits at rank 1 from 77 to 86, but at nearly 20 times the latency. Increasing k from 10 to 20 offers limited additional coverage and doubles latency. The re-ranker's benefit decreases with poorer representations, whether by adding data (Section 5.2.2) or key-value pairs (Section 5.2.3), as it only reorders candidates and cannot recover missing ones. These results show that different retrieval setups lead to different performance, confirming that retrieval success depends on the technique and representation. This provides evidence against the assumption of equal performance (H0). Changing the retrieval technique, representation, or query consistently results in measurable differences in Top-k accuracy and rank-1 performance. Notably, significant practical differences exist among the retrieval techniques used for root-cause analysis.

Regarding RQ2, Section 5.3 (Retrieval Quality Evaluation of Structured RAG for Question-Answering) analyses three techniques: baseline retrieval, Structured RAG, and Structured RAG with reranking. It reveals significant differences in both effectiveness (accuracy, rank-1) and efficiency (latency). Structured RAG significantly boosts coverage, from 356 to 484 correct Top-5 hits, and improves rank-1 accuracy, from 240 to 471 hits. Reranking provides only a minor gain, from 471 to 479 rank-1 hits for Top-5, but adds roughly a 10 times increase in latency, from 0.013s to 0.121s. This addresses RQ2 by showing a clear gap in effectiveness: 71.20 to 96.80 and 77.20 to 97.20 for Top-5 and Top-10, respectively. It suggests that the retriever's choice greatly influences how often the system retrieves correct question-answering contexts. Rank-1 quality is crucial for troubleshooting tools, since technicians usually review only the top suggestion. The data indicate accuracy at rank 1: 240/500 for baseline, 471/500 for Structured RAG, and up to 479/500 for Top-5, with 482/500 at rank 1 for Top-10 with reranking. For latency, baseline runs around 0.011 s, Structured RAG around 0.013–0.014s (almost the same), while reranking takes roughly 0.121–0.233 s, about 10-20 times slower. Thus, Structured RAG outperforms the baseline with higher accuracy and similar latency, whereas reranking involves a trade-off between performance and latency with no significant gain. These findings indicate that the chosen retrieval technique substantially impacts the troubleshooting assistant's ability to deliver accurate, top-ranked answers, contradicting the assumption of equal performance (H0) and supporting H1.

In Section 5.4 (Similarity Evaluation – Retrieval of Root-Cause Analysis), for the Structured RAG setup, when the correct NRC row is retrieved, the semantic similarity between the structured query and the retrieved condition text is consistently high: cosine mean ≈ 0.74, BERTScore_F1 ≈ 0.79, BERTScore_R ≈ 0.86. Lexical overlap is low, as expected, so the high semantic scores cannot be explained by superficial copying. On the one hand, this means that Top-k corresponds to a high-quality, semantically appropriate NRC explanation of the same underlying condition. Conversely, if a configuration with Data or key-value yields lower Top-k accuracy, that directly translates into fewer queries for which we have this high-quality semantic match. So, Section 5.4 helps argue for RQ2 by showing that, when comparing techniques in Section 5.2, differences in retrieval accuracy are meaningful. In the best setup, we not only retrieve the correct NRC row more often, but Section 5.4 suggests that these retrieved rows are semantically very close to the ideal description. In other words, higher Top-k accuracy means more queries are backed by semantically reliable root-cause information,

which is crucial for a generative assistant. This confirms that performance differences between retrieval techniques are not purely formal but translate into meaningful differences in the quality of the grounding available to the generative assistant.

Section 5.5 (Similarity Evaluation - TinyLlama for Question-Answering) presents descriptive statistics showing substantial differences in mean metric values. The increasing means across metrics from plain to reranked Structured RAG (Cosine: 0.4586 to 0.7172, ROUGE-L: 0.1271 to 0.2103, Jaccard: 0.1324 to 0.2459, BERTScore_F1: 0.8303 to 0.858) demonstrate that TinyLlama with Structured RAG and reranking outperforms Structured RAG alone and that Structured RAG outperforms plain TinyLlama, in both lexical and semantic similarity. This directly answers RQ2 at a descriptive level, confirming that the techniques do not perform equally.

Regarding RQ2, Section 5.5 evaluates and statistically compares these techniques using a one-way ANOVA across three variants: No RAG, RAG, and RAG with re-ranking. The ANOVA results reveal statistically significant differences among configurations for Cosine, ROUGE-L, BLEU, Jaccard, BERTScore_R, and BERTScore_F1 (Cosine $F = 29.73$, $p = 0$; BERTScore_F1 $F = 12.56$, $p \approx 9\times10^{-6}$). This provides strong evidence that retrieval augmentation is essential for accurate question-answering. These results fail to reject the null hypothesis (H0) of equal performance for TinyLlama, demonstrating that retrieval augmentation, especially with Structured RAG and re-ranking, is key to delivering accurate, semantically faithful troubleshooting responses.

Section 5.6 (Similarity Evaluation - OpenAI for Question-Answering) provides statistical evidence of systematic differences between plain and Structured RAG. The metrics (ROUGE-L: 0.0787 to 0.1206, BLEU: 0.0122 to 0.0243, Jaccard: 0.1117 to 0.1537, BERTScore_F1: 0.8179 to 0.839) suggest that Structured RAG responses become more lexically aligned with synthetic target values and are more semantically complete. Cosine similarity remains stable (0.6661-0.688), indicating that global sentence-level similarity is high across all setups, though improvements are observed in fine-grained overlap metrics, such as lexical overlap and BERTScore. This suggests that different techniques influence performance: OpenAI model with Structured RAG variants differ from the plain model, particularly in synthetic target value coverage and wording similarity.

Regarding RQ2, Section 5.6 presents a quantitative comparison of three techniques: plain OpenAI, Structured RAG-based OpenAI, and Structured RAG-based OpenAI with re-ranking, using one-way ANOVA and Tukey's HSD. The ANOVA results indicate statistically significant differences across configurations for metrics such as ROUGE-L, BLEU, Jaccard, and BERTScore_P, BERTScore_R, and BERTScore_F1 (ROUGE-L $F = 17.09$, $p = 0$; BERTScore_F1 $F = 25.73$, $p = 0$), while cosine similarity shows no statistically significant difference ($F = 0.70$, $p \approx 0.50$). Post hoc tests show that the Structured RAG-based OpenAI model significantly outperforms the plain model on nearly all lexical and BERTScore metrics. By contrast, the reranked Structured RAG variant does not show consistent additional benefits under the current setup. These results reject the null hypothesis of equal performance (H0) for most metrics and suggest that Structured RAG is the most effective enhancement for OpenAI in this troubleshooting context, while reranking appears optional and less impactful in present scenarios.

Section 5.7 (Cost Effectiveness) compares predictable usage-based pricing with hidden infrastructure costs. OpenAI GPT-5 nano offers a stable, predictable cost per query, based mainly on token exchange, simplifying budgeting and enabling cost optimisation through retrieval strategies such as smaller chunks, lower Top-k, and selective context injection. Experiments show that structured RAG increases token use, while reranking adds processing overhead and slightly raises per-query costs. Improving retrieval can enhance answers but may also increase costs and latency. Conversely, TinyLlama avoids token-based billing by shifting costs to deployment factors such as compute, energy, engineering, and reliability. It demonstrated higher, more variable latency, indicating that comparable responsiveness would require better hardware or scaling, raising infrastructure expenses and reducing its cost benefits. OpenAI is more cost-effective when transparency, speed, and reliability matter. By contrast, TinyLlama can be appealing in cost-sensitive or privacy-focused scenarios if infrastructure is optimised and RAG is designed to keep prompt size small without sacrificing quality. Both the OpenAI

and TinyLlama models require infrastructure costs for the automotive diagnostic communication platform.

6.3 Patterns and Unexpected Findings

Across all three experimental setups, the same patterns emerged. When queries include domain-specific details such as diagnostic service name, service ID, NRC, and error type, the retriever generally outperforms the simple baseline. Using a cross-encoder for reranking does not increase the total number of successful queries but ensures the correct result is at the top, which is crucial for a troubleshooting assistant where users rarely look beyond the first answer. By contrast, making text representations more low-level or overly structured, such as raw bytes or key-value pairs, does not benefit the embedding model and may slightly reduce retrieval performance. Key-value encoding particularly degrades root-cause retrieval, highlighting that added structure does not automatically enhance semantic search. We also observed that increasing k from 10 to 20 offered only slight improvements in accuracy but approximately doubled the retrieval and reranking time. Surprisingly, the OpenAI baseline without retrieval was sometimes rated as clearer than the RAG-based variants. The smaller TinyLlama model sometimes matched or even exceeded the larger OpenAI baseline in accuracy and cost, suggesting that bigger models are not always the most practical option, especially when strong, structured content like NRC codes and recurring incident patterns already guide the process.

6.4 Limitations

We must consider limitations when interpreting the results and assessing their applicability to real diagnostic settings.

6.4.1 Data Availability and Quality

Due to the company's ongoing AI enablement process, full access to internal diagnostic logs, incident reports, and knowledge bases was unavailable. Thus, root-cause analysis relied on 130 synthetic log lines generated in accordance with ISO 14229-1, focusing on NRC tables from 26 diagnostic services. Similarly, question-answering used synthetic queries and synthetic target values constructed from a limited knowledge base derived from the resolved incidents database. Real logs and user queries are expected to be noisier and more diverse, and may contain incomplete, inconsistent, or ambiguous information. The most accurate answer or synthetic target value may not be as clearly defined as in this controlled evaluation. Also, the actual data were unstructured and lacked validation; there was no established process to clean, label, and approve them for AI use. This restricts the external applicability of the results and highlights the need for data validation and curation pipelines before deploying in production.

6.4.2 Evaluation Design

This thesis did not involve evaluations by domain experts due to limited access to personnel with specialised knowledge of UDS diagnostics and troubleshooting, and the time required for structured annotation or rating. As a result, the quality of the retrieved context and generated answers was assessed solely using automatic metrics, including Top-k retrieval accuracy, cosine similarity, ROUGE-L, BLEU, Jaccard, and BERTScore. These metrics measure semantic and lexical similarity to synthetic target values but do not fully capture perceived relevance, clarity, or correctness from the viewpoint of technicians or engineers in real-world workshops, production, and aftermarket settings.

6.4.3 Model and Compute Constraints

Available computing resources and licensing restrictions limited this work to a single OpenAI model. Running TinyLlama on CPU for local inference was slow, taking around 30 minutes to produce 10 responses. Similar slowness was observed during exploratory experiments with LLaMa3 on a local machine. Although alternatives like llama.cpp exist, they require additional implementation work and a C++ environment, which was beyond the scope of this thesis. On the hosted side, OpenAI models faced API rate and token limits, which occasionally caused rate limit errors and limited the number of runs and the length of prompts. OpenAI's embedding models were unavailable under the current plan, so all retrieval experiments used a single open-source embedding model (all-MiniLM-L6-v2). Fine-tuning of

LLMs or retrievers was not attempted due to insufficient data and infrastructure. As a result, all results are based on retrieval augmentation rather than fully optimised or fine-tuned solutions.

6.4.4 Retrieval and Document Processing

All retrieval experiments used a single embedding model (all-MiniLM-L6-v2) and a single vector store (FAISS), without native metadata filtering. This setup is suitable for studying structured representations and query expansion. However, it does not cover other approaches that may be important in production, such as alternative embedding models, dense–sparse hybrid retrieval, or metadata-aware indexing. In addition, the ISO 14229-1 standard required layout-aware, template-sensitive PDF extraction, which was challenging because of the mixture of headings, paragraphs, tables, and figures. The focus of this thesis was on NRC tables; other parts of the standard and documents within a realistic diagnostic knowledge base were not included, which limits the coverage of diagnostic information.

6.4.5 Organisational and Environmental Constraints

The experiments were conducted without a sandboxed environment, which would enable safe integration with live systems or access to sensitive internal data. For security and legal reasons, it was not possible to connect external LLM providers via internal diagnostic tools. As a result, there were no opportunities to evaluate additional proprietary models such as Claude or Gemini in the same setup, limiting comparisons to the two primary model families used (TinyLlama and OpenAI). Human evaluation was not conducted due to time limitations within the organisation. Finally, the integration of models and RAG components remains at a prototype stage, tested in a controlled offline environment rather than in a real diagnostic scenario. Factors such as network stability, concurrent user activity, user interface constraints, and technicians' multitasking behaviour were not simulated. These factors can significantly influence whether the observed performance reflects real-world value in a production diagnostic communication platform.

These limitations do not invalidate the findings, but they do narrow the scope of what can be concluded. The results should be interpreted as evidence of the relative effects of retrieval, RAG design choices, and model configurations in controlled settings, rather than as definitive claims about all models, data scenarios, or deployment environments in automotive diagnostics.

6.5 Impact on Validity and Reliability

6.5.1 External Validity

The validity and reliability of the results depend on the dataset, evaluation process, and metrics used. The dataset was constructed from automotive diagnostic cases, using both the ISO 14229-1 standard and 100 synthetic ECU flashing incidents. The evaluation involved 50 synthetic repeated diagnostic incident question-answer pairs, 130 synthetic problematic logs using the ISO 14229-1 standard as the knowledge base, and logs used to identify root causes. While these cases provide a realistic approximation of off-board diagnostic scenarios, they may not fully capture the diversity of real-world diagnostic situations, including multiple systems and vehicle configurations, ECU flashing incidents, and failure modes worldwide. This limits the external validity of the findings, as different vehicle configurations, tools, or data-collection methods might produce different results in real-world off-board diagnostic support scenarios.

6.5.2 Internal Validity

Regarding internal validity, the benchmarking was designed to specifically examine the effects of model selection and architecture, while controlling for invalidating factors by using consistent techniques across identical cases and data sources. That said, the evaluation of prompt templates and RAG setups was limited by available time and computational resources. Some models may be more sensitive to prompt design than others. As a result, differences in performance may partly reflect differences in implementation effort rather than model capabilities.

6.5.3 Construct Validity

Construct validity remains a concern because metrics such as technical correctness, relevance, and clarity do not capture the value that an expert provides in a workshop or production. Essential factors

such as time-to-resolution, technician trust, and long-term learning were not evaluated. While metrics such as cosine similarity, ROUGE-L, BLEU, Jaccard, and BERTScore are appropriate for assessing the quality of troubleshooting assistance, they do not capture all the benefits of real-world workshop, production, or aftermarket scenarios. As a result, models optimised for these metrics might not perform as well in real-world diagnostic support situations.

Finally, the models were evaluated in a controlled setting rather than in real diagnostic situations. Factors such as network stability, multitasking, and user interface limitations were not fully simulated, which may affect how well the conclusions transfer to production environment.

6.6 Implications for Research and Practice

6.6.1 Implications for Research

This thesis suggests that, in automotive diagnostics, the effectiveness of Structured RAG and Domain-Adaptive query expansion depends more on the approach than on the choice of base LLM. For tasks such as root-cause retrieval and question-answering, organising knowledge into semantically meaningful categories, such as service name, service ID, NRC, error type, incident description, symptoms, root cause, and solutions, and adjusting queries accordingly, significantly outperforms simple text or byte-level representations. The evaluation setup, which combines Top-k retrieval with multiple similarity metrics, including cosine similarity, ROUGE-L, BLEU, Jaccard, and BERTScore, along with statistical tests such as ANOVA and Tukey, offers a flexible framework for future comparisons of RAG techniques, embedding models, or LLM families. Moving forward, testing these methods on real workshop data with noise and involving diagnostic experts in evaluations, alongside automatic metrics, is essential.

6.6.2 Implications for Practice

The results indicate that, for industry applications, organisations should allocate at least as many resources to developing structured diagnostic knowledge and logs as to selecting a particular LLM provider. The best-performing setups in this thesis use curated, structured knowledge bases, such as NRC tables and resolved incidents, and query rewriting techniques that translate free-text questions into these schemas. A practical architecture often combines a robust base model, such as OpenAI, or a compact local model, such as TinyLlama, with Structured RAG, and employs reranking only when the slight increase in rank-1 accuracy justifies the extra latency. The study also emphasises the importance of data validation pipelines, metadata-aware retrieval systems, and sandboxed environments to safely incorporate these assistants into diagnostic workflows as decision-support tools.

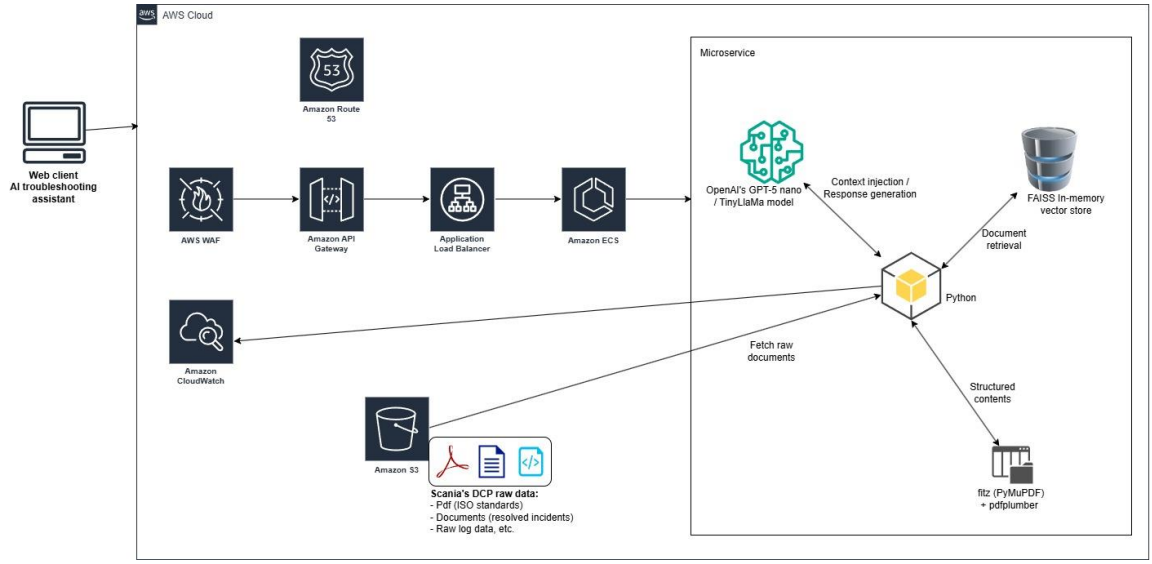


Figure 6.1: Proposed an architecture of Generative AI-powered troubleshooting assistant using AWS for deployment

6.7 Ethical, Societal, and Sustainability Aspects

We consider the following ethical, societal, and sustainability aspects:

6.7.1 Ethical Aspect

Using generative AI in a diagnostic communication platform raises significant data protection and safety concerns. Diagnostic logs and incident reports often contain sensitive technical details about vehicles and control units, and may also include customer or operational data. Deployment must therefore follow strict access controls, anonymisation, data protection laws, and internal governance, especially with cloud-based LLMs. Using cloud LLM providers also requires careful assessment of data transfer, retention policies, and contractual protections to prevent unintentional disclosure of proprietary or personal information. Because the assistant operates in a safety-critical environment, incorrect or hallucinated suggestions could lead to misdiagnosis or unnecessary repairs. The findings indicate that structured retrieval helps limit hallucinations by grounding responses in standards and resolved incidents, but it does not eliminate the need for human-in-the-loop. In workshop, production and aftermarket settings, the system should serve as a decision-support tool, with technicians responsible for decisions and able to review transparent references to source documents to verify the evidence behind each recommendation.

6.7.2 Societal Aspect

From a societal perspective, such assistants can both support and influence diagnostic processes. They could help less experienced technicians access expert knowledge, leading to more consistent results across different workshops. That said, there is a danger that users might become overly dependent on them, causing a decline in their diagnostic skills if they stop practising their own judgement. To prevent this, deploying such tools should include training, well-defined usage guidelines, and feedback channels, enabling users to challenge and refine the system rather than viewing it as infallible.

6.7.3 Sustainability Aspect

There are sustainability considerations linked to the energy consumption of large-scale AI systems. Training and running LLMs, especially in the cloud, consume computational and energy resources. Meanwhile, better diagnostics can reduce unnecessary component replacements, repeat repairs, and workshop time, thereby contributing to more efficient use of vehicles and resources. The benchmarking in this thesis suggests that smaller models combined with effective retrieval can achieve competitive performance, indicating a path towards more energy-efficient solutions that balance accuracy with computational cost. In workshop, production and aftermarket settings, organisations may need to weigh the environmental footprint of different model configurations and deployment options, such as cloud vs on-premise and large vs small models, against the potential gains in diagnostic efficiency and reduced waste, and integrate these considerations into their broader sustainability strategies.

7 CONCLUSION AND FUTURE WORK

This thesis aimed to benchmark machine learning techniques for a generative AI-powered troubleshooting assistant within an automotive diagnostic communication platform. It addressed two research questions: first, identifying appropriate state-of-the-art techniques (RQ1), and second, comparing their performance (RQ2).

Regarding RQ1, the findings indicate that the most effective techniques are RAG with Domain-Adaptive and Structured representations, rather than plain LLMS or baseline methods. For root-cause analysis, the best-performing results were obtained when NRC entries from ISO 14229-1 were represented as layout-aware rows with meaningful fields such as service name, service ID, NRC, and error type, and when queries were adapted to reflect this structure. More complex representations, which focus on raw UDS bytes or key-value syntax, did not improve retrieval accuracy and sometimes reduced it. In question-answering, Structured RAG applied to resolved incidents, combined with schema-aligned query expansion, achieved very high Top-k retrieval accuracy, demonstrating that structured, domain-aware retrieval is both practical and powerful for this application.

Regarding RQ2, the thesis compared retrieval and generation setups for two model families, TinyLlama and OpenAI, using Top-k retrieval, similarity metrics, and hypothesis testing. The findings indicate statistically significant performance differences across techniques and reject the null hypothesis of equal performance for most metrics. For TinyLlama, Structured RAG and reranking notably enhanced both lexical and semantic similarity to synthetic target values compared with the plain model. For OpenAI, Structured RAG improved lexical overlap and semantic coverage, while additional reranking offered only limited improvements under the evaluated configuration. The study suggests that retrieval and grounding choices strongly influence both retrieval and generative answer quality, especially for smaller local models, providing evidence for RQ2.

To sum up, the thesis contributes, firstly, a Structured and Domain-Adaptive RAG design tailored explicitly to UDS diagnostics and incident knowledge; secondly, a benchmarking framework for evaluating retrieval and generation strategies; and thirdly, practical guidelines on representations, query expansion, reranking, and the balance between latency and quality for generative troubleshooting assistants.

7.1 Future Work

This work also highlights multiple directions for further research and development. A natural next step is to move from synthetic to real workshop data. The methods in this thesis were evaluated mainly on ISO-based problematic log lines and constructed incident cases. Future work should test Structured RAG and the benchmarked models on real diagnostic logs and technician queries, supported by data validation and labelling pipelines to make target value more transparent and closer to actual practice. There is also scope to broaden and deepen the technical choices. All experiments used a single embedding model (all-MiniLM-L6-v2) and FAISS without metadata filtering, and both TinyLlama and OpenAI were used off-the-shelf. Future work could compare alternative embeddings and vector stores, explore hybrid retrieval, investigate lightweight Domain-Adaptive adapters, and evaluate more advanced reranking strategies that better balance quality and latency.

Future research should incorporate both human-centred and system-level evaluation. This involves engaging diagnostic engineers and technicians to evaluate the relevance, clarity, and correctness of responses, complementing the automatic metrics used in this study. Deploying the assistant within a sandboxed diagnostic environment would enable analysis of integration factors, including the user interface, workflow compatibility, feedback mechanisms, monitoring for hallucinations or outdated knowledge, failure modes in real-world scenarios, and their impact on troubleshooting efficiency and decision-making quality. Comparing a broader range of model families, including additional open-source LLMs and commercial providers, under similar retrieval setups would help determine whether the findings regarding Structured RAG and model selection extend beyond the specific TinyLlama and OpenAI configurations evaluated in this study.

8 REFERENCES

[1] A. S. Dhillon and A. Torresin, ‘Advancing Vehicle Diagnostic: Exploring the Application of Large Language Models in the Automotive Industry’, 2024, Accessed: Mar. 13, 2025. [Online]. Available: http://hdl.handle.net/20.500.12380/308097

[2] N. AKBARI, ‘A review on Graph RAG methods’, Dec. 2025, Accessed: Jan. 04, 2026. [Online]. Available: https://thesis.unipd.it/handle/20.500.12608/102077

[3] L. Huang *et al.*, ‘A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions’, *ACM Trans Inf Syst*, vol. 43, no. 2, p. 42:1-42:55, Jan. 2025, doi: 10.1145/3703155.

[4] A. Narimani and S. Klarmann, ‘Integration of Large Language Models for Real-Time Troubleshooting in Industrial Environments based on Retrieval-Augmented Generation (RAG)’, in *Proceedings of the International Conference on Industrial Engineering and Operations Management*, Augsburg (Greater Munich), Germany: IEOM Society International, Jul. 2024. doi: 10.46254/EU07.20240085.

[5] G. Agrawal, T. Kumarage, Z. Alghamdi, and H. Liu, ‘Mindful-RAG: A Study of Points of Failure in Retrieval Augmented Generation’, in *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, Nov. 2024, pp. 607–611. doi: 10.1109/FLLM63129.2024.10852457.

[6] V. Zolfaghari, N. Petrovic, F. Pan, K. Lebioda, and A. Knoll, ‘Adopting RAG for LLM-Aided Future Vehicle Design’, presented at the 2024 2nd International Conference on Foundation and Large Language Models, FLLM 2024, 2024, pp. 437–442. doi: 10.1109/FLLM63129.2024.10852467.

[7] Y. Chang *et al.*, ‘A Survey on Evaluation of Large Language Models’, *ACM Trans Intell Syst Technol*, vol. 15, no. 3, p. 39:1-39:45, Mar. 2024, doi: 10.1145/3641289.

[8] X. Yang *et al.*, ‘CRAG - Comprehensive RAG Benchmark’, presented at the Advances in Neural Information Processing Systems, 2024.

[9] F. Liu, Z. Kang, and X. Han, ‘Optimizing RAG Techniques for Automotive Industry PDF Chatbots: A Case Study with Locally Deployed Ollama Models’, Aug. 12, 2024, *arXiv*: arXiv:2408.05933. doi: 10.48550/arXiv.2408.05933.

[10] L.-B. Hernandez-Salinas *et al.*, ‘IDAS: Intelligent Driving Assistance System Using RAG’, *IEEE Open J. Veh. Technol.*, vol. 5, pp. 1139–1165, 2024, doi: 10.1109/OJVT.2024.3447449.

[11] P. Pirasteh *et al.*, ‘Interactive feature extraction for diagnostic trouble codes in predictive maintenance: A case study from automotive domain’, in *Proceedings of the Workshop on Interactive Data Mining*, in WIDM’19. New York, NY, USA: Association for Computing Machinery, Feb. 2019, pp. 1–10. doi: 10.1145/3304079.3310288.

[12] K. Kieu and O. Bergstrand, ‘Empowering Automotive Software Development with LLM-RAG Integration - A study on leveraging the RAG-framework for AUTOSAR and automotive safety standards and specifications’, Oct. 2024, Accessed: May 22, 2025. [Online]. Available: https://gupea.ub.gu.se/handle/2077/83663

[13] A. Sticha, N. Braunschweiler, R. S. Doddipatla, and K. M. Knill, ‘Advancing Faithfulness of Large Language Models in Goal-Oriented Dialogue Question Answering’, in *Proceedings of the 6th ACM Conference on Conversational User Interfaces*, in CUI ’24. New York, NY, USA: Association for Computing Machinery, Jul. 2024, pp. 1–7. doi: 10.1145/3640794.3665573.

[14] R. Omar, O. Mangukiya, and E. Mansour, ‘Dialogue Benchmark Generation from Knowledge Graphs with Cost-Effective Retrieval-Augmented LLMs’, *Proc ACM Manag Data*, vol. 3, no. 1, p. 31:1-31:26, Feb. 2025, doi: 10.1145/3709681.

[15] B. Sarmah, B. Hall, R. Rao, S. Patel, S. Pasquali, and D. Mehta, ‘HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction’, Aug. 09, 2024, *arXiv*: arXiv:2408.04948. doi: 10.48550/arXiv.2408.04948.

[16] I.-S. Han, M.-I. Roh, and M.-C. Kong, ‘A RAG-based Q&A System for Ship Regulations Applying Domain Adaptation’, *Int. J. Nav. Archit. Ocean Eng.*, p. 100735, Dec. 2025, doi: 10.1016/j.ijnaoe.2025.100735.

[17] N. Ampazis, ‘Improving RAG Quality for Large Language Models with Topic-Enhanced Reranking’, in *Artificial Intelligence Applications and Innovations*, Springer, Cham, 2024, pp. 74–87. doi: 10.1007/978-3-031-63215-0_6.

[18] R. Jia, B. Zhang, S. J. R. Méndez, and P. G. Omran, 'StructRAG: Structure-Aware RAG Framework with Scholarly Knowledge Graph for Diverse Question Answering', in *Companion Proceedings of the ACM on Web Conference 2025*, in WWW '25. New York, NY, USA: Association for Computing Machinery, May 2025, pp. 2567–2573. doi: 10.1145/3701716.3717819.

[19] M. Rani, B. K. Mishra, D. Thakker, and M. N. Khan, 'To Enhance Graph-Based Retrieval-Augmented Generation (RAG) with Robust Retrieval Techniques', in *2024 18th International Conference on Open Source Systems and Technologies (ICOSST)*, Dec. 2024, pp. 1–6. doi: 10.1109/ICOSST64562.2024.10871140.

[20] S. Siriwardhana, R. Weerasekera, E. Wen, T. Kaluarachchi, R. Rana, and S. Nanayakkara, 'Improving the Domain Adaptation of Retrieval Augmented Generation (RAG) Models for Open Domain Question Answering', *Trans. Assoc. Comput. Linguist.*, vol. 11, pp. 1–17, Jan. 2023, doi: 10.1162/tacl_a_00530.

[21] K. Mao, Z. Liu, H. Qian, F. Mo, C. Deng, and Z. Dou, 'RAG-Studio: Towards In-Domain Adaptation of Retrieval Augmented Generation Through Self-Alignment', in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds., Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 725–735. doi: 10.18653/v1/2024.findings-emnlp.41.

[22] R. Xu *et al.*, 'SimRAG: Self-Improving Retrieval-Augmented Generation for Adapting Large Language Models to Specialized Domains', in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, L. Chiruzzo, A. Ritter, and L. Wang, Eds., Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 11534–11550. doi: 10.18653/v1/2025.naacl-long.575.

[23] D. Lee, J. Kim, J. Kim, S. Hwang, and J. Park, 'tRAG: Term-level Retrieval-Augmented Generation for Domain-Adaptive Retrieval', in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, L. Chiruzzo, A. Ritter, and L. Wang, Eds., Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 6566–6578. doi: 10.18653/v1/2025.naacl-long.334.

[24] A. Dholakia, D. Ellison, M. Hodak, D. Dutta, and C. Binnig, 'Benchmarking Generative AI Performance Requires a Holistic Approach', in *Performance Evaluation and Benchmarking*, R. Nambiar and M. Poess, Eds., Cham: Springer Nature Switzerland, 2024, pp. 34–43. doi: 10.1007/978-3-031-68031-1_3.

[25] H. Djeddal *et al.*, 'An Evaluation Framework for Attributed Information Retrieval using Large Language Models', in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, in CIKM '24. New York, NY, USA: Association for Computing Machinery, Oct. 2024, pp. 5354–5359. doi: 10.1145/3627673.3679172.

[26] H. Sánchez-Navalón, C. Monserrat, D. Garigliotti, and C. Ferri, 'Evaluating Performance and Trustworthiness of RAG Systems for Generating Administrative Text', in *Intelligent Data Engineering and Automated Learning – IDEAL 2024*, Springer, Cham, 2025, pp. 410–421. doi: 10.1007/978-3-031-77731-8_37.

[27] N. Hu *et al.*, 'Benchmarking Large Language Models in Complex Question Answering Attribution using Knowledge Graphs', Jan. 26, 2024, *arXiv*: arXiv:2401.14640. doi: 10.48550/arXiv.2401.14640.

[28] N. Arabzadeh and E. Bagheri, 'A self-supervised language model selection strategy for biomedical question answering', *J. Biomed. Inform.*, vol. 146, p. 104486, Oct. 2023, doi: 10.1016/j.jbi.2023.104486.

[29] H. Chen, Z. Fang, Y. Singla, and M. Dredze, 'Benchmarking Large Language Models on Answering and Explaining Challenging Medical Questions', Feb. 02, 2025, *arXiv*: arXiv:2402.18060. doi: 10.48550/arXiv.2402.18060.

[30] R. Dubey, A. Gunasekaran, and T. Papadopoulos, 'Benchmarking operations and supply chain management practices using Generative AI: Towards a theoretical framework', *Transp. Res. Part E Logist. Transp. Rev.*, vol. 189, p. 103689, Sep. 2024, doi: 10.1016/j.tre.2024.103689.

[31] B. Piryani, J. Mozafari, and A. Jatowt, 'ChroniclingAmericaQA: A Large-scale Question Answering Dataset based on Historical American Newspaper Pages', in *Proceedings of the 47th*

International ACM SIGIR Conference on Research and Development in Information Retrieval, in SIGIR '24. New York, NY, USA: Association for Computing Machinery, Jul. 2024, pp. 2038–2048. doi: 10.1145/3626772.3657891.

[32] I. Jahan, M. T. R. Laskar, C. Peng, and J. X. Huang, 'A comprehensive evaluation of large Language models on benchmark biomedical text processing tasks', *Comput. Biol. Med.*, vol. 171, p. 108189, Mar. 2024, doi: 10.1016/j.compbiomed.2024.108189.

[33] K. S. Kiangala and Z. Wang, 'An experimental hybrid customized AI and generative AI chatbot human machine interface to improve a factory troubleshooting downtime in the context of Industry 5.0', *Int. J. Adv. Manuf. Technol.*, vol. 132, no. 5, pp. 2715–2733, May 2024, doi: 10.1007/s00170-024-13492-0.

[34] I. Lamaakal *et al.*, 'Tiny Language Models for Automation and Control: Overview, Potential Applications, and Future Research Directions', *Sensors*, vol. 25, no. 5, Art. no. 5, Jan. 2025, doi: 10.3390/s25051318.

[35] Y.-C. Liu and W.-T. Chu, 'Chart Question Answering based on Modality Conversion and Large Language Models', in *Proceedings of the 1st ACM Workshop on AI-Powered Q&A Systems for Multimedia*, in AIQAM '24. New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 19–24. doi: 10.1145/3643479.3662057.

[36] S. Sinha and Y. M. Lee, 'Challenges with developing and deploying AI models and applications in industrial systems', *Discov. Artif. Intell.*, vol. 4, no. 1, p. 55, Aug. 2024, doi: 10.1007/s44163-024-00151-2.

[37] C. Trivedi *et al.*, 'Explainable AI for Industry 5.0: Vision, Architecture, and Potential Directions', *IEEE Open J. Ind. Appl.*, vol. 5, pp. 177–208, 2024, doi: 10.1109/OJIA.2024.3399057.

[38] S. Wu *et al.*, 'Benchmarking Large Language Models for Conversational Question Answering in Multi-instructional Documents', Oct. 01, 2024, *arXiv*: arXiv:2410.00526. doi: 10.48550/arXiv.2410.00526.

[39] H. Zhang, R. Zhang, J. Guo, M. de Rijke, Y. Fan, and X. Cheng, 'Are Large Language Models Good at Utility Judgments?', in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, in SIGIR '24. New York, NY, USA: Association for Computing Machinery, Jul. 2024, pp. 1941–1951. doi: 10.1145/3626772.3657784.

[40] R. Saha Roy, J. Schlotthauer, C. Hinze, A. Foltyn, L. Hahn, and F. Kuech, 'Evidence Contextualization and Counterfactual Attribution for Conversational QA over Heterogeneous Data with RAG Systems', presented at the WSDM 2025 - Proceedings of the 18th ACM International Conference on Web Search and Data Mining, 2025, pp. 1040–1043. doi: 10.1145/3701551.3704126.

[41] C. Jeong, 'A Study on the Implementation of Generative AI Services Using an Enterprise Data-Based LLM Application Architecture', *Adv. Artif. Intell. Mach. Learn.*, vol. 03, no. 04, pp. 1588–1618, 2023, doi: 10.54364/AAIML.2023.1191.

[42] H. A. Alawwad, A. Alhothali, U. Naseem, A. Alkhathlan, and A. Jamal, 'Enhancing textual textbook question answering with large language models and retrieval augmented generation', *Pattern Recognit.*, vol. 162, p. 111332, Jun. 2025, doi: 10.1016/j.patcog.2024.111332.

[43] K. Kharitonova, D. Pérez-Fernández, J. Gutiérrez-Hernando, A. Gutiérrez-Fandiño, Z. Callejas, and D. Griol, 'Leveraging Retrieval-Augmented Generation for Reliable Medical Question Answering Using Large Language Models', in *Hybrid Artificial Intelligent Systems*, Springer, Cham, 2025, pp. 141–153. doi: 10.1007/978-3-031-74186-9_12.

[44] Y. Lee, 'Developing a computer-based tutor utilizing Generative Artificial Intelligence (GAI) and Retrieval-Augmented Generation (RAG)', *Educ. Inf. Technol.*, pp. 1–22, Nov. 2024, doi: 10.1007/s10639-024-13129-5.

[45] X. Li and C. Jiang, 'Leveraging Prompt Tuning-Based Cognitive Attention to Enhance Logical Inference in Large Language Models', in *Proceedings of the First ACM International Workshop on Resource-efficient Mobile and Embedded LLM System in AIoT*, in RMEL '24. New York, NY, USA: Association for Computing Machinery, Nov. 2024, pp. 6–12. doi: 10.1145/3698383.3699622.

[46] C. Njeh, H. Nakouri, and F. Jaafar, 'Enhancing RAG-Retrieval to Improve LLMs Robustness and Resilience to Hallucinations', in *Hybrid Artificial Intelligent Systems*, Springer, Cham, 2025, pp. 201–213. doi: 10.1007/978-3-031-74186-9_17.

[47] Y. Ojima *et al.*, ‘Knowledge Management for Automobile Failure Analysis Using Graph RAG’, in *2024 IEEE International Conference on Big Data (BigData)*, Dec. 2024, pp. 6624–6631. doi: 10.1109/BigData62323.2024.10826046.

[48] S. Rajapaksha, R. Rani, and E. Karafili, ‘A RAG-Based Question-Answering Solution for Cyber-Attack Investigation and Attribution’, in *Computer Security. ESORICS 2024 International Workshops*, Springer, Cham, 2025, pp. 238–256. doi: 10.1007/978-3-031-82362-6_15.

[49] S. Packowski, I. Halilovic, J. Schlotfeldt, and T. Smith, ‘Optimizing and Evaluating Enterprise Retrieval-Augmented Generation (RAG): A Content Design Perspective’, presented at the ICAAI 2024 - Conference Proceedings of the 2024 8th International Conference on Advances in Artificial Intelligence, 2025, pp. 162–167. doi: 10.1145/3704137.3704181.

[50] B. V. Balu *et al.*, ‘Towards Automated Safety Requirements Derivation Using Agent-based RAG’, Apr. 15, 2025, *arXiv*: arXiv:2504.11243. doi: 10.48550/arXiv.2504.11243.

[51] M. M. Hussien, A. N. Melo, A. L. Ballardini, C. S. Maldonado, R. Izquierdo, and M. Á. Sotelo, ‘RAG-based explainable prediction of road users behaviors for automated driving using knowledge graphs and large language models’, *Expert Syst. Appl.*, vol. 265, p. 125914, Mar. 2025, doi: 10.1016/j.eswa.2024.125914.

[52] M. Dimitrova, ‘Retrieval-Augmented Generation (RAG): Advances and Challenges’, *Probl. Eng. Cybern. Robot.*, vol. 83, Jul. 2025, doi: 10.7546/PECR.83.25.03.

[53] B. Li, Z. Mao, R. Yan, A. Ling, Q. Hu, and Q. Zeng, ‘Knowledge Graph Completion using RAG and Improved Structural Information’, in *2025 IEEE 2nd International Conference on Electronics, Communications and Intelligent Science (ECIS)*, May 2025, pp. 1–5. doi: 10.1109/ECIS65594.2025.11087034.

[54] P. Pichappan, ‘Adaptive and Enhanced Retrieval Augmented Generation (RAG) Systems: A Summarised Survey’, *J. Digit. Inf. Manag.*, vol. 23, no. 3, pp. 190–195, Sep. 2025, doi: 10.6025/jdim/2025/23/3/190-195.



Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

9 APPENDIX

This section represents the tables those are almost identical in the Results and Analysis section (Section 5), but we compared these tables with other tables in Results and Analysis section. We also keep a big table (Table 9.5) in this section.

	Experiment 1 (Top-20)	**Experiment 2 (Top-20, query expanded)**	**Experiment 3 (Top-20, query expanded, cross-encoder reranking)**
Total Queries Evaluated	130	130	130
Found in Top-20	84	86	86
Not Found in Top-20	46	44	44
Retrieval Accuracy	64.62%	66.15%	66.15%
Match Position Breakdown	Matched index 1.0 24 2.0 14 3.0 12 4.0 7 5.0 8 6.0 4 7.0 8 8.0 2 9.0 1 10.0 1 12.0 1 13.0 2	Matched index 1.0 77 2.0 4 3.0 3 4.0 1 8.0 1	Matched index 1.0 86
Average Time	0.007 seconds	0.008 seconds	0.309 seconds

Table 9.1: Top-20 retrieval evaluation with query expansion and reranking (Sub-section 5.2.1)

	Experiment 1 (Top-20, query expanded)	**Experiment 2 (Top-20, query expanded, cross-encoder reranking)**
Total Queries Evaluated	130	130
Found in Top-20	85	85
Not Found in Top-20	45	45
Retrieval Accuracy	65.38%	65.38%
Match Position Breakdown	Matched index 1.0 71 2.0 4 3.0 4 8.0 1 9.0 1 14.0 1 15.0 1 18.0 2	Matched index 1.0 85
Average Time	0.009 seconds	0.336 seconds

Table 9.2: Top-20 retrieval evaluation with more query expansion and reranking (Sub-section 5.2.2)

	Experiment 1 (Top-20, query expanded)	Experiment 2 (Top-20, query expanded, cross-encoder reranking)
Total Queries Evaluated	130	130
Found in Top-20	81	81
Not Found in Top-20	49	49
Retrieval Accuracy	62.31%	62.31%
Match Position Breakdown	Matched index 1.0 36 2.0 19 3.0 7 4.0 2 5.0 3 6.0 3 7.0 4 8.0 1 9.0 3 10.0 1 11.0 1 14.0 1	Matched index 1.0 81
Average Time	0.010 seconds	0.308 seconds

Table 9.3: Top-20 retrieval evaluation with more query expansion and reranking (Sub-section 5.2.3)

	Experiment 1 (Top-10)	Experiment 2 (Top-10, query expanded)	Experiment 3 (Top-10, query expanded, cross-encoder reranking)
Total Queries Evaluated	500	500	500
Found in Top-10	386	486	486
Not Found in Top-10	114	14	14
Retrieval Accuracy	77.20%	97.20%	97.20%
Match Position Breakdown	Matched index 1.0 240 2.0 39 3.0 41 4.0 16 5.0 20 6.0 8 7.0 7 8.0 2 9.0 7 10.0 6	Matched index 1.0 471 2.0 9 3.0 3 5.0 1 6.0 1 7.0 1	Matched index 1.0 482 2.0 2 3.0 1 6.0 1
Average Time	0.011 seconds	0.014 seconds	0.233 seconds

Table 9.4: Top-10 retrieval evaluation with query expansion and reranking (Sub-section 5.3)

group1	group2	meandiff	p-adj	lower	upper	reject	Metric
Plain	Structured RAG	0.1911	0	0.1298	0.2523	TRUE	Cosine
Plain	Structured Reranked RAG	0.1691	0	0.1079	0.2303	TRUE	Cosine
Structured RAG	Structured Reranked RAG	-0.0219	0.6735	-0.0832	0.0393	FALSE	Cosine
Plain	Structured RAG	0.0444	0	0.0241	0.0647	TRUE	ROUGE_L
Plain	Structured Reranked RAG	0.0026	0.9523	-0.0178	0.0229	FALSE	ROUGE_L
Structured RAG	Structured Reranked RAG	-0.0419	0	-0.0622	-0.0216	TRUE	ROUGE_L
Plain	Structured RAG	0.0168	0	0.0099	0.0237	TRUE	BLEU
Plain	Structured Reranked RAG	0.0048	0.238	-0.0022	0.0117	FALSE	BLEU
Structured RAG	Structured Reranked RAG	-0.012	0.0002	-0.019	-0.0051	TRUE	BLEU
Plain	Structured RAG	0.0584	0	0.0355	0.0813	TRUE	Jaccard
Plain	Structured Reranked RAG	0.0164	0.2095	-0.0065	0.0393	FALSE	Jaccard
Structured RAG	Structured Reranked RAG	-0.042	0.0001	-0.0649	-0.0191	TRUE	Jaccard
Plain	Structured RAG	0.0166	0	0.0088	0.0244	TRUE	BERTScore_P
Plain	Structured Reranked RAG	0.0071	0.0838	-0.0007	0.0148	FALSE	BERTScore_P
Structured RAG	Structured Reranked RAG	-0.0095	0.0119	-0.0173	-0.0018	TRUE	BERTScore_P
Plain	Structured RAG	0.0264	0	0.0183	0.0346	TRUE	BERTScore_R
Plain	Structured Reranked RAG	0.0183	0	0.0101	0.0264	TRUE	BERTScore_R
Structured RAG	Structured Reranked RAG	-0.0081	0.0506	-0.0163	0	FALSE	BERTScore_R
Plain	Structured RAG	0.0212	0	0.0142	0.0282	TRUE	BERTScore_F1
Plain	Structured Reranked RAG	0.0124	0.0002	0.0053	0.0194	TRUE	BERTScore_F1

Structured RAG	Structured Reranked RAG	-0.0088	0.0096	-0.0158	-0.0018	TRUE	BERTScore_F1

Table 9.5: Tukey's HSD post-hoc test results across Plain OpenAI, Structured RAG-based OpenAI, Reranked Structured RAG-based OpenAI (Sub-sections 5.6.4)